

Estructuras de SO's

1. SOPORTE HARDWARE (HW) AL SISTEMA OPERATIVO

1.1 ELEMENTOS BÁSICOS

Existen cuatro elementos estructurales principales:

- **Procesador.** Controla el funcionamiento del computador y realiza sus funciones de procesamiento de datos. Cuando sólo hay uno, se denomina **unidad central de proceso** (*Central Processing Unit, CPU*).
- **Memoria Principal.** Almacena datos y programas. Esta memoria es habitualmente volátil; es decir, cuando se apaga el computador, se pierde su contenido. En contraste, el contenido de la memoria del disco se mantiene. También se le denomina *memoria real* o *primaria*.
- **Módulos de E/S.** Transfieren los datos entre el computador y su entorno externo. El entorno externo está formado por diversos dispositivos, incluyendo dispositivos de memoria secundarias, equipos de comunicaciones y terminales.
- **Bus del sistema.** Proporciona comunicación entre los procesadores, la memoria principal y los módulos de E/S.

Una de las funciones del procesador es el intercambio de datos con la memoria. Para ello, se utilizan normalmente dos registros internos: un **registro de dirección de memoria (MAR)**, que especifica la dirección de memoria de la siguiente lectura o escritura; y un **registro de datos de memoria (MBR)**, que contiene los datos que se van a escribir o que recibe los datos leídos. De manera similar existe un **registro de direcciones de E/S (I/O AR)**, y un **buffer de E/S (I/O BR)**.

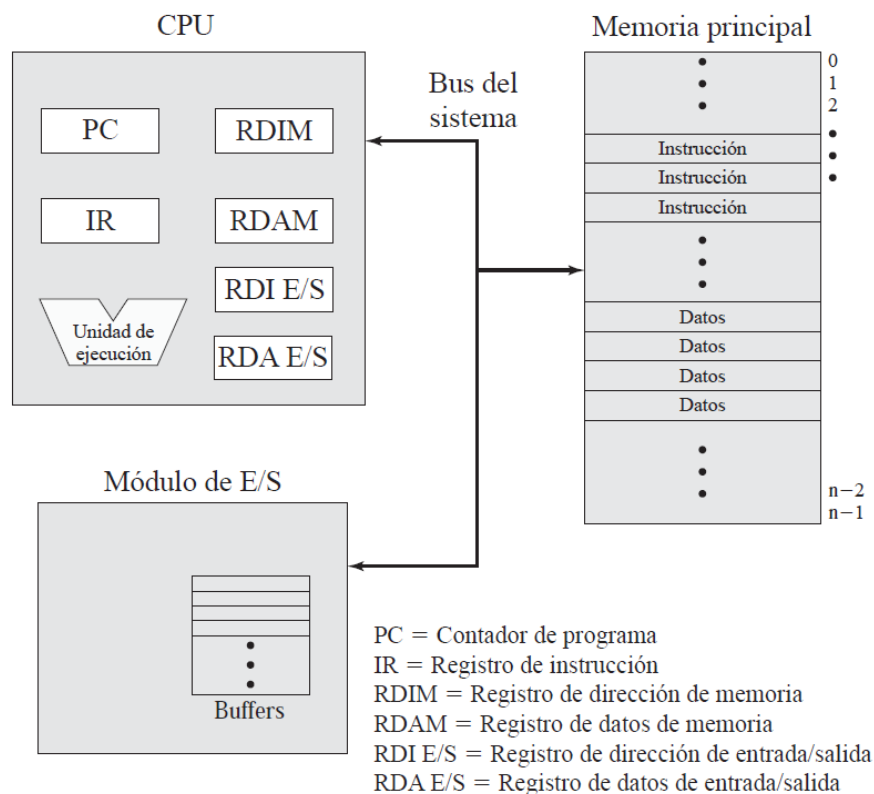


Figura 1.1. Componentes de un computador: visión al más alto nivel

1.2 REGISTROS DEL PROCESADOR

Un procesador incluye un conjunto de registros que proporcionan un tipo de memoria que es más rápida y de menor capacidad que la principal. Los registros del procesador sirven para dos funciones:

- **Registros visibles para el usuario.** Permite al programador minimizar las referencias a memoria principal optimizando el uso de los registros.
- **Registros de control y estado.** Usados por el procesador para controlar su operación y por rutinas privilegiadas del sistema operativo para controlar la ejecución de programas.

No existe una clasificación nítida entre estas dos categorías, pero es conveniente utilizarlas.

1.2.1 REGISTROS VISIBLES PARA EL USUARIO

Se puede acceder a ellos mediante el lenguaje de máquina ejecutado por el procesador, generalmente disponible para todos los programas. Los tipos de registros normalmente disponibles son registros de datos, de dirección y de códigos de condición.

Los **registros de datos** se utilizan para diversas funciones. En algunos casos son de propósito general y pueden usarse con cualquier instrucción de máquina que realice operaciones sobre datos.

Los **registros de dirección** contienen direcciones de memoria principal de datos e instrucciones. Estos pueden ser de propósito general o pueden estar dedicados a una forma particular de direccionamiento de memoria.

- **Registro índice.** Modo de direccionamiento que implica sumar un índice a un valor base.
- **Puntero de segmento.** La memoria se divide en segmentos, bloques de palabras¹ de longitud variable. Una referencia de memoria consta de una referencia a un determinado segmento y un desplazamiento dentro del segmento.
- **Puntero de pila.** Si hay direccionamiento de pila² visible, hay un registro dedicado que apunta a la cima de la pila.

1.2.2 REGISTROS DE CONTROL Y ESTADO

Se emplean varios registros del procesador para controlar el funcionamiento del mismo. En la mayoría de máquinas, muchos de ellos no son visibles para el usuario. A algunos se pueden acceder mediante instrucciones de máquina ejecutadas en lo que se denomina modo de control o de sistema operativo. Además de los registros mencionados anteriormente, los siguientes son esenciales para la ejecución de instrucciones:

- **Contador de programa (PC).** Contiene la dirección de la próxima instrucción que se leerá de la memoria.
- **Registro de instrucción (IR).** Contiene la última instrucción leída.
- **Program Status Word (PSW).** Contiene códigos de condición, información para interrupciones y modo de ejecución del procesador.

Los **códigos de condición** (indicadores) son bits cuyo valor lo asigna el hardware de procesador teniendo en cuenta el resultado de las operaciones. Además de almacenarse el resultado, se fija un código de condición en concordancia con el resultado de la ejecución de la instrucción. Posteriormente, se puede comprobar el código de condición como parte de una operación de salto condicional. Los bits de código de condición se agrupan en uno o más registros. Normalmente, forman parte de un registro de control.

1.3 MEMORIA PRINCIPAL (RAM)

La **memoria principal (RAM)** es un modelo abstracto que se representa como una tabla o array lineal compuesto por un número de elementos (**celdas/palabras de memoria**) con un tamaño. Las palabras son direccionables con números naturales desde el 0, cada número es la **dirección de memoria** (*memory address*) de la correspondiente celda de memoria. El espacio de direcciones (*address space*) es el conjunto de números que representa las direcciones de una memoria. Las operaciones que realiza la memoria principal son las siguientes:

- **Lectura (R).** La memoria recibe una dirección y devuelve el byte contenido en la celda identificada por dicha dirección.
- **Escritura (W).** La memoria recibe una dirección y un byte y sobrescribe el byte contenido en la celda indicada por la dirección.

Las direcciones se codifican en **base 2**, al igual que su contenido. Esto influye directamente en el tamaño; si utilizamos n bits para codificar las direcciones, el espacio de direcciones tiene una cardinalidad de **2^n direcciones, $[0, 2^n - 1]$** . Por lo tanto, el tamaño de la memoria es 2^n multiplicado por el tamaño de la palabra.

En IA-32 los programas no acceden directamente a la memoria física sino indirectamente utilizando los modelos de memoria.

- **Flat memory model.** Espacio lineal con direcciones consecutivas en el rango $[0, 2^n - 1]$ (*linear address space*), permite direccionar con granularidad de un byte.
- **Segmented memory model.** Los programas ven el espacio de memoria como un grupo de espacios independientes llamados segmentos. Para acceder a un byte de un segmento el programa utiliza una dirección bidimensional.
- **Real-address mode memory model.** Implementación de segmentación con limitaciones en el tamaño de los segmentos, 64KB, y en el espacio de memoria final accesible, 2^{20} bytes.

¹ Conjunto ordenado de bytes o bits, transmite u opera la información dentro de un determinado computador.

² Una pila se almacena en memoria principal y es un conjunto secuencial de posiciones a las que se hace referencia de manera similar a como ocurre con una pila física de papeles, insertando y extrayendo de la cima.

1.4 EJECUCIÓN DE INSTRUCCIONES

Se denomina **ciclo de instrucción** al procesamiento requerido por una única instrucción. Consta de dos pasos denominados **fase de búsqueda** y de **ejecución**. La ejecución del programa se detiene únicamente si se apaga la máquina, se produce algún tipo de error irrecuperable o se ejecuta una instrucción del programa que pare al procesador.

Al principio de cada ciclo de instrucción, el procesador lee una instrucción de la memoria. A menos que se indique otra cosa, el procesador siempre incrementa el PC después de cada instrucción, de manera que se leerán en orden secuencial.

La instrucción leída se carga dentro del registro de instrucción (IR), esta contiene bits que especifican la acción que debe realizar el procesador. Se interpreta la instrucción y se lleva a cabo la acción requerida. En general, estas acciones se dividen en cuatro categorías:

- **Procesador-memoria.** Transferir datos del procesador a la memoria o viceversa.
- **Procesador-E/S.** Enviar datos a un dispositivo periférico o recibirlos desde el mismo, transfiriéndolos entre el procesador y un módulo de E/S.
- **Procesamiento de datos.** El procesador puede realizar algunas operaciones aritméticas o lógicas sobre los datos.
- **Control.** Una instrucción puede especificar que se va a alterar la secuencia de ejecución.

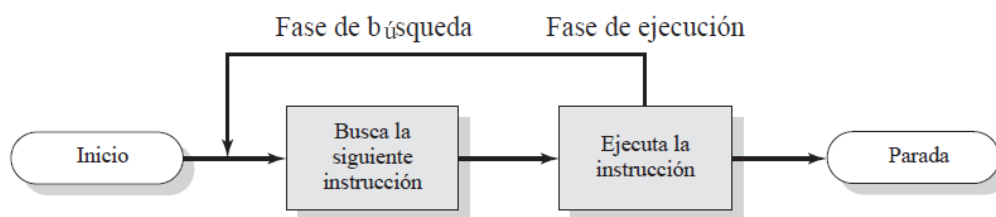


Figura 1.2. Ciclo de instrucción básico

1.5 MÓDULO DE E/S

Se pueden intercambiar datos directamente entre un módulo de E/S y el procesador. Al igual que el procesador puede iniciar una lectura/escritura en memoria, también puede hacerlo en un módulo E/S. En este caso, el procesador identifica un dispositivo específico que está controlado por un determinado módulo de E/S. Las funciones de un módulo de E/S son:

- **Control y temporización.**
- **Comunicación con el procesador y con los dispositivos.**
- **Almacenamiento temporal de datos (buffers).** La velocidad de transferencia entre los dispositivos y el módulo E/S es mucho menor.
- **Detección de errores.**

1.6 INTERRUPCIONES

Las interrupciones constituyen una manera de mejorar la utilización del procesador. Prácticamente todos los computadores proporcionan un mecanismo por el cual otros módulos pueden interrumpir el secuenciamiento normal del procesador. La tabla 1.1 detalla los tipos más comunes de interrupciones.

Tabla 1.1. Clases de interrupciones.

De programa	Generada por alguna condición que se produce como resultado de la ejecución de una instrucción, tales como un desbordamiento aritmético, una división por cero, un intento de ejecutar una instrucción de máquina ilegal, y las referencias fuera del espacio de la memoria permitido para un usuario.
Por temporizador	Generada por un temporizador del procesador, Permite al sistema operativo realizar ciertas funciones de forma regular.
De E/S	Generada por un controlador de E/S para señalar la conclusión normal de una operación o para indicar diversas condiciones de error.
Por fallo del hardware	Generada por un fallo, como un fallo en el suministro de energía o un error de paridad en la memoria.

1.6.1 INTERRUPCIONES Y EL CICLO DE INSTRUCCIÓN

Cuando el dispositivo externo esté listo para ser atendido, el módulo de E/S de este dispositivo manda una señal de petición de interrupción al procesador. El procesador responde suspendiendo la ejecución del programa actual, saltando a la rutina de servicio específica del dispositivo, conocida como **manejador de interrupción**, y reanudando la ejecución original después de haber atendido al dispositivo.

El programa de usuario no tiene que contener ningún código especial para tratar interrupciones; el procesador y el sistema operativo son los responsables de suspender el programa de usuario y de reanudarlo en el mismo punto.

Para tratar las interrupciones, se añade una fase de interrupción al ciclo de instrucción. En esta fase, el procesador comprueba si se ha ocurrido cualquier interrupción, si no hay ninguna pendiente continúa con la fase de búsqueda. Si existe una interrupción, se suspende el programa actual y ejecuta la rutina del manejador de interrupción, la cual es parte del sistema operativo.

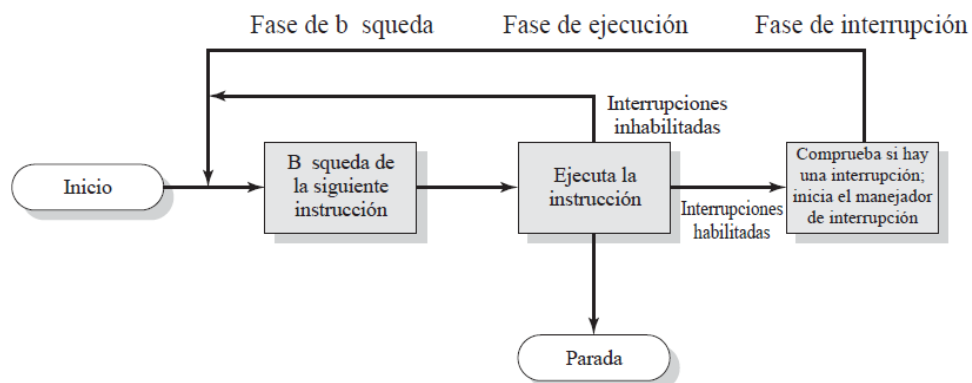


Figura 1.3. Ciclo de instrucción con interrupciones

1.6.2 PROCESAMIENTO DE INTERRUPCIONES

La aparición de una interrupción dispara varios eventos, tanto en el hardware del procesador como el software. Cuando un dispositivo de E/S completa una operación de E/S, se produce la siguiente secuencia de eventos en el hardware:

1. El dispositivo genera una señal de interrupción hacia el procesador.
2. El procesador termina la ejecución de la instrucción actual antes de responder.
3. El procesador comprueba si hay una petición de interrupción pendiente, determina que hay una y manda una señal de reconocimiento al dispositivo que la produjo, lo cual permite que el dispositivo elimine su señal de interrupción.
4. El procesador salva la información requerida para reanudar el programa actual en el momento de la interrupción. La información mínima requerida es la PSW y la posición de la siguiente instrucción que se va a ejecutar.
5. El procesador carga el PC con la posición del punto de entrada de la rutina de manejo de interrupción que responderá a esta interrupción.

Una vez que se ha cargado el PC, el procesador continúa con el siguiente ciclo de instrucción, que comienza con una lectura de instrucción. Se transfiere el control del programa al manejador de interrupción. La ejecución de este programa conlleva las siguientes operaciones:

6. El PC y la PSW se han almacenado en la pila del sistema. Sin embargo, hay otra información que se considera parte del estado del programa en ejecución. Se necesita salvar el contenido de los registros del procesador. Así como cualquier información de estado.
7. Se puede empezar a procesar la interrupción.
8. Cuando se completa el procesamiento de la interrupción, se recuperan los valores de los registros salvados en la pila y se restituyen en los registros.
9. Restituir de la pila los valores de la PSW y el PC. La siguiente instrucción que se va a ejecutar corresponderá al programa previamente interrumpido.

1.6.3 MÚLTIPLES INTERRUPCIONES

Se pueden considerar dos alternativas a la hora de tratar con múltiples interrupciones. La primera es inhabilitar las interrupciones mientras se esté procesando una. Una **interrupción inhabilitada** significa que el procesador ignorará cualquier nueva señal de petición de interrupción. Si se produce una interrupción durante este tiempo, quedará pendiente de ser procesada, el procesador solo la comprobará después de que se reabiliten las interrupciones. Las interrupciones se manejan en estricto orden secuencial. La desventaja es que no se tienen en cuenta la prioridad relativa o el grado de urgencia de las interrupciones.

Una segunda estrategia es **definir prioridades** para las interrupciones y permitir que una interrupción de más prioridad cause que se interrumpa la ejecución de un manejador de una interrupción de menor prioridad.

1.7 TÉCNICAS DE COMUNICACIÓN DE E/S

1.7.1 E/S PROGRAMADA

El procesador debe determinar cuándo se completa la instrucción de E/S. Para ello, comprueba periódicamente el estado del módulo de E/S hasta que se encuentra que se ha completado la operación. El procesador es responsable de extraer los datos de la memoria principal en una operación de salida y de almacenarlos en ella en una operación de entrada. El juego de instrucciones incluye instrucciones de E/S de las siguientes categorías:

- **Control.** Utilizadas para activar un dispositivo externo y especificarle qué debe hacer.
- **Estado.** Utilizadas para comprobar diversas condiciones de estado asociadas a un módulo de E/S y sus periféricos.
- **Transferencia.** Utilizadas para leer y/o escribir datos entre los registros del procesador y los dispositivos externos.

1.7.2 E/S DIRIGIDA POR INTERRUPCIONES

Una alternativa es que el procesador genera un mandato de E/S para un módulo y continúa realizando algún otro trabajo útil. El módulo de E/S interrumpirá más tarde al procesador para solicitar su servicio cuando esté listo. El procesador ejecutará la transferencia de datos y reanudará el procesamiento previo.

1.7.3 ACCESO DIRECTO A MEMORIA

Cuando se van a transferir grandes volúmenes de datos, se requiere de una técnica más eficiente: el **acceso directo a memoria (Direct Memory Access, DMA)**. Cuando el procesador desea leer o escribir un bloque de datos, genera un mandato al módulo de DMA, enviándole la siguiente información:

- Si se trata de una lectura o una escritura.
- La dirección del dispositivo de E/S involucrado.
- La posición inicial de memoria en la que se desea leer los datos o donde se quieren escribir.
- El número de palabras que se pretenden leer o escribir.

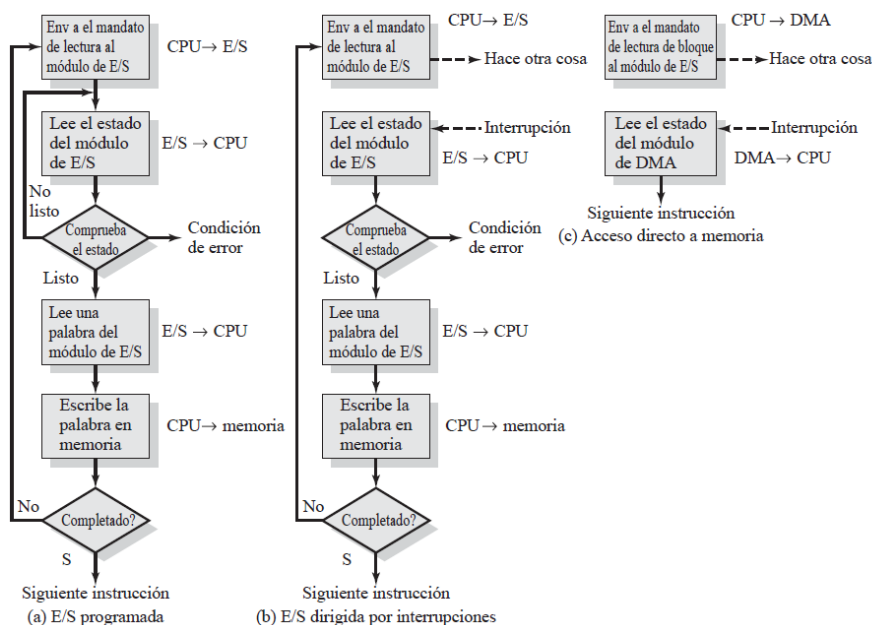


Figura 1.4. Tres técnicas para leer un bloque de datos

2. ESTRUCTURA DEL SISTEMA OPERATIVO

2.1 OBJETIVOS DE LOS SISTEMAS OPERATIVOS

Un sistema operativo es un programa que controla la ejecución de aplicaciones y programas y que actúa como interfaz entre las aplicaciones y el hardware del computador. Se puede considerar que un sistema operativo tiene los siguientes objetivos:

- **Facilidad de uso.** Facilita el uso de un computador.
- **Eficiencia.** Permite que los recursos de un sistema de computación se puedan utilizar de una manera eficiente.
- **Capacidad para evolucionar.** Se debe construir de tal forma que puedan desarrollar, probar e introducir nuevas funciones en el sistema sin interferir con su servicio.

2.1.1 EL SISTEMA OPERATIVO COMO UNA INTERFAZ DE USUARIO/COMPUTADOR

El sistema operativo oculta los detalles del hardware al programador y le proporciona una interfaz apropiada para utilizar el sistema. Actúa como mediador, facilitando al programador y a la aplicación el acceso y uso de dichas utilidades y servicios. El sistema operativo proporciona normalmente servicios en las siguientes áreas:

- **Desarrollo de programas.** Proporciona una variedad de utilidades y servicios para asistir al programador en la creación de programas.
- **Ejecución de programas.** Las instrucciones y datos se deben cargar en memoria principal. Los dispositivos de E/S y los ficheros se deben inicializar, y otros recursos deben prepararse.
- **Acceso a dispositivos de E/S.** Cada dispositivo de E/S requiere su propio conjunto peculiar de instrucciones o señales de control para cada operación. El SO proporciona una interfaz uniforme que esconde esos detalles de forma que los programadores puedan acceder a dichos dispositivos utilizando lecturas y escrituras sencillas.
- **Acceso controlado a los ficheros.** Debe reflejar una compresión detallada no sólo de la naturaleza del dispositivo de E/S sino también una estructura de los datos contenidos en los ficheros del sistema de almacenamientos. Adicionalmente, en el caso de un sistema con múltiples usuarios, puede proporcionar mecanismos de protección.
- **Acceso al sistema.** Controla el acceso al sistema completo y a recursos del sistema específicos. Debe proporcionar protección a los recursos y a los datos.
- **Detección y respuesta a errores.** El SO debe proporcionar una respuesta que elimine la condición de error, suponiendo el menor impacto en las aplicaciones en ejecución.
- **Contabilidad.** Recogerá estadísticas de uso de los diferentes recursos y monitorizará parámetros de rendimiento tales como el tiempo de respuesta.

2.1.2 FACILIDAD DE EVOLUCIÓN DE UN SISTEMA OPERATIVO

Un sistema operativo importante debe evolucionar en el tiempo por las siguientes razones:

- **Actualizaciones de hardware más nuevos tipos de hardware.**
- **Nuevos servicios.** En respuesta a la demanda del usuario o a las necesidades de los gestores de sistema.
- **Resolución de fallos.** Estos fallos se descubren con el transcurso del tiempo y se resuelven.

La necesidad de cambiar regularmente un SO introduce ciertos requisitos en su diseño. El SO debe tener un diseño modular, con interfaces entre los módulos claramente definidas, y que debe estar bien documentado.

2.2 FUNCIONALIDAD DE LOS SISTEMAS OPERATIVOS

2.2.1 FUNCIONALIDAD PARA PROCESOS

Proceso. Concepto que permite monitorizar y controlar sistemáticamente la ejecución de varios programas en el procesador. Un proceso debe tener asociados dos elementos: el **programa a ejecutar** y la información para supervisión y control de la ejecución del programa **PCB** (*Process Control Block*). La funcionalidad para procesos consiste en:

- **Creación** del PCB asociado a un programa que va a ejecutarse. **Eliminación** del PCB una vez finaliza la ejecución.
- **Bloqueo** (*sleep*) y **desbloqueo** (*wake up*) de los procesos dependiendo de los eventos por los que debe esperar un programa para continuar su ejecución.
- Proporcionar mecanismos que posibiliten la **comunicación** y **sincronización** entre procesos.

2.2.2 FUNCIONALIDAD PARA MEMORIA

- **Protección** de la región de memoria principal ocupada por el kernel y de las regiones de memoria principal ocupadas por los programas.
- **Compartición** de parte de las regiones ocupadas por los programas para permitir comunicación entre estos.
- Gestión automática de la **asignación/liberación** de la memoria disponible para un programa en cualquier nivel de la **jerarquía de memoria** y de forma transparente al programador.
- Mantener información sobre la **memoria asignada** a los procesos, núcleo, etc. Y la **memoria libre** en cualquier nivel de la jerarquía.
- Implementar algoritmos para decidir cuánta memoria asignar a cada proceso y cuando debe liberarse toda la memoria asociada a un proceso manteniéndolo en almacenamiento secundario.

2.2.3 FUNCIONALIDAD PARA ARCHIVOS Y DIRECTORIOS

Archivo. Colección de información identificada por nombre que reside en almacenamiento secundario. La funcionalidad relacionada con los archivos está asociada al sistema de archivos:

- Crear, eliminar, copiar, renombrar... archivos y directorios.
- Mantener los **atributos de archivo** en una estructura de datos (**inode** en UNIX/Linux).
- Abrir y cerrar sesiones de trabajo con archivos, leer, escribir, variar el puntero de lectura/escritura.
- Gestionar la asignación y liberación de espacio en disco para mantener la información de los archivos y directorios.
- Mantener la ubicación en disco de los datos asociados a archivos y directorios, así como los **metadatos del Sistema de Archivos**.

2.2.4 FUNCIONALIDAD PARA DISPOSITIVOS DE E/S

- Controlar el funcionamiento de los dispositivos de E/S.
- Proteger su utilización de forma directa por parte de los programas de aplicación: acceso mediante API de llamadas al sistema.
- Proporcionar una interfaz independiente de las particularidades de los dispositivos al resto del SO y a los programadores: interfaz estándar para todos los dispositivos.
- Manejador de dispositivo (**Device Driver**). Modulo software que gestiona un determinado dispositivo y se encarga de implementar las operaciones del interfaz, el manejo de interrupciones, etc.

2.2.5 FUNCIONALIDAD PARA RECURSOS DEL SO

PLANIFICACIÓN Y GESTIÓN DE RECURSOS

- **Equitatividad.** El acceso debe estar garantizado para todos los procesos.
- **Respuesta diferencial.** El SO debe planificar la asignación de los recursos teniendo en cuenta características diferenciales de los clientes, proceso u otras entidades.
- **Eficiencia.** El SO debe maximizar la productividad, minimizar el tiempo de respuesta y, en sistemas de tiempo compartido, permitir el trabajo simultáneo de tantos usuarios como sea posible.

PROTECCIÓN Y SEGURIDAD

- **Protección.** Cualquier mecanismo para controlar el acceso de procesos o usuarios a recursos del SO.
- **Seguridad.** Defensa del sistema frente a ataques internos o externos.
- Los sistemas distinguen usuarios para determinar quién puede hacer que. UID y GUI se asocian con procesos y archivos para determinar el control de acceso.

2.2.6 COMPONENTES GENERALES DE UN SO

Las funcionalidades descritas previamente se suelen agrupar en los siguientes componentes fundamentales.

- Gestor de procesos.
- Gestor de memoria.
- Gestor de archivos.
- Gestor de E/S.
- Gestor de comunicaciones.

2.3 INTERFACES ENTRE USUARIO Y SO

- **Command Line Interface (CLI).** A veces implementado en el kernel y la mayoría mediante shells. Los *shells* permiten ejecutar órdenes *built-in* (implementadas en el propio programa *shell*) u órdenes que residen en el FS.
- **Graphic User Interface (GUI).** Interfaz que utiliza la metáfora de escritorio (Desktop metaphor). Usa teclado/ratón.
- **System calls (Llamadas al sistema).** Interfaz de programación a los servicios proporcionados por el SO. Los programas lo utilizan mediante una API en lugar de acceso directo a la *system call*.

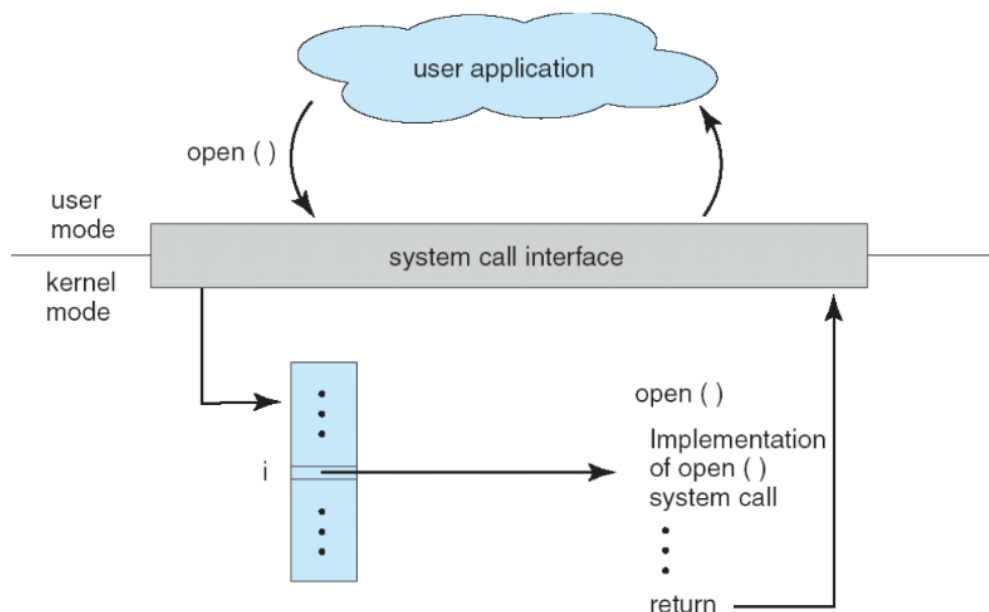


Figura 2.1. Componentes implicados en una system call.

2.4 LA EVOLUCIÓN DE LOS SISTEMAS OPERATIVOS

2.4.1 PROCESAMIENTO EN SERIE

Antes del surgimiento de los primeros sistemas operativos, se empleaba el denominado procesamiento en serie, los usuarios debían acceder a un computador en serie. El programador interactuaba directamente con el hardware del computador ya que no existía ningún sistema operativo. Estos sistemas iniciales presentaban dos **problemas**:

- **Planificación.** El tiempo era muy elevado lo que suponía un alto coste. Además, el usuario podía tener problemas si el programa no finalizaba en el tiempo asignado y era forzado a terminar antes de resolver el problema.
- **Tiempo de configuración.** Un único programa, denominado trabajo, podía implicar la carga en memoria del compilador y del programa en lenguaje de alto nivel y, a continuación, la carga y el enlace del programa objeto y las funciones comunes. De manera que se utilizaba una cantidad de tiempo considerable en configurar el programa que se iba a ejecutar.

2.4.2 SISTEMAS EN LOTES SENCILLOS

La idea central bajo el esquema de procesamiento en lotes sencillo es el uso de una pieza de software denominado **monitor**. El usuario no tiene que acceder directamente a la máquina. En su lugar, envía trabajo al operador del computador, que crea un sistema por lotes con todos los trabajos enviados y coloca la secuencia de trabajos en el dispositivo de entrada, para que lo utilice el monitor. Cuando un programa finaliza su procesamiento, devuelve el control al monitor. Para comprender cómo funciona este esquema, se puede analizar desde dos puntos de vista:

- **Punto de vista del monitor.** El monitor controla la secuencia de eventos. Para ello, una gran parte del monitor debe estar en memoria principal y disponible para la ejecución. El monitor lee de uno en uno los trabajos desde el dispositivo de entrada y coloca el trabajo actual en el área de programa de usuario y se le pasa el control. Cuando el trabajo se ha completado, devuelve el control al monitor.
- **Punto de vista del procesador.** En cierto punto, el procesador ejecuta instrucciones de la zona de memoria principal que contiene el monitor. Estas instrucciones provocan que se lea el siguiente trabajo y se almacene en otra zona de memoria principal. Una vez que el trabajo se ha leído, el procesador encontrará una instrucción de salto en el monitor que le indica al procesador que continúe la ejecución al inicio del programa usuario hasta que encuentra una condición de finalización o error.

El principal problema del procesamiento por lotes eran las largas esperas entre lotes de trabajo

2.4.3 SISTEMAS EN LOTES MULTIPROGRAMADOS

En los sistemas por lotes simples, el procesador se encuentra frecuentemente ocioso. El problema consiste en que los dispositivos de E/S son lentos comparados con el procesador. Debido a esto, la **multiprogramación** se impuso en los sistemas de lotes como una forma de aprovechar el tiempo empleado en las operaciones de E/S.

Mediante la multiprogramación, se empezó aprovechar el tiempo ya que se podía mantener varios trabajos simultáneamente en memoria principal y en realizar operaciones de E/S por acceso directo a memoria.

2.4.4 SISTEMAS DE TIEMPO COMPARTIDO

Del mismo modo que la multiprogramación permite al procesador gestionar múltiples trabajos en lotes en un determinado tiempo, también se puede usar para gestionar múltiples trabajos interactivos. En este último caso, la técnica se denomina **tiempo compartido**, porque se comparte el tiempo de procesador entre múltiples usuarios. En un sistema de tiempo compartido, múltiples usuarios acceden a la vez al sistema a través de terminales, siendo el SO el encargado de entrelazar la ejecución de cada programa de usuario en pequeños intervalos de tiempo o cuantos de computación.

Tabla 2.1. Multiprogramación en lotes frente a tiempo compartido.

	MULTIPROGRAMACIÓN EN LOTES	TIEMPO COMPARTIDO
Objetivo principal	Maximizar el uso del procesador	Minimizar el tiempo de respuesta
Fuente de directivas al sistema operativo	Mandatos del lenguaje de control de trabajos proporcionados por el trabajo	Mandatos introducidos al terminal

La compartición de tiempo y la multiprogramación implican nuevos problemas para el sistema operativo. Si existen múltiples trabajos en memoria, éstos deben protegerse para evitar que interfieran entre sí. Con los múltiples usuarios interactivos, el sistema de ficheros debe ser protegido, de forma que solo los usuarios autorizados tengan acceso a un fichero en particular. También deben gestionarse los conflictos entre los recursos, tal como impresoras y dispositivos de almacenamientos masivo.

2.5 ESTRUCTURA DEL SISTEMA

Para SO grandes no es suficiente la programación modular. La estructura jerárquica de un SO moderno separa sus funciones de acuerdo a las características de su escala de tiempo y nivel de abstracción. Se puede ver el sistema como una serie de niveles. Cada nivel realiza un subconjunto relacionado de funciones requeridas por el SO. Dicho nivel confía en los niveles inmediatamente inferiores. En general, las capas inferiores tratan con una escala de tiempo menor.

Tabla 2.4. Jerarquía de diseño del sistema operativo.

Nivel	Nombre	Objetos	Ejemplo de operaciones
13	Intérprete de mandatos	Entorno de programación de usuario	Sentencias en lenguaje del intérprete de mandatos
12	Procesos de usuario	Procesos de usuario	Salir, matar, suspender, continuar
11	Directorios	Directorios	Crear, destruir, insertar entrada, eliminar entrada, buscar, listar
10	Dispositivos	Dispositivos externos	Abrir, cerrar, leer, escribir
9	Sistema de ficheros	Ficheros	Crear, destruir, abrir, cerrar, leer, escribir
8	Comunicaciones	Tuberías	Crear, destruir, abrir, cerrar, leer, escribir
7	Memoria virtual	Segmentos, páginas	Leer, escribir, cargar
6	Almacenamiento secundario local	Bloques de datos, canales de dispositivo	Leer, escribir, asignar, liberar
5	Procesos primitivos	Procesos primitivos, semáforos, lista de procesos listos	Suspender, continuar, esperar, señalizar
4	Interrupciones	Programas de gestión de interrupciones	Invocar, enmascarar, desenmascarar, reintentar
3	Procedimientos	Procedimientos, pila de llamadas, registro de activación	Marcar la pila, llamar, retomar
2	Conjunto de instrucciones	Pila de evaluación, intérprete de microprogramas, datos escalares y vectoriales	Cargar, almacenar, sumar, restar, saltar
1	Circuitos electrónicos	Registros, puertas, buses, etc.	Poner a 0, transferir, activar, complementar

El área sombreada en gris representa al hardware

2.6 DESARROLLOS QUE HAN LLEVADO A LOS SO MODERNOS

Una **arquitectura microkernel** asigna solo unas pocas funciones esenciales al núcleo, incluyendo los espacios de almacenamiento, comunicación entre procesos (IPC) y planificación básica.

Multithreading es una técnica en la cual un proceso se divide en una serie de hilos o *threads* que pueden ejecutarse concurrentemente.

- **Thread o hilo.** Se trata de una unidad de trabajo. Incluye el contexto del procesador y su propia área de datos para una pila. Se ejecuta de forma secuencial y se puede interrumpir de forma que el procesador pueda dar paso a otro hilo.
- **Proceso.** Es una colección de uno o más hilos y sus recursos de sistemas asociados.

Para lograr mayor eficiencia y fiabilidad, una técnica consiste en emplear **multiprocesamiento simétrico (Symmetric Multi-Processing, SMP)**, un término que se refiere a la arquitectura del hardware del computador y también al comportamiento del sistema operativo que explota dicha arquitectura. Se puede definir como un sistema de computación aislado con las siguientes características:

1. Tiene múltiples procesadores.
2. Estos procesadores comparten las mismas utilidades de memoria principal y de E/S, interconectadas por un bus de comunicación u otro esquema de conexión interna.
3. Todos los procesadores pueden realizar las mismas funciones.

El sistema operativo de un SMP planifica procesos o hilos a través de todos los procesadores. Tiene diversas ventajas sobre las arquitecturas monoprocesador:

- **Rendimiento.** Alcanzará mayor rendimiento.
- **Disponibilidad.** El fallo de un solo procesador no para la máquina.
- **Crecimiento incremental.** Se puede mejorar el rendimiento añadiendo un procesador adicional.
- **Escalado.** Los fabricantes pueden ofrecer un rango de productos con diferentes precios y técnicas basadas en el número de procesadores configurando el sistema.

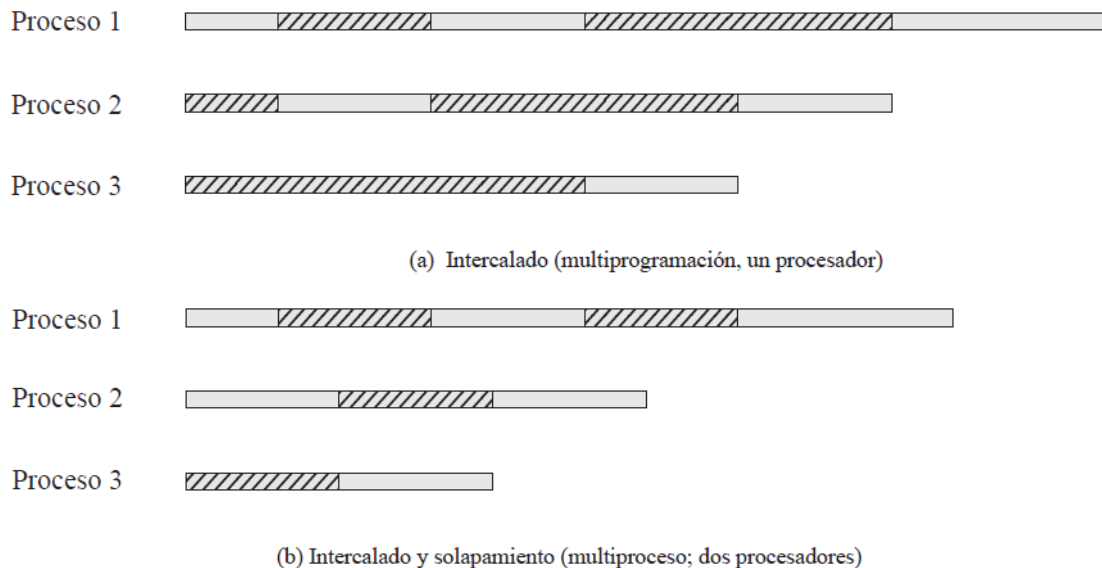


Figure 2.4. Multiprogramación y multiproceso

Un **sistema operativo distribuido** proporciona la ilusión de un solo espacio de memoria principal y un espacio de memoria secundario, más otras utilidades de acceso unificadas.

El **diseño orientado a objetos** introduce una disciplina al proceso de añadir extensiones modulares a un pequeño núcleo. Permite a los programadores personalizar un SO sin eliminar la integridad del sistema.

3. ARQUITECTURAS MONOLÍTICAS, MICROKERNEL Y MÁQUINAS VIRTUALES

El software debe ser modular, ya que ayuda a organizar el proceso de desarrollo de software y limita el esfuerzo de diagnosticar y corregir errores. Las interfaces deben estar bien definidas y tan sencillas como sea posible (con mínimas interfaces entre los módulos, se puede modificar un módulo con un mínimo impacto en los otros módulos)

Para sistemas operativos grandes, no es suficiente modular la programación. La estructura jerárquica separa sus funciones según a sus características de tiempo y nivel de abstracción. Se puede ver el sistema como una serie de niveles, cada nivel confía en los inmediatamente inferiores para realizar funciones más primitivas y ocultar detalles de esas funciones. Cada nivel da servicios a la capa superior

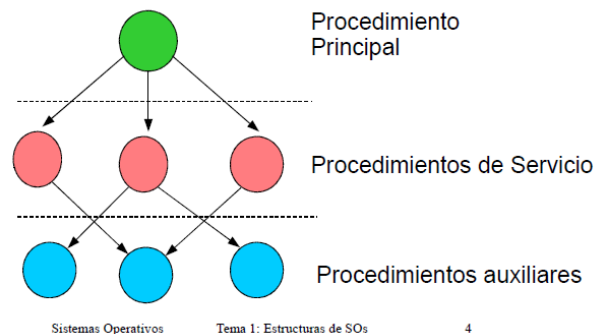
3.1 SISTEMAS MONOLÍTICOS

Hasta hace poco, la mayoría de SO estaban formados por un gran núcleo monolítico, que proporcionaba la mayoría de funcionalidades del SO, incluyendo planificación, los sistemas de ficheros, las redes, los controladores de dispositivos, la gestión de memoria y otras funciones. Normalmente, un núcleo monolítico se implementa como un único proceso, con todos los elementos compartiendo el mismo espacio de direcciones.

Un programa principal invoca el procesamiento de servicio solicitado -> un conjunto de procedimientos de servicios llama al sistema -> un conjunto de procedimientos utilitarios ayuda a los procedimientos de servicio. El SO está formado por conjuntos de procedimientos para que cada uno pueda llamar a los demás cuando lo requiera, todos se ejecutan en modo supervisor y el SO es un único archivo ejecutable.

Problemas.

- No tienen una estructura bien definida
- No se aplica el principio de ocultación
- Son difíciles de comprender y de modificar
- No fiables: un error en alguna parte puede provocar la caída del sistema
- Difíciles de mantener. Desde el principio, los diseñadores han buscado formas de organizar el SO para simplificar su diseño y construcción



Sistemas Operativos

Tema 1: Estructuras de SOs

4

3.2 SISTEMA DE CAPAS

Estos SO son un poco más modernos que los anteriores. El sistema se implementa como un conjunto de capas, cada capa es una máquina más abstracta para la capa superior. Por modularidad, las capas se seleccionan para que cada una utilice funciones sólo de las capas inferiores (adyacentes). Cada capa debe ser desarrollada, codificada y compilada de forma independiente al resto de capas, se debe modularizar el sistema.

Al modularizar el sistema, las capas se seleccionan para que cada capa sólo use las funciones de las capas inmediatamente inferiores, por lo que hay más abstracción en las capas superiores. El proceso se repite de forma sucesiva, causando una sobrecarga en las comunicaciones entre las capas. Si necesito modificar una capa, no hace falta decidir sobre las inferiores. El tiempo de acceso es mayor, para acceder a la capa N, hay que acceder a la N-1 anteriores.

Las funciones se organizan jerárquicamente. En general, las capas inferiores tratan con una escala de tiempo menor, ya que algunas partes del SO interaccionan directamente con el hardware, dónde los eventos pueden tener una escala de tiempo ínfima. Por otra parte, algunas interaccionan con el usuario, por lo que su escala es más larga. Con este enfoque, la mayor parte o todas las capas ejecutan en modo núcleo

3.2.1 EL SISTEMA THE

5: Programas de Usuario
4: Búfering para dispositivos de E/S
3: Manejador de consola del operador
2: Gestión de memoria
1: Planificación de la CPU
Nivel 0: Hardware

El sistema estaba compuesto de una serie de procesos secuenciales

Los procesos se sincronizan con declaraciones explícitas de sincronización

Se puede probar y verificar de forma independiente cada proceso

Problemas.

Los sistemas de capas deben ser jerárquicos pero los sistos. reales son más complejos.

Sobrecarga de comunicaciones entre procesos de distintas capas

A menudo, los sistemas están modelados con esta estructura, pero no están así contruidos

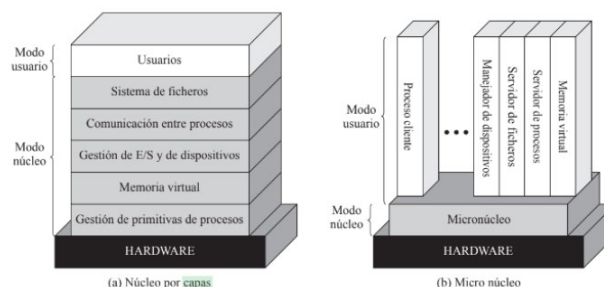
3.3 MICROKERNEL O MICRÓNÚCLEO

Filosofía: sólo las funciones absolutamente esenciales del SO están en el núcleo, las aplicaciones y servicios menos esenciales se construyen sobre el micronúcleo y se ejecutan en modo usuario. Por lo tanto, se reduce el SO a un núcleo mínimo, lo que es igual a una mayor flexibilidad.

Núcleo/kernel es una parte del SO que incluye las funciones + frecuentemente usadas y otras porciones del SO que se estén usando. Gestiona la planificación de hilos, intercambio de procesos, las excepciones, etc. No es paginable ni expulsable, no puede contener errores.

Microkernel: tipo de núcleo de un SO que provee un conjunto de funciones primitivas o llamadas mínimas al sistema para implementar servicios básicos.

El **micronúcleo** funciona como un **intercambiador de mensajes** (hay más sobrecarga por paso de mensajes): válida mensajes, los pasa entre los componentes, y concede acceso al hardware. El programa cliente y el servicio nunca interactúan de forma directa, sino que se comunican indirectamente intercambiando mensajes. El micronúcleo **protege**; previene el paso de mensajes si el intercambio no está permitido. Impone una interfaz uniforme en las peticiones realizadas por un proceso.

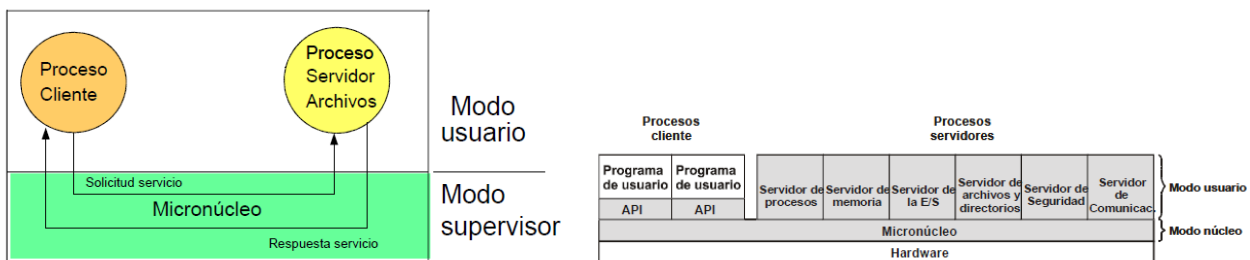


Algunos Sistemas Micronúcleo permiten servidores en modo sistema, lo cual es más eficiente, pero rompe la filosofía micronúcleo. Los servidores son programas independientes, pero se ejecutan en mismo espacio de direcciones del micronúcleo no usan IPCs para comunicarse.

Ventajas:

- **Interfaces uniformes:** impuesta x el micronúcleo en las peticiones realizadas por un proceso, x lo que los procesos no tienen que diferencias entre servicios a nivel de núcleo y de usuario.
- **Extensibilidad:** permite agregar nuevos servicios y realizar varios servicios de la misma área funcional.
- **Flexibilidad:** se pueden añadir nuevas características al SO, y las existentes se pueden eliminar para una implementación más pequeña y eficiente.
- **Portabilidad:** todo o gran parte del código específico del procesador está en el micronúcleo = cambios necesarios para transferir el sistema a un nuevo procesador son menores y tienden a estar unidos en grupos lógicos
- **Fiabilidad:** se puede verificar de forma rigurosa, debido al uso de un num. pequeño de API (interfaces de programación de aplicaciones) ya que hace que sea + sencillo hacer un código de calidad para servicios del SO fuera del núcleo. Hay interacción limitada y nº limitado de API = + difícil afectar negativamente a otros componentes.
- **Soporte de sist. Distribuidos.**
- Da más seguridad ya que la mayor parte de los servicios se ejecutan como procesos de usuario.

La gran desventaja es el rendimiento, ya que lleva más tiempo construir y enviar un mensaje mediante el micronúcleo y aceptar y decodificar la respuesta, que sólo hacer una llamada a un servicio.



3.4 MÁQUINAS VIRTUALES

Tipos de software que simulan una **copia exacta del software**. Implementan una máquina virtual (igual o distinta de una máquina real). El SO crea una máquina virtual pero extendida (abstracción del hardware), permite a una máquina subyacente multiplicarse en varias máquinas virtuales, cada una ejecutando su propio SO. La capacidad de procesamiento actual mitiga la ineficiencia de las MV, ya que los procesadores más actuales incluyen soporte para la misma.

Hipervisor: capa de software que permite la virtualización.

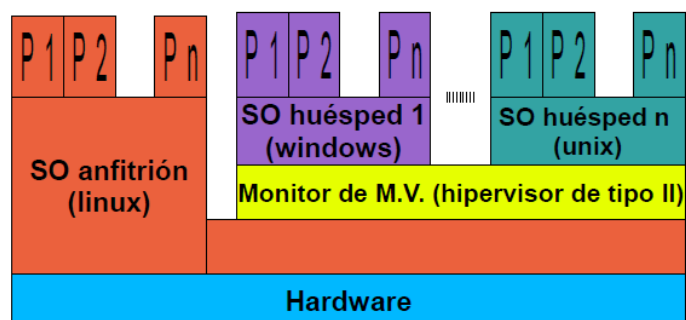
Una petición de servicio es atendida por la copia de SO sobre la que se ejecuta.

Ventajas:

- Aislamiento de cada MV asegura la protección de recursos.
- Útil para investigar y desarrollar SO, no interrumpe funcionamiento del sistema (podemos usar sus herramientas)
- Permite ejecución de aplicaciones realizadas para otro SO.

Desventajas:

- Dificiles de implementar: es muy complejo crear un duplicado del hardware.
- La compartición de recursos es compleja.



4. SO DE PROPÓSITO ESPECÍFICO

4.1 SO DE TIEMPO REAL

Se usan cuando debemos asegurar una respuesta en un tiempo dado, se implementan en el SO unas restricciones temporales. Se usan para aplicaciones especializadas. El problema que suponen es planificar las actividades con el fin de satisfacer todos los requisitos críticos. Esto se debe a que algunas de las tareas son a tiempo real y tienen cierto grado de urgencia, ya que tratan de controlar o reacciones a eventos que ocurren en el mundo exterior.

La computación en tiempo real puede definirse como aquella en la que la corrección del sistema depende no sólo del resultado lógico de la computación sino también del momento en el que se producen los resultados.

El sistema debe planificar todos los recursos que necesita un proceso para que pueda ejecutarse en el tiempo restringido, cómo esta estimación es difícil, no existe ningún SO de tiempo real que sea 100% fiable.

Arquitectura Multiprocesador: no se usan, desde el punto de vista del user, todos funcionan como un mismo SO. Parte de los recursos son compartidos. Se crearon cuando se necesitaba hacer muchos cálculos, cada computador los realizaba y después se coordinaban los resultados.

4.1.1 TIPOS DE TAREAS

Tarea de tiempo real duro: aquella que debe cumplir su plazo límite, o si no se produce un daño inaceptable o error fatal en el sistema.

Tarea de tiempo real suave: tiene asociado plazo límite deseable pero no obligatorio; sigue teniendo sentido planificar y completar la tarea si éste ha vencido.

Tarea aperiódica: tiene plazo en el cual debe finalizar/empezar, o una restricción de esos instantes.

Tarea periódica: requisito puede ser enunciado como "una vez x periodo T"

4.1.2 CARACTERÍSTICAS DE SISTEMAS DE TIEMPO REAL

Determinismo: realiza operaciones en instantes de tiempo fijos predeterminados o dentro de intervalos de tiempo predeterminados. Para medirlo: retardo máximo desde la llegada de una interrupción de un dispositivo de alta prioridad hasta que comienza el servicio. Cuanto tarda el SO antes de reconocer una interrupción.

Reactividad: cuanto tiempo tarda el SO, tras reconocerla, en servir la interrupción. Incluye el tiempo necesario para manejar la interrupción y comenzar a ejecutar la rutina de servicio de la interrupción (RSI), cantidad de tiempo para la RSI, efecto de anidamiento de interrupciones.

Determinismo y reactividad = conforman tiempo de respuesta a eventos externos.

Control de usuario: generalmente mucho mayor en un SO de tiempo real que en SO ordinarios.

Fiabilidad.

Cambio de proceso, funcionalidades mínimas, poder responder rápido a interrupciones externas, multitarea con herramientas para la comunicación entre procesos, etc.

4.2 SO DE RED

En esta configuración hay una red de máquinas, estaciones de trabajo de un sólo usuario, y una o más máquinas servidoras que proporcionan servicios de red o aplicaciones. Cada computador tiene su propio SO y tiene sus propios usuarios. El SO de red es un añadido al SO local, que permite a las máquinas interactuar con los servidores. Usuario conoce la existencia de varios computadores y debe trabajar con ellos explícitamente.

Lo que les distingue de los SO de un procesador es la necesidad de software especial como controlador de interfaz de la red, y programas de conexión y acceso a archivos remoto. Se usan para grandes cálculos y que cada uno de los nodos del SO cooperen en red

4.3 SO DISTRIBUIDOS

Un SO común compartido por una red de computadores. A los usuarios les parece un SO normal centralizado (una misma máquina), pero les da acceso transparente a recursos de diversas máquinas. Puede depender de una arquitectura de comunicaciones para las funciones básicas de comunicación, pero suele haber funciones de comunicación más sencillas para más eficiencia.

El propio SO reparte las tareas entre los nodos del sistema. Todos los nodos tienen el mismo SO para agilizar las tareas y que funcione de manera homogénea.

Son sistemas débilmente acoplados, no tienen memoria común, y su rasgo fundamental es la transparencia. Aumentan la fiabilidad.

ARQUITECTURA MULTIPROCESADOR

El SO tiene que controlar acceso en exclusión mutua a recursos, ya que hay varios procesadores con memoria compartida. Cada procesador tiene varios niveles de caché, si uno de los procesadores modifica un recurso en memoria principal, el resto de los procesadores lo deben tener en cuenta. Se usan políticas de planificación.

4.4 SO PARALELOS

Son sistemas Multiprocesadores fuertemente acoplados, comparten memoria y el reloj. Hay 2 tipos de multiprocesamiento:

- **Simétrico (SMP):** cada procesador ejecuta copia idéntica del SO (buen rendimiento).
- **Asimétrico (ASMP):** procesador maestro ejecuta SO, procesadores esclavos ejecutan procesos de usuario (peor escalabilidad). No se usa.

Soportan aplicaciones que desean obtener aumento de velocidad de tareas computacionalmente complejas.

Necesitan primitivas básicas para dividir una tarea en varias actividades.

Proporciona comunicación y sincronización eficiente entre esas actividades.

Tiene tolerancia a fallos.

Hay sistemas donde no hace falta que exista paralelismo.

5. COSAS BÁSICAS SOBRE LOS SISTEMAS OPERATIVOS

5.1 CARACTERÍSTICAS DE UN SO

Corrección. Debe poder corregir fallos durante la ejecución.

Eficiencia.

Fiabilidad. Debe dar respuestas predecibles a ciertas condiciones y proteger el computador y el software vs ataques o software erróneo.

Extensibilidad. Se deben poder añadir nuevas funciones sin interferir en los servicios ya establecidos.

Tolerancia a fallos.

Reentrancia.

5.2 SISTEMAS MULTIPROGRAMADOS

Multiprogramación es la capacidad de un SO de realizar varios programas que residen en memoria principal al mismo tiempo, aunque requiere interrupciones para las operaciones de E/S. El SO debe:

- Proporcionar rutinas de gestión y control sobre las operaciones de E/S.
- Gestionar memoria principal, ya que varios procesos deben estar preparados para ejecutarse.
- Planificar uso del procesador, ya que debe decidir qué proceso ejecutar.

Se ejecutan varios programas, por lo que no es necesario esperar a que termine uno para que comience otro, tienen alta productividad.

Procesos y Hebras

1. DESCRIPCIÓN Y CONTROL DE PROCESOS

Los requisitos que un SO debe cumplir (con referencia a los procesos) se expresan de la siguiente manera:

- El SO debe intercalar la ejecución de múltiples procesos, para maximizar la utilización del procesador mientras se proporciona un tiempo de respuesta razonable.
- El SO debe reservar recursos para los procesos conforme a una política específica mientras que, al mismo tiempo, evita interbloqueos.
- Un SO puede requerir dar soporte a la comunicación entre procesos y la creación de procesos, mediante las cuales ayuda a la estructuración de las aplicaciones.

PROCESOS Y BLOQUE DE CONTROL DE PROCESOS (PCB)

Un proceso se puede definir como:

- Un programa en ejecución.
- Una instancia de un programa ejecutado en un computador.
- La entidad que se puede asignar y ejecutar en un procesador.
- Una unidad de actividad que se caracteriza por la ejecución de una secuencia de instrucciones, un estado actual, y un conjunto de recursos del sistema asociados.

También puede expresarse como una entidad que consiste en un número de elementos. Los dos elementos esenciales serían el **código de programa** y un **conjunto de datos asociados a dicho código**. Además, supongamos que el procesador empieza a ejecutar este código de programa, nos referimos a dicha entidad de ejecución como un **proceso**. En cualquier instante puntual del tiempo, *mientras el proceso está en ejecución*, este proceso se puede caracterizar por una serie de elementos:

- **Identificador (PID)**. Para distinguirlo del resto de procesos.
- **Estado**. Si el proceso está corriendo, está en el estado en ejecución.
- **Prioridad**. Nivel de prioridad relativo al resto de procesos.
- **Contador de programa**. La dirección de la siguiente instrucción del programa que se ejecutará.
- **Punteros a memoria**. Incluye los punteros al código de programa y los datos asociados a dicho proceso, además de cualquier bloque de memoria compartido con otros procesos.
- **Datos de contexto**. Están presente en los registros del procesador cuando el proceso está corriendo.
- **Información de estado de E/S**. Incluye las peticiones de E/S pendientes, dispositivos de E/S asignados a dicho proceso, una lista de los ficheros en uso por el mismo, etc.
- **Información de auditoría**. Puede incluir la cantidad de tiempo de procesador y de tiempo de reloj utilizados, así como los límites de tiempo, registros contables, etc.

Esta información se almacena en una estructura de datos, que suele llamarse **bloque de control de proceso (Process Control Block, PCB)**, que el sistema operativo crea y gestiona. El PCB es la herramienta clave que permite al SO dar soporte a la multiprogramación. Contiene suficiente información de forma que es posible interrumpir el proceso y reanudar su estado de ejecución como si no hubiera habido interrupción.

1.1 ESTADOS DE LOS PROCESOS

Para que un programa se ejecute, se debe crear un proceso o tarea para dicho programa. Desde el **punto de vista del procesador**, él ejecuta instrucciones de su repertorio de instrucciones en una secuencia dictada por el cambio de los valores del registro contador de programa. Desde el **punto de vista de un programa**, su ejecución implica una secuencia de instrucciones dentro de dicho programa.

Se puede saber el comportamiento de un determinado proceso, listando la secuencia de instrucciones que se ejecutan para dicho proceso. A esta lista se la denomina **traza del proceso**.

De manera adicional, existe un **programa activador (dispatcher)** que intercambia el procesador de un proceso a otro.

1.1.1 MODELO DE PROCESO DE DOS ESTADOS

La responsabilidad principal del sistema operativo es controlar la ejecución de los procesos; esto incluye determinar el patrón de entrelazado para la ejecución y asignar recursos a los procesos.

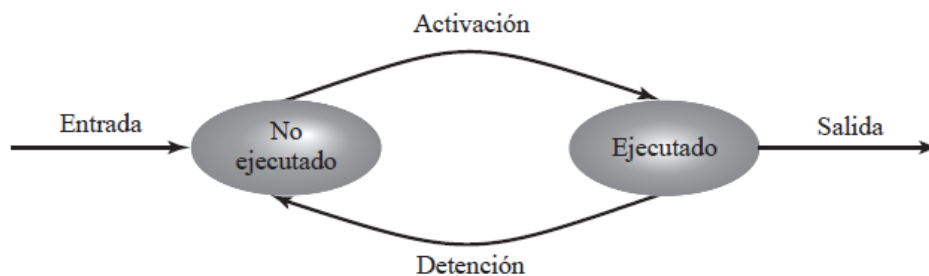
Se puede construir el modelo más simple posible observando que, en un instante dado, un proceso está siendo ejecutado por el procesador o no. Cuando el SO crea un nuevo proceso, crea el bloque de control de proceso para el nuevo proceso e inserta dicho proceso en el sistema en estado No Ejecutando. De cuando en cuando, el proceso actualmente en ejecución se interrumpirá y el activador seleccionará otro proceso a ejecutar.

Cada proceso debe representarse de forma que el sistema operativo pueda seguirle la pista. Es decir, debe haber información correspondiente a cada proceso, incluyendo el **estado actual** y su **localización en memoria**; esto es el boque de control de programa.

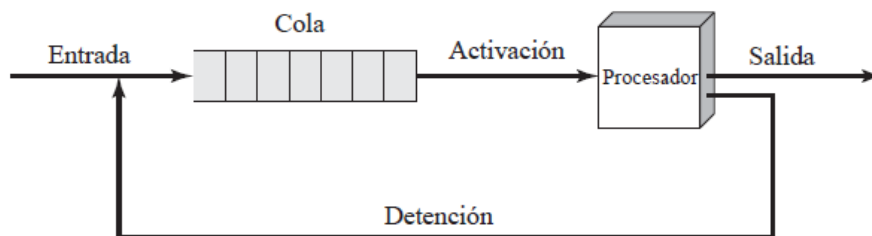
Los procesos que no están ejecutándose deben estar en una cola, esperando su turno de ejecución. Podríamos tener dos implementaciones en esta cola:

- Una cola cuyas entradas son punteros al PCB de un proceso.
- Una cola que consiste en una lista enlazada de bloques de datos, en la que cada bloque representa un proceso.

Un proceso que se interrumpe, pasa a la cola de procesos en espera. Alternativamente, si el proceso ha finalizado o ha sido abortado se sale del sistema.



(a) Diagrama de transiciones de estados



(b) Modelos de colas

1.1.2 CREACIÓN Y TERMINACIÓN DE PROCESOS

CREACIÓN DE UN PROCESO

Al crear un proceso, el SO construye las estructuras de datos que se usan para manejar el proceso y se reserva el espacio de direcciones en memoria principal para el proceso.

Tabla 1.1. Razones para la creación de un proceso

Nuevo proceso de lotes	El So dispone de un flujo de control de lotes de trabajos, habitualmente una cinta un disco. Cuando el SO está listo para procesar un nuevo trabajo, leerá la siguiente secuencia de mandatos de control de trabajos.
Sesión interactiva	Un usuario desde un terminal entra en el sistema.
Creado por el SO para proporcionar un servicio	El SO puede crear un proceso para realizar una función en representación de un programa de usuario, sin que el usuario tenga que esperar.
Creado por un proceso existente	Por motivos de modularidad o para explotar el paralelismo, un programa de usuario puede ordenar la creación de un número de procesos.

En el caso de que un proceso cree un nuevo proceso, al primero se le denominará **proceso padre** y al proceso creado se le denomina **proceso hijo**.

TERMINACIÓN DE PROCESOS

Todo sistema debe proporcionar los mecanismos mediante los cuales un proceso indica la finalización, o que ha completado su tarea.

Tabla 1.2. Razones para la terminación de un proceso.

Finalización normal	El proceso ejecuta una llamada al sistema operativo para indicar que ha completado su ejecución.
Línea de tiempo excedido	El proceso ha ejecutado más tiempo del especificado en un límite máximo. Existen varias posibilidades para medir dicho tiempo. Estas incluyen el tiempo total utilizado, el tiempo utilizado únicamente en ejecución, y, en el caso de procesos interactivos, la cantidad de tiempo desde que el usuario realizó la última entrada.
Memoria no disponible	El proceso requiere más memoria de la que el sistema puede proporcionar.
Violaciones de frontera	El proceso trata de acceder a una posición de memoria a la cual no tiene acceso permitido.
Error de protección	El proceso trata de usar un recurso, por ejemplo, un fichero, al que no tiene permitido acceder, o trata de utilizarlo de una forma no apropiada.
Error aritmético	El proceso trata de realizar una operación de cálculo no permitida, tal como una división por 0, o trata de almacenar números mayores de los que la representación hardware puede codificar.
Límite de tiempo	El proceso ha esperado más tiempo que el especificado en un valor máximo para que se cumpla un determinado evento.
Fallo de E/S	Se ha producido un error durante una operación de entrada o salida, por ejemplo, la imposibilidad de encontrar un fichero, fallo en la lectura o escritura después de un límite máximo de intentos, o una operación inválida.
Instrucción no válida	El proceso intenta ejecutar una instrucción inexistente.
Instrucción privilegiada	El proceso intenta ejecutar una instrucción reservada al SO.
Uso inapropiado de datos	Una porción de datos es de tipo erróneo o no se encuentra inicializada.
Intervención del operador por el SO	Por alguna razón, el operador o el SO ha finalizado el proceso.
Terminación del proceso padre	Cuando un proceso padre termina, el SO puede automáticamente finalizar todos los procesos hijos descendientes.
Solicitud del proceso padre.	Un proceso padre puede finalizar la ejecución de sus procesos hijos mediante la llamada kill() .

1.1.3 MODELO DE PROCESO DE CINCO ESTADOS

El modelo anterior es inadecuado, ya que algunos procesos que están en cola están listos para ejecutar, mientras que otros no. Por tanto, utilizando una única cola, el activador no puede seleccionar únicamente los procesos que lleven más tiempo en la cola. En su lugar, debería recorrer la lista buscando los procesos que no estén bloqueados y que lleven en la cola más tiempo.

Una forma más natural para manejar esta situación es dividir el estado de No Ejecutando en dos estados, Listo y Bloqueado. Para gestionarlo correctamente, se han añadido dos estados adicionales. Estos cinco estados en el nuevo diagrama son los siguientes:

- **Ejecutando.** El proceso está actualmente en ejecución.
- **Listo.** Un proceso que se prepara para ejecutar cuando tenga oportunidad.
- **Bloqueado.** Un proceso que no se puede ejecutar hasta que se cumpla determinado evento o se complete una operación de E/S.
- **Nuevo.** Un proceso que se acaba de crear y que aún no ha sido admitido en el grupo de procesos ejecutables por el SO.
- **Saliente.** Un proceso que ha sido liberado del grupo de procesos ejecutables por el SO.

1.1.4 PROCESOS SUSPENDIDOS

Los tres principales estados descritos (Listo, Ejecutando, Bloqueado) proporcionan una forma sistemática de modelizar el comportamiento de los procesos y diseñar la implementación del SO.

Sin embargo, para este modelo, cada proceso debe cargarse completamente en memoria principal, suponemos el sistema no utiliza memoria virtual. Esto supondría una mayor necesidad de memoria principal, con su respectivo coste.

La solución a este problema es la **memoria de intercambio (swapping)**, que implica mover parte o todo el proceso de memoria principal al disco. El *swapping* es una operación de E/S, y por tanto existe el riesgo de hacer que el problema empeore. Pero debido a que la E/S en disco es habitualmente más rápida que la E/S sobre otros sistemas, el *swapping* habitualmente mejora el rendimiento del sistema.

Con el uso del *swapping*, debe añadirse un nuevo estado a nuestro modelo de comportamiento de procesos: el estado Suspendido.

Cuando todos los procesos en memoria principal se encuentran en estado bloqueado, el SO puede suspender un proceso poniéndolo en Suspendido y transfiriéndolo a disco, para traer en su lugar un proceso preparado. Existen dos opciones para seleccionar el nuevo proceso de disco:

- Admitir un proceso nuevo.
- Traer un proceso que estuviese suspendido.

Sin embargo, éste último puede significar que cuando el proceso llegue a memoria principal siga bloqueado, por no haber terminado aún el evento al que esperaba (recordemos que estar en estado Suspendido significa que estuvo bloqueado previamente en memoria principal).

Para solucionar este posible problema, necesitamos implementar más estados:

- **Listo.** En memoria principal disponible para su ejecución.
- **Bloqueado.** En memoria principal y esperando un evento.
- **Bloqueado/Suspendido.** En almacenamiento secundario y esperando un evento.
- **Listo/Suspendido.** En almacenamiento secundario pero disponible para su ejecución.

Podemos definir un proceso suspendido como el que cumple las siguientes características:

- No está inmediatamente disponible para su ejecución.
- Puede estar o no a la espera de un evento, si es así, la condición de bloqueo es independiente de la condición de estar suspendido, y si sucede el evento que lo bloquea, eso no habilita al proceso para su ejecución inmediata.
- El proceso fue puesto en estado suspendido por un agente.
- No puede ser recuperado de este estado hasta que explícitamente así lo indique.

Tabla 1.3. Razones para la suspensión de un proceso.

<i>Swapping</i>	El SO necesita liberar suficiente memoria principal para traer un proceso en estado Listo de ejecución.
Otras razones del SO	El SO puede suspender un proceso en segundo plano o de utilidad o un proceso que se sospecha puede causar algún problema.
Solicitud interactiva del usuario	Un usuario puede desear suspender la ejecución de un programa con motivo de su depuración o porque está utilizando un recurso.
Temporización	Un proceso puede ejecutarse periódicamente y puede suspenderse mientras espera el siguiente intervalo de ejecución.
Solicitud del proceso padre	Un proceso padre puede querer suspender la ejecución de un descendiente para examinar o modificar dicho proceso suspendido, o coordinar la actividad de varios procesos descendientes.

1.2 DESCRIPCIÓN DE PROCESOS

El mecanismo universal para proporcionar información respecto al estado actual de cada proceso es la construcción de tablas de información sobre cada entidad que gestiona el SO.

Las **tablas de memoria** se usan para mantener un registro tanto de la memoria principal como de la secundaria. Deben incluir la siguiente información:

- Reservas de memoria principal por parte de los procesos.
- Reservas de memoria secundaria por parte de los procesos.
- Todos los atributos de protección que restringe el uso de la memoria principal y virtual.
- La información necesaria para manejar la memoria virtual.

Se utilizan las **tablas de E/S** para gestionar los dispositivos de E/S y los canales del computador. Si una operación se está realizando, el SO necesita conocer el estado de la operación y la dirección de memoria principal del área usada como fuente o destino de la transferencia.

El sistema operativo también puede mantener las **tablas de ficheros**. Estas tablas proporcionan información sobre la existencia de ficheros, su posición en almacenamiento secundarios, su estado actual y otros atributos.

Las **tablas de procesos** se utilizan para gestionar los procesos.

1.2.1 ESTRUCTURAS DE CONTROL DE PROCESO

LOCALIZACIÓN DE LOS PROCESOS

Un proceso debe consistir en una cantidad suficiente de memoria para almacenar el programa y datos del mismo. Adicionalmente, la ejecución incluye una pila. Por último, cada proceso está asociado a un número de atributos que son utilizados por el sistema operativo para controlar el proceso. El conjunto de estos atributos se denomina **bloque de control del proceso (PCB)**. Nos podemos referir al conjunto de programa, datos, pila, y atributos, como la **imagen del proceso**.

Para que el SO pueda gestionar el proceso, al menos una pequeña porción de su imagen se debe mantener en memoria principal. Para ejecutarlo, la imagen completa se debe cargar en memoria principal, o al menos en memoria virtual.

Tabla 1.4. Elementos típicos de la imagen de un proceso

Datos del usuario	La parte modificable del espacio de usuario. Puede incluir datos de programa, área de pila de usuario, y programas que puedan ser modificados.
Programa de usuario	El programa a ejecutar.
Pila de sistema	Cada proceso tiene una o más pilas de sistema (LIFO) asociadas a él. Una pila se utiliza para almacenar los parámetros y las direcciones de retorno de los procedimientos y llamadas al sistema.
Bloque de control de proceso	Datos necesarios para que el sistema operativo pueda controlar los procesos.

ATRIBUTOS DE PROCESO

Tabla 1.5. Elementos típicos de un PCB.

Identificación del proceso	
Identificadores	
- Identificadores del proceso - Identificador del proceso que creó a este proceso - Identificador del usuario	
Información de estado del procesador	
Registros visibles por el usuario	
Aquél al que se puede hacer referencia por medio del lenguaje máquina cuando se está en modo usuario.	
Registros de estado y control	
Contador de programa. Contiene la dirección de la siguiente instrucción a ejecutar Códigos de condición. Resultan de la operación lógica o aritmética más reciente Información de estado. Incluyen los <i>flags</i> de interrupciones habilitadas/deshabilitadas	
Puntero de pila	
Apunta a la parte más alta de la pila asociada al proceso.	
Información de control de proceso	
Información de estado y de planificación	
Información que el sistema operativo necesita para analizar las funciones de planificación. Elementos típicos:	
Estado del proceso. Indica si está listo o no el proceso para ser planificado para su ejecución. Prioridad. Uno o más campos que se pueden utilizar para escribir la prioridad de planificación del proceso. Información relativa a planificación. Depende del algoritmo de planificación utilizado. Evento. Identificar el evento al cual el proceso está esperando.	
Estructuración de datos	
Un proceso puede estar enlazado con otro en una cola, anillo o con cualquier otra estructura. El PCB puede contener punteros a otros procesos para dar soporte a estas estructuras.	
Comunicación entre procesos	
Se pueden asociar diferentes <i>flags</i> , señales y mensajes relativos a la comunicación entre dos procesos.	
Privilegios de proceso	
Adquieren privilegios de acuerdo a la memoria que van a acceder y los tipos de instrucciones que pueden ejecutar.	
Gestión de memoria	
Esta sección puede incluir punteros a tablas de segmentos y/o páginas que describen la memoria virtual asignada	
Propia de recursos y utilización	
Se deben indicar los recursos controlados por el proceso. También se puede incluir un histórico de utilización del procesador o de otros recursos.	

INFORMACIÓN DE ESTADO DE PROCESO

Indica los contenidos de los registros del procesador. Cuando un proceso se interrumpe, esta información debe salvaguardarse de forma que se pueda restaurar cuando el proceso continúe con su ejecución. Normalmente, el conjunto de registros incluye registros visibles por usuario, registros de control y de estado y punteros de pila.

El diseño de todos los procesadores incluye un registro o conjunto de ellos, conocidos como **palabras de estado de programa (PSW)**, que contiene códigos de condición, así como otra información de estado.

EL PAPEL DEL BLOQUE DE CONTROL DE PROCESO (PCB)

El PCB es la estructura de datos más importante del sistema operativo. Cada PCB contiene toda la información sobre un proceso que necesita el SO. Se puede decir que el conjunto de bloques de control de proceso define el estado del sistema operativo y/o modifican totalmente los módulos del SO,

Un gran número de rutinas dentro del SO necesitan acceder a la información de los PCB. Cada proceso, como lleva un único identificador, podemos utilizarlos para indexar dentro de una tabla de punteros a bloques de control de proceso. Puede haber problemas de protección:

- Un fallo en una simple rutina puede dañar los PCBs, lo cual puede destruir la capacidad del sistema para manejar los procesos afectados.
- Un cambio en la estructura de los PCBs puede afectar a un gran número de módulos del SO.

Estos problemas se pueden tratar obligando a que todas las rutinas del SO pasen a través de una rutina de manejador, cuyo único trabajo es proteger a los PCBs, y que es la única que realiza el arbitraje en las operaciones de lectura y escritura de dichos bloques.

1.3 CONTROL DE PROCESOS

1.3.1 MODOS DE EJECUCIÓN

Con respecto a la ejecución en CPU se utilizan dos niveles de privilegio (modos de ejecución): usuario y kernel. Para saber en qué modo se está ejecutando, existe un bit en la palabra de estado que indica el modo de ejecución. Otros procesadores incluyen un registro de estado (psr) que incluye un campo de dos bits siendo el 0 el nivel con más privilegio y el 3 con menos.

Con respecto a la memoria:

- Espacio protegido para el kernel:
 - La CPU opera sobre este espacio solo en modo kernel
 - Para trabajar sobre este espacio se requiere una secuencia especial de instrucciones para cambiar de modo *user* a modo *kernel*.
 - El código de este espacio es **reentrante**.
- Cada proceso tiene su propio espacio de direcciones protegido:
 - Es necesario tener información sobre el estado de la memoria asociada a cada proceso. Esta información se guarda en espacio de memoria del kernel.
 - Todos los procesos comparten el mismo espacio de kernel.
 - Existe una pila de kernel por cada proceso ya que el kernel es un código reentrante.

1.3.2 CREACIÓN DE PROCESOS

Una vez que el sistema operativo decide crear un proceso, procederá de la siguiente manera:

1. **Asignar un identificador de proceso único al proceso.** En este instante, se añade una nueva entrada a la tabla primaria de procesos, que contiene una entrada por proceso.
2. **Reservar espacio para proceso.** Para ello el sistema operativo debe conocer cuanta memoria se requiere para el espacio de direcciones privados (programas y datos) y para la pila de usuario. Por último, se debe reservar espacio para el PCB.
3. **Inicialización del bloque de control de proceso.** Es la información de estado de proceso, habitualmente se inicializa con la mayoría de entradas a 0, excepto por el contador de programa y los punteros de pila de sistema. La información de control de procesos se inicializa en base a los valores por omisión, considerando también los atributos que han sido solicitados para este proceso. La prioridad se puede fijar, por defecto, a la más baja, a menos que una solicitud explícita la eleve a una prioridad mayor. Inicialmente el proceso no debe poseer ningún recurso a menos que exista una indicación explícita o que haya sido heredados del padre.
4. **Establecer los enlaces apropiados.** Por ejemplo, si el SO mantiene cada cola del planificador como una lista enlazada, el nuevo proceso debe situarse en la cola de Listos o Listos/Suspendidos.
5. **Creación o expansión de otras estructuras de datos.** Por ejemplo, el SO puede mantener un registro de auditoría por cada proceso que se puede utilizar posteriormente a efectos de facturación y/o análisis de rendimiento del sistema.

En el caso de que un proceso cree un nuevo proceso, el proceso hijo obtiene sus recursos de:

- Directamente del SO. Padre e hijo no comparten recursos.
- Comparte todos los recursos con el padre.
- Comparte un subconjunto de los recursos del padre.

Las posibilidades de ejecución del proceso padre e hijo: padre e hijo se ejecutan concurrentemente o el padre espera a que el hijo termine.

Las posibilidades en cuanto a espacio de direcciones son que el espacio de direcciones del hijo es un duplicado del padre (UNIX-like OS) o que el espacio del hijo contenga un programa nuevo distinto al asociado al proceso padre (VMS).

UNIX-LIKE OS

La llamada al sistema **fork()** crea un nuevo proceso.

La llamada **exec()** reemplaza el espacio de direcciones con el programa pasado como argumento.

Si se realiza una llamada **fork()** seguida de una llamada **exec()** se consigue un proceso hijo que ejecuta el programa pasado como argumento en **exec()**.

fork() crea un nuevo hijo que hereda:

- Una copia idéntica de la memoria del padre.
- Una copia idéntica de los registros de CPU del padre.

Los procesos padre e hijo se ejecutan en el mismo punto de ejecución: tras el retorno de **fork()** y, por convenio:

- **fork()** devuelve un 0 en el proceso hijo.
- **fork()** devuelve el identificador de proceso del hijo en el proceso padre.

1.3.3 CAMBIO DE PROCESOS

Un cambio de procesos puede ocurrir en cualquier instante en el que el sistema operativo obtiene el control sobre el proceso actualmente en ejecución.

Consideremos las interrupciones del sistema. Podemos distinguir dos tipos diferentes de interrupciones de sistema, unas simplemente denominadas interrupciones, y las otras denominadas *traps*.

Interrupciones. Suceden por un evento externo e independiente al proceso en ejecución. El control se transfiere inicialmente al manejador de interrupción, que realiza determinadas tareas internas y que posteriormente salta a una rutina del sistema operativo, encargada de cada uno de los tipos de interrupciones. Algunos ejemplos son:

- **Interrupción de reloj.** El SO determina si el proceso ha excedido o no la unidad máxima de tiempo de ejecución, denominada **rodaja de tiempo** (*time slice*). En este caso, el proceso puede pasar al estado de Listo.
- **Interrupción de E/S.** El SO determina qué acción de E/S ha ocurrido. Si la acción de E/S constituye un evento por el cual están esperando uno o más procesos, se mueven esos procesos al estado Listos (Listos/Suspendidos). El SO puede decidir si reanuda la ejecución del proceso actual o si lo expulsa para ejecutar uno con mayor prioridad.
- **Fallo de memoria.** El procesador se encuentra con una referencia a una dirección de memoria virtual, a una palabra que no se encuentra en memoria principal. El SO debe traer el bloque (página o segmento) que contiene la referencia desde memoria secundaria a memoria principal. Tras solicitarse la operación, el proceso que causó el fallo de memoria pasa al estado Bloqueado, después de que el bloque se haya traído pasará a Listo.

Traps. Asociadas a una condición error o excepción generada dentro del proceso que está ejecutando. El sistema operativo conoce si una condición de error o de excepción es irreversible. Si es así, el proceso en ejecución se pone en estado Saliente y se hace un cambio de proceso. De no ser así, el SO actuará dependiendo de la naturaleza del error y del propio diseño.

Llamada al sistema. El sistema operativo se puede activar por medio una llamada al sistema procedente del programa en ejecución. Esta llamada implica un salto a una rutina que es parte del código del SO. La realización de una llamada al sistema puede implicar en algunos casos que el proceso que la realiza pase al estado Bloqueado.

CAMBIO DE MODO

En cada ciclo de instrucción tenemos una fase de interrupción. En esta, el procesador comprueba que no exista ninguna interrupción pendiente: si no la hay, el procesador pasa a la fase de búsqueda de instrucción, siguiendo con el programa del proceso actual.

Cuando un proceso está ejecutándose, el programa asociado está utilizando los registros de la CPU: PC, PSW, SP... En otras palabras, los registros contienen el estado actual de la computación. Si hay una interrupción pendiente, el SO detiene el proceso que está ejecutándose, salva los valores actuales de estos registros (contexto de registros) en el PCB de dicho proceso. La acción de cambiar al proceso que está usando la CPU por otro proceso se denomina cambio de contexto.

Si hay una interrupción pendiente, el proceso actúa de la siguiente manera:

- Coloca el PC en la dirección de comienzo de la rutina del programa manejador de la interrupción.
- Cambia de modo usuario a modo núcleo de forma que el código de tratamiento de la interrupción pueda incluir instrucciones privilegiadas.

Acto seguido, el procesador pasa a la fase de búsqueda y busca la primera instrucción del programa de manejo de interrupción. En este punto, la información del proceso que ha sido interrumpido se guarda en su PCB. Se salvaguarda toda la información que se puede ver alterada por la ejecución de la rutina de interrupción y que se necesitará para la continuación del proceso que ha sido interrumpido. Se debe guardar la parte del PCB que hace referencia a la información de estado del procesador, esto incluye el PC, otros registros del procesador, y la información de la pila.

Si a la interrupción le sigue un cambio de proceso a otro, se necesita hacer algunas tareas más. Sin embargo, en muchos SO, la existencia de una interrupción no implica un cambio de proceso. Sólo es necesario guardar la información del estado del procesador cuando se produce la interrupción y restablecerlo cuando se reanude.

CAMBIOS DE ESTADO DEL PROCESO

El cambio de modo es un concepto diferente del cambio de proceso. Un cambio de modo puede ocurrir sin que se cambie el estado del proceso. Los pasos que se realizan para un cambio de proceso completo son:

1. Salvar el estado del procesador, incluyendo el PC y otros registros.
2. Actualizar el PCB del proceso que está actualmente ejecutándose.
3. Mover el PCB a la cola apropiada.
4. Seleccionar un nuevo proceso a ejecutar.
5. Actualizar el bloque de control del proceso elegido.
6. Actualizar las estructuras de datos de gestión de memoria.
7. Restaurar el estado que tenía el procesador en el momento en el que el proceso seleccionado salió del estado Ejecutando la última vez.

2. PROCESOS E HILOS

Hasta el momento se ha presentado el concepto de un proceso como poseedor de dos características:

- **Propiedad de recursos.** Incluye un espacio de direcciones virtuales para el manejo de **la imagen del proceso**, colección de programas, datos, pila y atributos definidos en el PCB. De vez en cuando a un proceso se le puede asignar control o propiedad de recursos tales como la memoria principal, canales E/S, dispositivos E/S y archivos. El sistema operativo realiza la función de protección para evitar interferencias no deseadas entre procesos en relación con los recursos.
- **Planificación/ejecución.** La ejecución de un proceso sigue una ruta de ejecución (traza) a través de uno o más programas. Esta ejecución puede estar intercalada con ese u otros procesos. De esta manera, un proceso tiene un estado de ejecución y una prioridad de activación y ésta es la entidad que se planifica y activa por el sistema operativo.

Estas dos características son independientes y podrían ser tratadas como tales por el SO. Para distinguir estas dos características, la unidad que se activa se suele denominar **hilo (thread)**, o **proceso ligero**, mientras que la unidad de propiedad de recursos se suele denominar **proceso** o **tarea**.

Una hebra es la unidad básica de utilización de la CPU y el *kernel* requiere la siguiente información para gestionarla:

- Contexto de registros: PC, PSW, SP...
- Pila de ejecución.
- Estado.

Una hebra comparte con sus hebras pares una tarea (proceso) que mantiene el resto de información necesaria:

- Sección de código (texto).
- Sección de datos.
- Recursos del SO (archivos abiertos, señales...).

Se entiende que un sistema operativo es **single threading** cuando el *kernel* no reconoce el concepto de hebra. Se entiende que un sistema operativo es **multithreading** cuando el *kernel* soporta múltiples hebras dentro de un proceso.

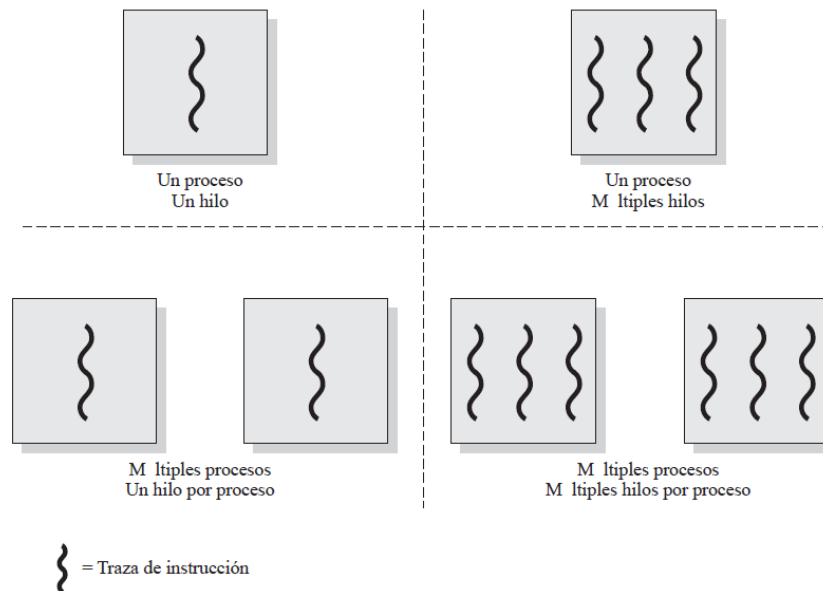


Figure 2.1. Hilos y procesos.

2.1 MULTITHREAD

Capacidad del SO de dar soporte a múltiples hilos de ejecución en un solo proceso. En el enfoque tradicional de un solo hilo por proceso, donde no se identifica el concepto de hilo, se conoce como estrategia **monohilo**.

En un entorno multihilo, un proceso se define como la unidad de asignación de recursos y una unidad de protección. Se asocian con procesos los siguientes:

- Un espacio de direcciones virtuales que soporta la imagen del proceso.
- Acceso protegido a procesadores, otros procesos, archivos y recursos E/S.

Dentro de un proceso puede haber uno más hilos, cada uno con:

- Un estado de ejecución por hilo.
- Un contexto de hilo que se almacena cuando no está en ejecución; una forma de ver a un hilo es como un contador de programa independiente dentro de un proceso.
- Una pila de ejecución.
- Por cada hilo, espacio de almacenamiento para variables locales.
- Acceso a la memoria y recursos de su proceso, compartido con todos los hilos de su mismo proceso.

En un modelo **monohilo** la representación de un proceso incluye su PCB y espacio de direcciones de usuario. En un entorno **multihilo** existen varias pilas para cada hilo, así como un PCB para cada hilo.

De esta forma, todos los hilos de un proceso comparten el estado y los recursos de ese proceso, residen en el mismo espacio de direcciones y tienen acceso a los mismos datos.

Los mayores beneficios de los hilos provienen de las consecuencias del rendimiento:

- Lleva mucho menos tiempo crear un nuevo hilo en un proceso existente que crear un proceso totalmente nuevo.
- Lleva menos tiempo finalizar un hilo que un proceso.
- Lleva menos tiempo cambiar entre dos hilos dentro del mismo proceso.
- Los hilos mejoran la eficiencia de la comunicación entre diferentes programas que están ejecutando.

A veces los hilos son también útiles en un solo procesador ya que ayudan a simplificar la estructura de programas que realizan varias funciones diferentes. Cuatro ejemplos de uso de hilos en un sistema de multiprocesamiento de un solo usuario:

- **Trabajo en primer y segundo plano.** Esta forma de trabajo incrementa la velocidad que se percibe de la aplicación, permitiendo al programa solicitar el siguiente mandato antes de que el anterior esté completado.
- **Procesamiento asíncrono.** Los elementos asíncronos se pueden implementar como hilos.
- **Velocidad de ejecución.** Un proceso multihilo puede computar una serie de datos mientras que lee los siguientes de un dispositivo. En un sistema multiprocesador puede estar ejecutándose simultáneamente múltiples hilos de un mismo proceso.
- **Estructura modular de programas.** Los programas que realizan diversas tareas o que tienen varias fuentes y destinos de entrada y salida, se pueden diseñar e implementar más fácilmente usando hilos.

En un sistema operativo que soporte hilos, la planificación y la activación se realizan a nivel de hilo. Existen, sin embargo, diversas acciones que afectan a todos los hilos de un proceso y que el sistema operativo debe gestionar a nivel de proceso.

2.2 FUNCIONALIDAD DE LOS HILOS

2.2.1 ESTADOS DE LOS HILOS

Hay cuatro operaciones básicas relacionadas con los hilos que están asociadas con un cambio de estado de hilo:

- **Creación.** Cuando se crea un nuevo proceso, también se crea un hilo de dicho proceso. Posteriormente, un hilo del proceso puede crear otro hilo dentro del mismo proceso, proporcionando un puntero a las instrucciones y los argumentos para el nuevo hilo. Al nuevo hilo se le proporciona su propio registro de contexto y espacio de pila y se coloca en la cola de Listos.
- **Bloqueo.** Cuando un hilo necesita esperar por un evento se bloquea, almacenando los registros de usuario, contador de programa y punteros de pila.
- **Desbloqueo.** Cuando sucede el evento por el que el hilo está bloqueado, el hilo se pasa a la cola de Listos.
- **Finalización.** Cuando se completa un hilo, se liberan su registro de contexto y pilas.

2.2.2 SINCRONIZACIÓN DE HILOS

Todos los hilos de un proceso comparten el mismo espacio de direcciones y otros recursos, por ejemplo, los archivos abiertos. Cualquier alteración de un recurso por cualquiera de los hilos, afecta al entorno del resto de los hilos del mismo proceso. Por tanto, es necesario sincronizar las actividades de los hilos para que no interfieran entre ellos o corrompan las estructuras de datos.

2.3 HILOS A NIVEL DE USUARIO Y DE NIVEL DE NÚCLEO

2.3.1 HILOS DE NIVEL DE USUARIO

En el entorno UTL (*user-level thread*), la aplicación gestiona todo el trabajo de los hilos y el núcleo no es consciente de la existencia de estos. Se consigue a través del uso de una biblioteca de hebras. La biblioteca se encarga de proporcionar operaciones para:

- Crear/finalizar hebras.
- Gestionar el modelo de estados de las hebras.
- Planificar hebras y salvar/cargar el contexto de hebra.
- Permitir la comunicación entre hebras.

El uso de ULT presenta las siguientes ventajas:

- El cambio de hilo no requiere privilegios porque todas las estructuras de datos de gestión de hilos están en el espacio de direcciones de usuario de un solo proceso. Esto ahorra la sobrecarga de dos cambios de modo.
- La planificación puede especificarse por parte de la aplicación.
- Los ULT pueden ejecutarse en cualquier sistema operativo.

Hay dos desventajas de los ULT:

- Muchas llamadas al sistema son bloqueantes. Por lo tanto, cuando un ULT realiza una llamada al sistema, no sólo se bloquea ese hilo, sino todos los hilos del proceso.
- En una estrategia ULT, una aplicación multihilo no puede sacar ventaja del multiproceso.

Una forma de solucionar las llamadas al sistema bloqueante es el **jacketing** (revestimiento), consiste en convertir una llamada al sistema bloqueante en una llamada al sistema no bloqueante.

2.3.2 HILOS A NIVEL DE NÚCLEO

En un entorno KLT (*kernel-level thread*), el núcleo gestiona todo el trabajo de gestión de hilos por lo que este tiene que mantener información para procesos y hebras. No hay código de gestión de hilos en la aplicación, solo una interfaz de programación de aplicación (API) para acceder a las utilidades de hilos de núcleo. El SO proporciona un conjunto de llamadas al sistema para la gestión de hebras. Muchas de las funciones que realiza el kernel son gestionadas mediante **hebras kernel**.

Este enfoque resuelve los dos principales inconvenientes del enfoque ULT:

- El kernel puede planificar distintas hebras de la misma tarea en diferentes procesadores.
- El bloqueo de una hebra de una tarea no provoca el bloqueo del resto de hebras pares.
- Las rutinas del kernel pueden ser multihebras.

La principal desventaja es que la transferencia de control de un hilo a otro del mismo proceso requiere un cambio de modo.

2.3.3 ENFOQUES COMBINADOS

En un sistema combinado, la creación de hilos se realiza por completo en el espacio de usuario, como la mayor parte de la planificación y sincronización de hilos dentro de una aplicación. Los múltiples ULT de una aplicación se asocian a un número (menor o igual) de KLT. El programador debe ajustar el número de KLT para una máquina y aplicación en particular para lograr los mejores resultados posibles.

En los enfoques combinados, múltiples de la misma aplicación pueden ejecutarse en paralelo en múltiples procesadores, y una llamada al sistema bloqueante no necesita bloquear el proceso completo.

2.4 OTRAS CONFIGURACIONES

Tabla 2.1. Relación entre Hilos y Procesos.

Hilos:Procesos	Descripción	Sistemas de Ejemplo
1:1	Cada hilo de ejecución es un único proceso con su propio espacio de direcciones y recursos.	Implementaciones UNIX tradicionales
M:1	Un proceso define un espacio de direcciones y pertenencia dinámica de recursos. Se pueden crear y ejecutar múltiples hilos en ese proceso.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	Un hilo puede migrar de un entorno de proceso a otro. Esto permite a los hilos moverse fácilmente entre distintos sistemas.	Ra (Clouds), Emerald
M:M	Combina atributos de M:1 y casos 1:M	TRIX

2.4.1 RELACIÓN MUCHOS A MUCHOS

Dominio. Entidad estática, que consiste en un espacio de direcciones y “puertos” a través de los cuales se pueden enviar y recibir mensajes.

Hilo. Ruta de ejecución, con una pila de ejecución, estado del procesador e información de planificación.

Al igual que en el enfoque multihilo, múltiples hilos podrían ejecutar en un solo dominio, proporcionando las ventajas discutidas anteriormente. Sin embargo, también es posible realizar la actividad de usuario o ejecutar aplicaciones en múltiples dominios. En este caso hay un hilo que se puede mover entre dominios.

El uso de un solo hilo en múltiples dominios está motivado por el deseo de proporcionar herramientas de estructuración al programador. Por ejemplo, un programa que hace uso de un subprograma de E/S. En un entorno multiprogramado que permite procesos creados por los usuarios, el programa principal podría generar un nuevo proceso para el manejo de E/S. Sin embargo, si el futuro progreso del programa principal depende del funcionamiento de la E/S, entonces el programa principal tendrá que esperar a la finalización del otro programa. Hay varias formas de implementar esta aplicación:

- El programa completo puede implementarse con un solo proceso. Existen desventajas relativas a la gestión de memoria.
- El programa principal y el subprograma de E/S podrían implementarse como dos procesos independientes. Este enfoque presenta sobrecarga de la creación del proceso subordinado.
- Tratar al programa principal y al subprograma de E/S como una sola actividad que se puede implementar en un solo hilo. Se podría crear un espacio de direcciones (dominio) para el programa principal y otro para el subprograma. De esta forma, el hilo se podría mover entre los dos espacios de direcciones.

La tercera opción podría ser la más eficiente para algunas aplicaciones.

2.4.2 RELACIÓN UNO A MUCHOS

Hilo. Unidad de actividad. Una vez creado, un hilo comienza ejecutando en un proceso a través de la invocación de un punto de entrada de un programa en ese proceso. Los hilos se pueden mover entre espacios de direcciones (de un proceso a otro) e incluso fuera de los límites de la máquina (de un computador a otro).

Este enfoque proporciona una forma eficiente de aislar a los usuarios y programadores de los detalles del entorno distribuido. La actividad del usuario puede ser representada como un solo hilo, y el movimiento de ese hilo entre máquinas puede ser gestionado por el SO gracias a información relativa al sistema, tal como la necesidad de acceder a un recurso remoto o el equilibrado de carga.

3. PLANIFICACIÓN EN SO

Se puede definir la planificación como el conjunto de políticas y mecanismos contruidos dentro del SO que gobiernan la forma de conseguir que los procesos se ejecuten en el procesador. El objetivo de la planificación de procesos es asignar procesos a ser ejecutados por el procesador o procesadores a lo largo del tiempo.

La planificación está asociada a cuestiones como la colocación de procesos, el orden de ejecución, la decisión del momento en que introducir un nuevo proceso en el sistema y la distribución de carga si existe más de un procesador. La planificación de recursos debe maximizar el tiempo de uso del procesador, minimizar el tiempo gastado por un usuario en la cola de espera, dar servicio a múltiples usuarios, asegurar el reparto equitativo y tener en cuenta las prioridades.

Tabla 3.1. Tipos de planificación.

Planificación a largo plazo	La decisión de añadir un proceso al conjunto de procesos a ser ejecutados.
Planificación a medio plazo	La decisión de añadir un proceso al número de procesos que están parcialmente o totalmente en memoria principal.
Planificación a corto plazo	La decisión por la que un proceso disponible será ejecutado por el procesador.
Planificación de la E/S	La decisión por la que un proceso que está pendiente de una petición de E/S será atendido por un dispositivo de E/S disponible.

La planificación afecta al rendimiento del sistema porque determina qué proceso esperará y qué proceso progresará. Fundamentalmente, la planificación es un problema de manejo de colas para minimizar el retardo en la cola y para optimizar el rendimiento en un entorno de colas.

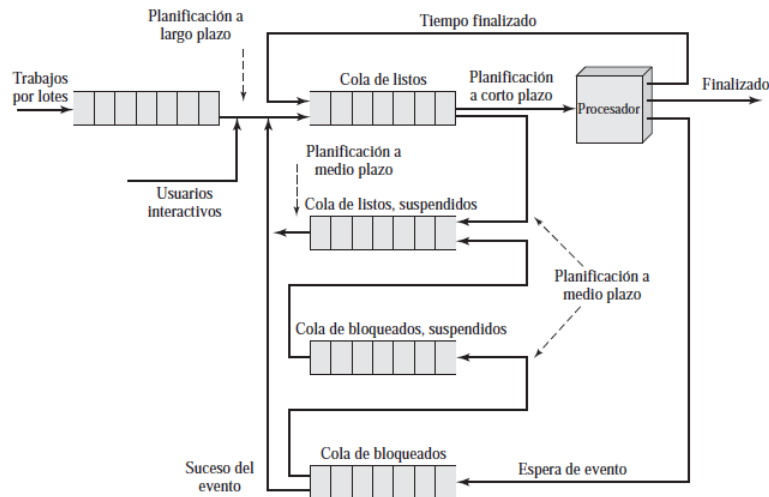


Figure 3.1. Diagrama de encolamiento para la planificación.

3.1 TIPOS DE PLANIFICACIÓN

3.1.1 PLANIFICACIÓN A LARGO PLAZO

El planificador a largo plazo determina qué programas se admiten en el sistema para su procesamiento. De esta forma, se controla el grado de multiprogramación. Una vez admitido, un programa de usuario se convierte en un proceso y se añade a la cola del planificador a corto plazo. En algunos sistemas, un proceso de reciente creación comienza en la zona de intercambio, en cuyo caso se añaden a la cola del planificador a medio plazo.

3.1.2 PLANIFICACIÓN A MEDIO PLAZO

En algunos SOs de tiempo compartido, a veces es necesario sacar procesos de la memoria (*swapping*). El planificador a medio plazo se encarga de devolver los procesos a memoria. La planificación a medio plazo es parte de la función de intercambio. En un sistema que no utiliza memoria virtual, la gestión de la memoria es también otro aspecto a tener en cuenta.

3.1.3 PLANIFICACIÓN A CORTO PLAZO

El planificador a corto plazo, o activador, se ejecuta mucho más frecuentemente que los otros planificadores. Se invoca siempre que ocurre un evento que puede conllevar el bloqueo del proceso actual y que puede proporcionar la oportunidad de expulsar al proceso actualmente en ejecución a favor de otro. Algunos ejemplos de estos eventos son:

- Interrupciones de reloj
- Interrupciones de E/S
- Llamadas al sistema
- Señales

DISPATCHER

El despachador es la función del SO que da el control de la CPU al proceso seleccionado por el planificador a corto plazo. Esto involucra:

- Cambio de contexto de registros desde el punto de vista de la CPU.
- Conmutación a modo usuario o a modo kernel dependiendo del nuevo contexto de registros.

Latencia de despacho. Tiempo que emplea el despachador en detener un proceso y comenzar a ejecutar otro.

El despachador puede actuar cuando:

- Un proceso no quiere (finaliza) o no puede continuar ejecutando instrucciones. En el segundo caso, la solicitud de recurso o comunicación por parte del proceso provoca que el SO lo pase a estado bloqueado.
- Un elemento del SO determina que el proceso no puede seguir ejecutándose.
- El proceso agota el quantum de tiempo asignado.
- Un suceso cambia al estado de un proceso de BLOQUEADO a LISTO.

3.2 MECANISMOS Y POLÍTICAS DE PLANIFICACIÓN

El objetivo principal de la planificación a corto plazo es asignar tiempo de procesador de tal forma que se optimicen uno o más aspectos del comportamiento del sistema. Para saber si un proceso obtiene un buen servicio, definiremos un conjunto de medidas: dado un **proceso P**, que necesita un **tiempo de servicio (ráfaga) r**:

- **Tiempo de respuesta (T)**. Tiempo transcurrido desde que se lanza una solicitud (entra en cola de listos) hasta que se produce la primera respuesta.
- **Tiempo de espera (M)**. Tiempo que un proceso ha estado esperando en la cola de listos: $T - r$.
- **Penalización (P)**. T/r
- **Índice de respuesta (R)**. Fracción de tiempo que P está recibiendo servicio: r/T .
- **Tiempo de retorno (turnaround time)**. Tiempo transcurrido desde que se lanza un proceso hasta que finaliza.
- **Fecha tope (deadlines)**. Cuando se puede especificar la fecha tope de un proceso, el planificador debe subordinar otros objetivos al de maximizar el porcentaje de fechas topes conseguidas.
- **Tiempo de núcleo**. Tiempo perdido por el SO tomando decisiones que afectan a la planificación de procesos y haciendo los cambios de contexto necesarios. En un sistema eficiente debe representar entre el 10% y el 30% del total del tiempo del procesador.
- **Tiempo de inactividad**. Tiempo en el que la cola de ejecutables está vacía y no se realiza ningún trabajo productivo.
- **Rendimiento**. La política de planificación debería de intentar maximizar el número de procesos completados por unidad de tiempo.
- **Utilización del procesador**. Es el porcentaje de tiempo que el procesador está ocupado.
- **Equidad**. En ausencia de orientación de los usuarios o proporcionada por otro sistema, los procesos deben ser tratados de la misma manera, y ninguno debe sufrir inanición.
- **Imposición de prioridades**. Cuando se asignan prioridades a los procesos, la política del planificador debería favorecer a los procesos con prioridades más altas.
- **Equilibrado de recursos**. La política del planificador debería mantener ocupados los recursos del sistema. Los procesos que utilicen poco los recursos que en un determinado momento están sobre utilizados, deberían ser favorecidos.

Las políticas de planificación se comportan de forma distinta dependiendo de la clase de procesos (cortos o largos). Ninguna política es completamente satisfactoria, cualquiera mejora en una clase de procesos es perder eficiencia en los de otra clase. Podemos clasificarlas en:

- **No apropiativas (no expulsivas)**. Una vez que se le asigna el procesador a un proceso, no se le puede retirar hasta que éste voluntariamente lo deje, finaliza o se bloquea. No son adecuados para sistemas de propósito general. Además, no proporcionan un buen reparto del procesador.
- **Apropiativas (expulsivas)**. Un proceso ejecutando en un determinado momento puede ser interrumpido y pasado al estado de listo por el sistema operativo. La decisión de expulsar puede ser tomada cuando llega un nuevo proceso, cuando llega una interrupción que pasa a un proceso bloqueado a estado listo, o periódicamente, basándose en las interrupciones del reloj. No son adecuados para sistemas multiusuarios.

Las políticas expulsivas tienen más sobrecarga que las no expulsivas, pero pueden proporcionar mejor servicio a la población total de procesos, ya que previenen que cualquier proceso pueda monopolizar el procesador durante mucho tiempo.

3.2.1 ALGORITMOS DE PLANIFICACIÓN NO APROPIATIVOS

PRIMERO EN LLEGAR, PRIMERO EN SERVIRSE (FCFS/FIFO)

Los procesos son servidos según el orden de llegada a la cola de ejecutables. Es un algoritmo fácil de implementar, pero pobre en cuanto a prestaciones. Todos los procesos pierden la misma cantidad de tiempo esperando en la cola de ejecutables independientemente de sus necesidades.

El problema de FCFS es que tiende a favorecer procesos limitados por el procesador sobre los limitados por la E/S. Cuando hay procesos bloqueados esperando un evento de E/S y este se cumple, pasarán al estado Listos. Mientras haya un proceso ejecutándose y no se bloquee, los dispositivos de E/S quedarán ociosos. Igualmente, si los procesos limitados por el procesador también están bloqueados, el procesador quedará ocioso.

EL MÁS CORTO PRIMERO (SHORTEST JOB FIRST)

Cuando el procesador queda libre, selecciona el proceso que requiera un tiempo de servicio menor. Si existen dos o más procesos en igualdad de condiciones se sigue FCFS.

El principal problema es que se necesita conocer explícitamente el tiempo estimado de ejecución y si el tiempo estimado es mucho menor que el tiempo actual de ejecución, el sistema podría abortar el trabajo. Además, disminuye el tiempo de respuesta para los procesos cortos y discrimina a los largos. Existe el riesgo de inanición para los procesos más largos, si hay una llegada constante de procesos más cortos.

PLANIFICACIÓN BASADA EN PRIORIDADES

La prioridad es asignada por el SO cuando se crea el trabajo. Se asigna la CPU al proceso con mayor prioridad. En el caso de haber múltiples procesos con la misma prioridad, deberá usarse otro algoritmo para decidir cuál se elige. Se puede implementar con modalidades apropiativas y no apropiativas.

Para evitar la inanición de los procesos con baja prioridad, se usan mecanismos de **envejecimiento**. Estos se encargan de aumentar paulatinamente la prioridad a los procesos que lleven un tiempo esperando a ser ejecutados.

PRIMERO EL DE MAYOR TASA DE RESPUESTA (HIGHEST RESPONSE RATIO NEXT, HRRN)

Cuando se completa o bloquea el proceso actual, elegir el proceso listo con el mayor valor de tasa de respuesta. Este enfoque tiene en cuenta la edad del proceso. Mientras que favorece a los procesos más cortos, el envejecimiento sin servicio incrementa la tasa, por lo que un proceso más largo podría competir con los trabajos más cortos.

Tasa de respuesta (R), tiempo invertido esperando por el procesador (w), tiempo de servicio esperado (s)

$$R = \frac{(w + s)}{s}$$

3.2.2 ALGORITMOS DE PLANIFICACIÓN APROPIATIVOS

TURNO ROTATORIO (ROUND ROBIN)

Cuando se asigna al procesador un proceso, se le concede un intervalo de tiempo máximo (**quantum**), de manera que si se cumple el plazo se realiza un cambio de contexto involuntario, pasando el proceso al final de la cola de preparados. Si el proceso finaliza o se bloquea antes de agotar el quantum, libera la CPU. Se toma el siguiente proceso de la cola de ejecutables y se le asigna un quantum completo.

Proporciona un reparto más equitativo del procesador y acota el tiempo de espera. Si un proceso está en la cola de Listos, y tiene delante N procesos, incluyendo el que está en ejecución, en un tiempo máximo de NxT el proceso estará ejecutándose.

Si se utiliza un quantum demasiado grande el tiempo de espera es mayor, y si es demasiado pequeño genera más sobrecarga. Una regla práctica es usar un quantum tal que los procesos con uso intensivo de E/S no consuman su quantum la mayoría de veces. De esta forma, se evita el cambio de contexto involuntario.

PRIMERO EL DE MENOR TIEMPO RESTANTE (SHORTEST REMAINING TIME FIRST)

Es la versión expulsiva del algoritmo SJF. Cada vez que entra un proceso a la cola de listos se comprueba si su tiempo de servicio es menor que el tiempo de servicio que le queda al proceso que está ejecutándose:

- Si es menor, se realiza un cambio de contexto y el proceso con menor tiempo de servicio es el que se ejecuta.
- No es menor, continúa el proceso que estaba ejecutándose.

El tiempo de espera es significativamente mejor que en la versión no apropiativa, pero no proporciona un reparto equitativo del procesador, ni implementa grados de prioridad

COLAS MÚLTIPLES (MULTILEVEL QUEUE)

La cola de listos se divide en varias colas y cada proceso es asignado permanentemente a una cola concreta. Cada cola puede tener su propio algoritmo de planificación. Este modelo se caracteriza por:

- Número de colas de procesos. Cuando se crea un proceso, pertenecerá a la misma clase que el proceso padre.
- El algoritmo de planificación que se usa para repartir el procesador entre los procesos de una determinada cola.
- El esquema de planificación que se usa para repartir el procesador entre las distintas colas
 - Planificación con prioridades fijas,
 - Tiempo compartido entre colas. Cada cola obtiene cierto tiempo de CPU.

COLAS MÚLTIPLES CON REALIMENTACIÓN (MULTILEVEL FEEDBACK QUEUE)

El So puede cambiar de cola a un proceso dependiendo de su evolución. Este modelo requiere definir el siguiente parámetro, además de los definidos anteriormente: Método utilizado para determinar cuándo trasladar un proceso a otra cola. Mide el tiempo de ejecución el comportamiento real de los procesos.

RETROALIMENTACIÓN (FEEDBACK)

Otra forma de establecer una preferencia para los trabajos más cortos es penalizar a los trabajos que han estado ejecutándose más tiempo. Es decir, si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento.

La planificación se realiza con expulsión (por rodajas de tiempo), y se utiliza un mecanismo de prioridades dinámico. Cada vez que un proceso es expulsado, se sitúa en la siguiente cola de menor prioridad. Un proceso corto se completará rápidamente, uno más largo irá degradándose gradualmente. De esta forma, se favorece a los procesos nuevos más cortos. Dentro de cada cola excepto en la de menor prioridad, se utiliza un mecanismo FCFS.

4. PLANIFICACIÓN MULTIPROCESADOR Y DE TIEMPO REAL

4.1 PLANIFICACIÓN EN MULTIPROCESADORES

Podemos clasificar los sistemas multiprocesador como sigue:

- Débilmente acoplado o multiprocesador distribuido, o clúster. Una colección de sistemas relativamente autónomos; cada procesador tiene su memoria principal y canales de E/S.
- Procesadores de funcionalidad especializada.
- Procesamiento fuertemente acoplado. Conjunto de procesadores que comparten la memoria principal y están bajo el control integrado de un único SO.

4.1.1 PLANIFICACIÓN DE PROCESOS

Para sistemas multiprocesadores no se necesitan nuevos algoritmos de planificación, pero sí es necesario incorporar ciertos ajustes a los esquemas de planificación para que tengan en cuenta aspectos como:

- La afinidad natural de los procesos hacia el último procesador donde ejecutaron.
- La afinidad estricta de los procesos que les permite restringir en qué subconjunto de los procesadores desea ejecutar.
- Evitar congestión en el acceso a las estructuras de datos del planificador que puede causar un cuello de botella en la ejecución del planificador.

Hay dos tipos de planificadores para multiprocesadores: los que utilizan una cola única para todo el sistema y aquellos que usan una cola por procesador.

4.1.2 PLANIFICACIÓN DE HILOS

Como hemos visto el concepto de ejecución se separa del resto de la definición de un proceso. Una aplicación puede ser implementada como un conjunto de hilos, que cooperan y ejecutan de forma concurrente en el mismo espacio de direcciones.

Los hilos pueden explotar paralelismo real dentro de una aplicación. Si están ejecutándose simultáneamente en procesadores separados, es posible una mejora drástica de sus prestaciones. Sin embargo, puede demostrarse que para aplicaciones que necesitan una interacción significativa entre hilos (grano medio), pequeñas diferencias en la gestión y planificación de hilos puede dar lugar a un impacto significativo.

Entre las muchas propuestas para la planificación multiprocesador de hilos y la asignación a procesadores, destacan cuatro enfoques generales:

- **Compartición de carga.** Los procesos no se asignan a un procesador particular.
- **Planificación en pandilla.** Un conjunto de hilos relacionados que se planifica para ejecutar sobre un conjunto de procesadores al mismo tiempo, en relación 1:1.
- **Asignación de procesador dedicado.** Opuesto a compartición de carga y proporciona una planificación implícita definida por la asignación de hilos a procesadores.
- **Planificación dinámica.** El número de hilos de un proceso puede cambiar durante el curso de su ejecución.

COMPARTICIÓN DE CARGA

Es el enfoque más simple, tiene algunas ventajas:

- La carga se distribuye uniformemente entre los procesadores.
- No se precisa un planificador centralizado; cuando un procesador queda disponible, la rutina de planificación del SO se ejecuta en dicho procesador.
- La cola global puede organizarse y ser accesible usando cualquiera de los esquemas expuestos.

Se analizan tres versiones diferentes de compartición de carga:

- **Primero en llegar, primero en ser servido (FCFS).** Cuando un procesador pasa a estar ocioso coge el siguiente hilo listo, que ejecuta hasta que finaliza o se bloquea.
- **Menor número de hilos primero.** La cola compartida se organiza como una cola de prioridad, con la mayor prioridad para los hilos e los trabajos con el menor número de hilos no planificados.
- **Menor número de hilos primero con expulsión.** Se le da mayor prioridad a los trabajos con el menor número de hilos no planificados. Si llega un trabajo con menor número de hilos que el que está ejecutándose se expulsarán los hilos pertenecientes al trabajo planificado.

La compartición de carga tiene varias desventajas:

- La cola central ocupa una región de memoria a la que debe accederse de manera que se cumpla exclusión mutua. Esto puede causar un cuello de botella.
- Es poco probable que los hilos expulsados retomen su ejecución en el mismo procesador. Si cada procesador está equipado con caché loca, ésta se volverá ineficaz.
- Si todos los hilos se tratan como un conjunto común de hilos, es poco probable que todos los hilos de un programa ganen acceso a procesadores a la vez.

A pesar de las potenciales desventajas, este es uno de los esquemas más utilizados en los multiprocesadores actuales.

PLANIFICACIÓN EN PANDILLA

El concepto de planificar un conjunto de procesos simultáneamente sobre un conjunto de procesadores es anterior al uso de hilos. Tiene los siguientes beneficios:

- Si se ejecutan en paralelo procesos estrechamente relacionados, puede reducirse el bloqueo por sincronización, pueden necesitarse menos cambios de proceso y las prestaciones aumentarán.
- La sobrecarga de planificación puede reducirse dado que una decisión única afecta a varios procesadores y procesos a la vez.

Es útil para aplicaciones cuyo rendimiento se degrada cuando alguna parte no puede ejecutarse.

ASIGNACIÓN DE PROCESADOR DEDICADO

Dedicar un grupo de procesadores a una aplicación durante toda su duración. Cada uno de los hilos de la aplicación se asigna a un procesador que permanece dedicado al hilo. Pueden realizarse dos observaciones en la defensa de esta estrategia:

- En un sistema altamente paralelo, con decenas de procesadores, la utilización del procesador deja de ser tan importante como la eficacia o rendimiento.
- Evitar totalmente el cambio de proceso durante la vida de un programa lleva a una sustancial mejora de velocidad.

PLANIFICACIÓN DINÁMICA

Algunas aplicaciones pueden proporcionar, en el lenguaje o en el sistema, herramientas que permitan que el número de hilos del proceso pueda ser alterado dinámicamente. Esto permitiría que el SO ajustase la carga para mejorar la utilización.

La responsabilidad de planificación del sistema operativo está limitada fundamentalmente a la ubicación del procesador y se realiza de acuerdo a la siguiente política. Cuando un trabajo solicita uno o más procesadores:

1. Si hay procesadores ociosos, utilizarlos
2. En otro caso, si el trabajo acaba de llegar, ubicarlo en un único procesador quitándoselo a cualquier trabajo que tenga más de un procesador.
3. Si no puede satisfacerse cualquier parte de la solicitud, mantenerla pendiente hasta que un procesador pase a estar disponible, o el trabajo rescinda la solicitud.
4. Examinar la cola actual de solicitudes de procesador no satisfechas. Asignar un único procesador a cada trabajo e la lista que no tenga actualmente procesadores, Luego volver a examinar la lista, volviendo a asignar el resto de los procesadores siguiendo la estrategia FCFS.

4.2 PLANIFICACIÓN DE TIEMPO REAL

La exactitud del sistema no depende solo del resultado lógico de un cálculo sino también del instante en que se produzca el resultado. Las tareas (o procesos) intentan controlar o reaccionar ante sucesos que se producen en "tiempo real" (eventos) y que tienen lugar en el mundo exterior. Las características de las tareas de tiempo real son:

- **Duro.** Tienen que terminar obligatoriamente antes de que venza su plazo de ejecución para que no haya consecuencias negativas para el sistema.
- **Suave.** La tarea tiene un tiempo límite, pero no es obligatorio cumplirlo.
- **Periódicas.** Se sabe cada cuánto tiempo se tiene que ejecutar.
- **Aperiódicas.** Tiene un plazo en el que debe comenzar o acabar o restricciones al respecto a esos tiempos, pero son impredecibles.

Los distintos enfoques de la planificación dependen de cuando el sistema realiza el análisis de planificabilidad; si lo hace, de si se realiza estáticamente o dinámicamente; y de si el resultado del análisis produce un plan de planificación de acuerdo al cual se desarrollarán las tareas en tiempo de ejecución. En base a estas consideraciones se identifican las siguientes clases de algoritmos:

- **Enfoques estáticos dirigidos por tablas.** Análisis estático que genera una planificación que determina cuando empezará cada tarea.
- **Enfoques estáticos expulsivos dirigidos por prioridad.** Análisis estático que no genera una planificación, solo se usa para dar prioridad a las tareas.
- **Enfoques dinámicos basados en un plan.** Se determina la viabilidad en tiempo de ejecución: se acepta una nueva tarea si es posible satisfacer sus restricciones de tiempo.
- **Enfoques dinámicos de mejor esfuerzo.** No se hace análisis de viabilidad. El sistema intenta cumplir todos los plazos y aborta ejecuciones si su plazo ha terminado.

4.3 PROBLEMA DE INVERSIÓN DE PRIORIDAD

Se da en un esquema de planificación expulsivo basado en prioridades, pero es particularmente relevante en el contexto de la planificación de tiempo real. Sucede cuando una tarea de mayor prioridad espera por una tarea de menor prioridad debido al bloqueo de un uso exclusivo.

Una situación más seria es la **inversión de prioridad ilimitada**, en la cual la duración de la inversión de prioridad depende no solo del tiempo necesario para conseguir el recurso compartido sino también de las acciones impredecibles de otras tareas no relacionadas.

Se usan dos enfoques para evitar la inversión de prioridad ilimitada: el protocolo de herencia y el protocolo de techo de prioridad.

Herencia de prioridad. La idea básica es que una tarea de menor prioridad heredará la prioridad de cualquier tarea de mayor prioridad pendiente de un recurso que compartan. Este cambio de prioridad sucede tan pronto como la tarea de mayor prioridad se bloquea en el recurso; y debe finalizar cuando la tarea de menor prioridad libera el recurso.

Techo de prioridad. Se asocia una prioridad con cada recurso. La prioridad asociada con un recurso es un nivel más alto que la prioridad de su usuario más prioritario. Entonces, el planificador asigna esta prioridad a cualquier tarea que acceda al recurso. Cuando la tarea terminada de usar el recurso, su prioridad se restablece al valor normal.

En ambos, la tarea menos prioritaria vuelve a tener valor de prioridad que tenía cuando libera el recurso.

5. PLANIFICACIÓN EN LINUX

5.1 IMPLEMENTACIÓN DEL CONCEPTO DE PROCESO/HILO

El núcleo identifica a los procesos (tareas - tasks) por su PID (Process Identifier). En Linux, proceso es la entidad que se crea con la llamada al sistema **fork()** (excepto el proceso 0) y **clone()**.

Procesos especiales que existen durante la vida del sistema:

- Proceso 0: creado "a mano" cuando arranca el sistema. Crea al proceso 1.
- Proceso 1 (init): antecesor de cualquier proceso del sistema.

5.1.1 PCB EN LINUX: STRUCT TASK_STRUCT

En Linux una tarea (task) engloba el concepto de proceso e hilo. El Kernel almacena la lista de procesos como una lista circular doblemente enlazada: **task list**. Cada elemento es un descriptor de proceso (PCB) definido en `</include/linux/sched.h >`

5.1.2 ESTADOS DE UNA TAREA

El campo state de task_struct especifica el estado actual de un proceso.

Tabla 5.1. Estados de una tarea.

Ejecución (task_running)	Se corresponde con dos: ejecutándose o preparado para ejecutarse (en la cola de procesos preparados)
Interrumpible (task_interruptible)	El proceso está bloqueado y sale de este estado cuando ocurre el suceso por el cual está bloqueado o porque le llegue una señal
No interrumpible (task_uninterruptible)	El proceso está bloqueado y sólo cambiará de estado cuando ocurra el suceso que está esperando (no acepta señales)
Parado (task_stopped)	El proceso ha sido detenido y sólo puede reanudarse por la acción de otro proceso (ejemplo, proceso parado mientras está siendo depurado)
TASK_TRACED	El proceso está siendo traceado por otro proceso
Zombie (exit_zombie)	El proceso ya no existe pero mantiene la estructura task hasta que el padre haga un wait() (EXIT_DEAD)

5.1.3 ÁRBOL DE PROCESOS

Si el proceso P0 hace una llamada al sistema `fork()`, genera el proceso P1, se dice que P0 es el proceso padre y P1 es el hijo. Si el proceso P0 hace varias llamadas al sistema `fork()` genera varios procesos hijo y la relación entre ellos es de **hermanos** (*sibling*). Todos los procesos son **descendientes** del proceso `init` (cuyo PID es 1)

Cada `task_struct` tiene un puntero a:

- La `task_struct` de su padre (**parent**): `struct task_struct *parent`
- Una lista de hijos (**children**): `struct list_head children`
- Una lista de sus hermanos (**sibling**): `struct list_head sibling`

El kernel dispone de procedimientos eficaces para las acciones usuales de manipulación de la lista.

5.1.4 IMPLEMENTACIÓN DE HILOS

Desde el punto de vista del kernel no hay distinción entre hebra y proceso. Linux implementa el concepto de hebra como un proceso sin más, que simplemente comparte recursos con otros procesos. Cada hebra tiene su propia `task_struct`. La llamada al sistema **clone()** crea un nuevo proceso o hebra dependiendo del parámetro **flags**.

5.1.5 HEBRAS KERNEL

A veces es útil que el kernel realice operaciones en segundo plano, para lo cual se crean hebras kernel. Las hebras kernel no tienen un espacio de direcciones. Se ejecutan únicamente en el espacio del kernel y son planificadas y pueden ser expropiadas. Terminan cuando realizan una operación `do_exit()` o cuando otra parte del kernel provoca su finalización.

5.1.6 CREACIÓN DE TAREAS

`fork()` → `clone()` → `do_fork()` → `copy_process()`

Actuación del **copy_process()**:

1. Crea la estructura **thread_info** (pila kernel) y la **task_struct** para la nueva tarea con los valores de la tarea actual.
2. Asigna valores iniciales a los campos de la **task_struct** de la tarea hija que deban tener valores distintos a los de la tarea padre.
3. Se establece el campo estado de la tarea hija `TASK_UNINTERRUPTIBLE` mientras se realizan las restantes acciones.
4. Se establecen valores adecuados para los flags de la **task_struct** de la tarea hija:
 - flag `PF_SUPERPRIV=0` (la tarea no usa privilegio de superusuario)
 - flag `PF_FORKNOEXEC=1` (el proceso ha hecho `fork()` pero no `exec()`)
5. Se llama a **alloc_pid()** para asignar un PID a la nueva tarea.
6. Según cuáles sean los flags pasados a **clone()**, duplica o comparte recursos como archivos abiertos, información de sistemas de archivos, manejadores de señales, espacio de direccionamiento del proceso,...
7. Se establece el estado de la tarea hija a **TASK_RUNNING**.
8. Finalmente **copy_process()** termina devolviendo un puntero a la **task_struct** de la tarea hija

5.1.7 TERMINACIÓN DE TAREAS

Cuando un proceso termina, el kernel libera todos sus recursos y notifica al padre su terminación. Normalmente un proceso termina cuando:

- Realiza la llamada al sistema `exit()`.
 - De forma explícita: el programador incluyó esa llamada en el código del programa.
 - De forma implícita: el compilador incluye automáticamente una llamada a `exit()` cuando `main()` termina.
- Recibe una señal que tiene la disposición por defecto de terminar al proceso (Term).

Actuación de `do_exit()`:

1. Activa el flag **PF_EXITING** de **task_struct**.
2. Para cada recurso que esté utilizando el proceso, se decrementa el contador correspondiente que indica el nº de procesos que lo están utilizando. Si contador es igual a 0, se realiza la operación de destrucción oportuna sobre el recurso, por ejemplo, si fuera una zona de memoria, se liberaría el descriptor de memoria (**mm_struct**) correspondiente.
3. El valor que se pasa como argumento a `exit()` se almacena en el campo **exit_code** de **task_struct** (Esta es la información de terminación para que el padre pueda hacer un `wait()` o `waitpid()` y recogerla).
4. Se manda una señal al padre (**SIGCHILD**) indicando la finalización de su hijo.
5. Si el proceso aún tiene hijos, se establece como padre de dichos hijos al proceso init (PID=1).

Nota: Dependiendo de las características del grupo de procesos al que pertenezca el proceso, podría ponerse como padre a otro miembro de ese grupo de procesos.

6. Se establece el campo **state** de **task_struct** a **EXIT_ZOMBIE**.
7. Se llama a `schedule()` para que el planificador elija un nuevo proceso a ejecutar.

Nota: Puesto que este es el último código que ejecuta un proceso, `do_exit()` nunca retorna.

5.2 PLANIFICACIÓN DE CPU EN LINUX

Cada clase de planificación tiene un rango de prioridad que determina el número de colas de prioridad, una por número dentro del rango. Se usa un algoritmo de planificación entre las clases de planificación por prioridades apropiativo. Cada clase de planificación usa una o varias políticas para gestionar los procesos incluidos en cada una de sus colas.

La planificación no opera únicamente sobre el concepto de proceso, sino que maneja conceptos más amplios en el sentido de manejar grupos de procesos: Entidad de planificación (`sched_entity`).

5.2.1 POLÍTICAS DE PLANIFICACIÓN

Políticas manejadas por el planificador CFS – **fair_sched_class**:

- **SCHED_NORMAL**. Se aplica a los procesos normales de tiempo compartido
- **SCHED_BATCH**. Tareas menos importantes, menor prioridad. Son procesos batch con gran proporción de uso de CPU para cálculos.
- **SCHED_IDLE**. Tareas de este tipo tienen una prioridad mínima para ser elegidas para asignación de CPU

Políticas manejadas por el planificador de tiempo real – **rt_sched_class**:

- **SCHED_RR**. Uso de una política Round-Robin.
- **SCHED_FIFO**. Uso de una política FCFS.

5.2.2 PRIORIDADES

Clases de prioridades en Linux:

- **SCHED_RR**. Hilos en tiempo real con planificación Round-Robin.
- **SCHED_OTHER**. Hilos que no son de tiempo real y otros.
- **SCHED_FIFO**. Hilos en tiempo real y con política FCFS.
 - El sistema no interrumpe ejecución excepto:
 - Pasa a Listo en otro con más prioridad.
 - Se bloquea por espera de evento
 - Abandona el procesador por ejecución `sched_yield`.
 - Cuando se interrumpe en ejecución pasa a cola asociada a su prioridad.
 - Cuando el que está Listo tiene mayor prioridad que el que está en ejecución, pasa a Ejecutar el de mayor prioridad. Si hay varios se escoge por mayor tiempo de espera.

Siempre se cumple que el proceso que está en ejecución es el más prioritario. Intervalo de valores de prioridad para **prio**:

- **[0, 99]** Prioridades para procesos de tiempo real.
- **[100, 139]** Prioridades para los procesos normales o regulares

5.2.3 PLANIFICADOR PERIÓDICO

Se implementa en **`scheduler_tick()`**, función llamada automáticamente por el kernel con frecuencia (HZ) constante cuyos valores están normalmente en el rango 1000 y 100Hz).

Tareas principales:

- Actualizar estadísticas del kernel.
- Activar el método de planificación periódico de la clase de planificación a que corresponde el proceso actual (**`task_tick()`**). Cada clase de planificación tiene implementada su propia función **`task_tick()`** (contabiliza el tº de CPU consumido).

Si hay que replanificar, el planificador de la clase concreta activará el flag **TIF_NEED_RESCHED** asociado al proceso en su **`thread_info`**, y provocará que se llame al planificador principal.

5.2.4 PLANIFICADOR PRINCIPAL: **SCHEDULE()**

Se implementa en la función **`schedule()`**, invocada en diversos puntos del kernel para tomar decisiones sobre asignación de la CPU. La función **`schedule()`** es invocada de forma explícita cuando un proceso se bloquea o termina. El kernel chequea el flag **TIF_NEED_RESCHED** del proceso actual al volver al espacio de usuario desde modo kernel.

Ya sea justo antes de volver de una llamada al sistema, de una rutina de servicio de interrupción (RSI) o de una rutina de servicio de excepción (RSE); si está activo este flag se invoca a **`schedule()`**.

Actuación de **`schedule()`**:

- Determina la actual **`runqueue`** y establece el puntero **`prev`** a la **`task_struct`** del proceso actual.
- Actualiza estadísticas y limpia el flag **TIF_NEED_RESCHED**.
- Si el proceso actual va a un estado **TASK_INTERRUPTIBLE** y ha recibido la señal que esperaba, se establece su estado a **TASK_RUNNING**.
- Se llama a **`pick_next_task`** de la clase de planificación a la que pertenezca el proceso actual para que se seleccione el siguiente proceso a ejecutar; y se establece **`next`** con el puntero a la **`task_struct`** de dicho proceso seleccionado.
- Si hay cambio en la asignación de CPU, se realiza el cambio de contexto llamando a **`context_switch()`** ¡Nuestro **`dispatcher()`**!

5.2.5 CLASES DE PLANIFICACIÓN

CLASE DE PLANIFICACIÓN CFS

Idea general: repartir el tiempo de CPU de forma imparcial, garantizando que todos los procesos se ejecutarán y, dependiendo del número de procesos en cola, asignarles más o menos tiempo de uso de CPU. Mantiene datos sobre los tiempos consumidos por los procesos. El kernel calcula un peso para cada proceso. Cuanto mayor sea el valor de la prioridad estática de un proceso, menor será el peso que tenga. **vruntime** (virtual runtime) de una entidad es el **tiempo virtual** que un proceso ha consumido y se calcula a partir del tiempo real que el proceso ha hecho uso de la CPU, su prioridad estática y su peso. El valor **vruntime** del proceso actual se actualiza:

- Periódicamente (el planificador periódico ajusta los valores de tiempo de CPU consumido)
- Cuando llega un nuevo proceso ejecutable.
- Cuando el proceso actual se bloquea.

Cuando se tiene que decidir qué proceso ejecutar a continuación, se elige el que tenga un valor menor de **vruntime**. Para implementar esto CFS utiliza un red black tree (**rbtree**), que es una estructura de datos árbol binario que almacena nodos identificados por una clave, **vruntime**, y que permite una búsqueda eficiente por valor de clave.

Cuando un proceso va a entrar en estado bloqueado:

1. Se añade a una cola asociada con el motivo del bloqueo.
2. Se establece el estado del proceso a **TASK_INTERRUPTIBLE** o a **TASK_NONINTERCEPTIBLE** según esté clasificado el motivo del bloqueo.
3. Se quita del **rbtree** de procesos ejecutables la referencia a la **task_struct** del proceso.
4. Se llama a **schedule()** para que se elija un nuevo proceso a ejecutar .

Cuando un proceso vuelve del estado bloqueado:

1. Se cambia su estado a ejecutable, **TASK_RUNNING**.
2. Se elimina de la cola de bloqueo en que estaba.
3. Se añade al **rbtree** de procesos ejecutables.

CLASE DE PLANIFICACIÓN DE TIEMPO REAL

Se define la clase de planificación **rt_sched_class**.

Los procesos de tiempo real son más prioritarios que los normales, y mientras existan procesos de tiempo real ejecutables éstos serán elegidos frente a los normales. Un proceso de tiempo real queda determinado por la prioridad que tiene cuando se crea. El kernel no incrementa o disminuye su prioridad en función de su comportamiento. Las políticas de planificación de tiempo real **SCHED_RR** y **SCHED_FIFO** posibilitan que el kernel Linux pueda tener un comportamiento soft real-time (Sistemas de tiempo real no estricto).

Al crear el proceso también se especifica la política bajo la cual se va a planificar y existe una llamada al sistema para cambiar la política asignada.

5.2.6 PLANIFICACIÓN DE CPU EN SMP

Para realizar correctamente la planificación en un entorno SMP (multiprocesador), el kernel deberá tener en cuenta:

- Se debe repartir equilibradamente la carga entre las distintas CPUs.
- Se debe tener en cuenta la afinidad de una tarea con una determinada CPU.
- El kernel debe ser capaz de migrar procesos de una CPU a otra (puede ser una operación costosa).

Periódicamente una parte del kernel deberá comprobar que se da un equilibrio entre las cargas de trabajo de las distintas CPUs y si detecta que una tiene más procesos que otra, reequilibra pasando procesos de una CPU a otra.

Gestión de Memoria

1. GESTIÓN DE MEMORIA PRINCIPAL

En un sistema multiprogramado, la parte de usuario de la memoria se debe subdividir posteriormente para acomodar varios procesos, el SO es el encargado de la tarea de subdivisión y a esta tarea se le llama **gestión de la memoria**. El diseño de la memoria de un computador debe basarse en 3 aspectos: capacidad, velocidad y coste. Requisitos de la gestión de memoria:

- **Reubicación:** no se puede conocer de manera anticipada dónde se va a colocar un programa y se debe permitir que los programas se puedan mover en la memoria principal -> intercambio o swap.
- **Protección:** cada proceso debe protegerse contra interferencias no deseadas por parte de otros procesos, sean accidentales o intencionadas.
- **Compartición:** cualquier mecanismo de protección debe tener la flexibilidad de permitir a varios procesos acceder a la misma porción de memoria principal.
- **Organización lógica:** segmentación para tratar de forma efectiva los programas de usuario y los datos en forma de módulo.
- **Organización física:** tarea de mover la información entre los dos niveles de la memoria es una responsabilidad del sistema (memoria principal y secundaria)

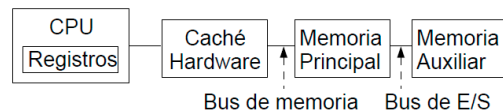
1.1 JERARQUÍA DE MEMORIA

Dos principios sobre memoria:

- **Menor cantidad,** acceso más rápido
- **Mayor cantidad,** menor coste por byte

Así, los elementos frecuentemente accedidos se ponen en memoria rápida, cara y pequeña; el resto, en memoria lenta, grande y barata

Se puede organizar los datos mediante la jerarquía de manera que el porcentaje de accesos a cada nivel sucesivamente más bajo es considerablemente menor que al nivel inferior. Normalmente el tipo de memoria más rápida, pequeña y costosa son los registros del procesador.



1.2 CONCEPTOS SOBRE CACHÉS

Caché. copia que puede ser accedida más rápidamente que el original

Idea: hacer los casos frecuentes eficientes, los caminos infrecuentes no importan tanto

Acierto de caché: ítem en la caché

Fallo de caché: ítem no en caché; hay que realizar la operación completa

$$\text{Tiempo de Acceso Efectivo (TAE)} = \text{Probabilidad_acierto} * \text{coste_acierto} + \text{Probabilidad_fallo} * \text{coste_fallo}$$

Funciona porque los programas no son aleatorios, explotan la **localidad** -> **principio de localidad**

Tamaño de la caché: si es lo suficientemente pequeño puede tener un impacto significativo en el rendimiento.

Tamaño del bloque: unidad de datos que se intercambia entre la caché y la memoria principal. Según se incrementa, aumenta la tasa de aciertos; pero cuando es excesivamente grande, disminuye, ya que la probabilidad de volver a usar los datos recientemente leídos se hace menor que la de utilizar los que se van a expulsar para dejar sitio al nuevo bloque.

Función de correspondencia: determina qué posición de la caché ocupará un bloque que se introduce en ella.

Algoritmo de reemplazo: selecciona, dentro de las restricciones de la función de correspondiente, qué bloque reemplazar cuando uno nuevo va a cargarse y no quedan huecos libres.

Política de escritura: determina cuando ocurre la operación de escritura en memoria cuando se altera el contenido de un bloque en la caché.

1.3 ESPACIO DE DIRECCIONES LÓGICO/FÍSICO

Espacio de direcciones lógico: conjunto de direcciones lógicas o virtuales generadas por un programa

Espacio de direcciones físico: conjunto de direcciones físicas correspondientes a las direcciones lógicas en un instante dado

1.4 MAPA DE MEMORIA DE UN PROCESO

Esta estructura está manejada principalmente por el hardware, ya que si lo hiciese el SO no sería realmente eficiente. Se define la imagen de un proceso como el par del mapa de memoria y el PCB.

Esencialmente, el mapa de memoria de un proceso muestra cómo se divide y se asigna la memoria virtual del proceso en la memoria física del sistema. Consta de varias secciones, cada una con su tamaño y dirección de inicio específicos.

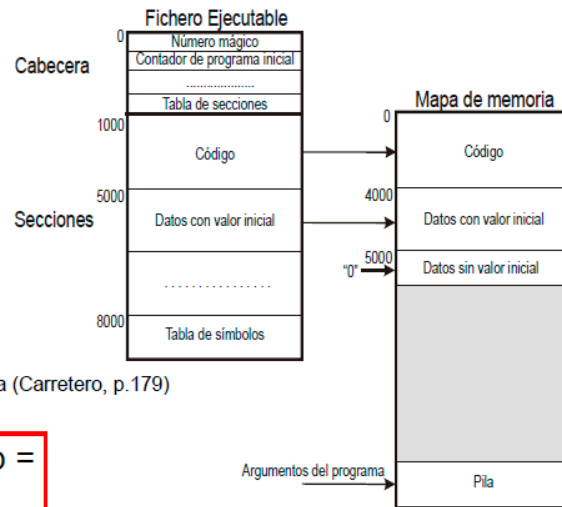


Figura (Carretero, p.179)

Imagen de proceso =
mapa + PCB

Tema 3: Gestión de Memoria

6

1.5 OBJETIVOS GENERALES DE LA GESTIÓN DE MEMORIA

Organización: ¿Cómo se divide la memoria de uno o varios procesos?

- **Organización contigua:** un solo bloque de posiciones seguidas (antiguo). Las particiones pueden ser fijas o variables. Mejoran y favorecen la velocidad (menos saltos tan grandes, y no hay que cambiar tanto de caché)
- **Organización no contigua:** permite dividir el programa en bloques colocados en zonas distintas (Paginación, Segmentación, Segmentación paginada). Aseguran que usemos lo máximo posible la memoria. Pros: todas las referencias a memoria son espacio de direcciones lógico, pueden retirarse de memoria y traerlo en otra área. Un programa se divide en trozos que no deben ser contiguos.

Gestión: dado un esquema de organización, que estrategias seguir para conseguir rendimiento óptimo.

- Estrategias de asignación de memoria: contigua, no contigua.
- Estrategias de sustitución de programas (o partes) en memoria principal.
- Estrategias de búsqueda de programas en memoria auxiliar.

Protección: el SO de los procesos de usuario, los procesos de usuario entre ellos

1.6 INTERCAMBIO (SWAPPING)

Idea: intercambiar procesos entre memoria (principal) y un almacenamiento auxiliar (secundario). El estado del proceso pasa a ser "Suspendido bloqueado", y pasando su espacio en MP a disco. El almacenamiento auxiliar debe ser un disco rápido con espacio para albergar las imágenes de memoria de los procesos de usuario.

1. El SO mantiene una lista de segmentos de procesos en memoria física y una lista de segmentos de procesos en disco.
2. Si se inicia un proceso y no hay suficiente memoria física disponible para alojar a todos sus segmentos, el sistema elige algunos de ellos y los escribe en el disco duro. Estos segmentos se conocen como "segmentos intercambiados".
3. El SO actualiza lista de segmento en memoria física y la lista de segmentos en disco para reflejar los cambios.
4. Si hay que ejecutar el proceso de nuevo, el SO cambia segmentos intercambiados del disco duro a la memoria física y actualiza de nuevo las listas de segmentos.
5. Proceso continúa la ejecución donde la dejó.

El factor principal en el tiempo de intercambio es el tiempo de transferencia de memoria principal (MP) a memoria auxiliar (MA). Sus responsabilidades son:

- Seleccionar procesos para retirarlos de MP y para incorporarlos a MP.
- Gestionar y asignar el espacio de intercambio.

2. MEMORIA VIRTUAL: ORGANIZACIÓN

Memoria Virtual: mecanismo usado para que un programa en ejecución pueda usar más memoria físicamente instalada en el sistema. La memoria virtual divide el espacio de direcciones lógicas del programa en páginas o bloques más pequeños.

Cuando un programa necesita acceder a una página que no está en memoria física, se produce una operación llamada "**fallo de página**" y la página correspondiente se trae desde el almacenamiento secundario a la memoria física.

La memoria virtual también evita que el proceso se dé cuenta de dichas particiones y evita tiempos prolongados, resuelve el problema del crecimiento dinámico de los procesos y permite aumentar el grado de multiprogramación.

Usa una combinación de memoria física RAM y almacenamiento secundario (disco duro u otros almacenamientos) para almacenar las páginas de memoria:

- **Memoria Principal** → partes del proceso necesarias en un momento dado.
- **Memoria Secundaria** → espacio de direcciones completo del proceso.

Es necesario saber:

- Disponer de la información de qué partes están en MP y que partes en MS.
- Tabla de Ubicación en Disco (TUD).
- Política de movimiento entre MP y MS.

2.1 UNIDAD DE GESTIÓN DE MEMORIA (MMU)

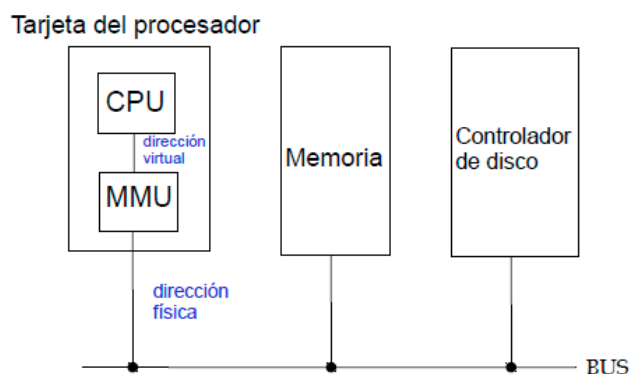
MMU: dispositivo hardware que traduce direcciones virtuales a direcciones físicas, este dispositivo es gestionado por el SO → aunque ningún SO puede gestionar la memoria como quiere ya que siempre están limitados por el hardware de gestión de memoria.

Si queremos volver a acceder a una dirección que ya hemos traducido no tenemos que traducirla de nuevo, pero si hacemos un cambio de contexto, como cambiamos de proceso, las direcciones anteriores ya no son válidas y hay que vaciar la caché → los cambios de contexto son muy costosos.

El programa usuario sólo trata con direcciones lógicas, nunca físicas. En el esquema más simple, el valor del registro base se añade a cada dirección generada por el proceso de usuario al mismo tiempo que es enviado a memoria.

- Detecta si la dirección aludida se encuentra o no en MP.
- Genera una excepción si no se encuentra en MP.

Tiene Translation Look-Aside Buffer (registros TLB) que guardan las correspondencias establecidas anteriormente, si acierta el TLB se realiza la traducción, y si falla la TLB usa el mapa de memoria para realizar la traducción.



2.2 PAGINACIÓN

El espacio de direcciones físicas de un proceso puede ser no contiguo. La memoria física se divide en bloques de tamaño fijo, denominados **marcos de página** (El tamaño es potencia de dos, de 0.5 a 8 Kb). El espacio lógico de un proceso se divide en bloques del mismo tamaño, denominados **páginas**. Los marcos de páginas contendrán páginas de los procesos.

Cuanto más pequeños son los marcos de página -> hay menos huecos de memoria sin usar -> pero hay más tiempo de acceso. **Cuanto más grandes son** -> hay más memoria contigua y mejor velocidad, pero más huecos en memoria.

Problemas: puede haber pérdidas de memoria, ya que hay páginas que no llegan a ocupar el tamaño total, y perdemos memoria que no hayamos asignado.

Las **direcciones lógicas**, que son las que genera la CPU, se dividen en **número de página (p)** y **desplazamiento** dentro de la página (d).

Las **direcciones físicas** se dividen en **número de marco (m)**, dirección base del marco donde está almacenada la página, y **desplazamiento (d)**.

Cuando la CPU genere una dirección lógica será necesario traducirla a la dirección física correspondiente, la **tabla de páginas** mantiene información necesaria para realizar dicha traducción. Existe una tabla de páginas por proceso.

- **Tabla de ubicación en disco** (una por proceso) ubicación de cada página en el almacenamiento secundario.
- **Tabla de marcos de página**, usada por el S.O. y contiene información sobre cada marco de página.

Una entrada por cada página del proceso:

- Número de marco (dirección base del marco) en el que está almacenada la página si está en MP
- Bit de presencia o bit válido
- Bit de modificación
- Modo de acceso autorizado a la página (bits de protección)

Nº de página ↓	nº marco	presencia	modificación	protección
	46	1	0	01

En el **esquema de traducción**, cada acceso a memoria implica 2 accesos de memoria, primero a la TP y luego a la memoria. Con el buffer evitamos 1 de los accesos a la TP. Si no está la página, el hardware llama al S.O. y este a medio de una rutina, se trae la página que corresponda a un marco que tenga libre. El proceso no se entera porque está bloqueado.

2.2.1 FALTA DE PÁGINA

1. Bloquear proceso
2. Encontrar la ubicación en disco de la página solicitada (tabla de ubicación en disco)
3. Encontrar un marco libre. Si no hubiera, se puede optar por desplazar una página de MP
4. Cargar la página desde disco al marco de MP
5. Actualizar tablas (bit presencia=1, no marco, ...)
6. Desbloquear proceso
7. Reiniciar la instrucción que origina la falta de página

2.2.2 IMPLEMENTACIÓN DE LA TABLA DE PÁGINAS

Se mantiene en memoria principal. El RBTP apunta a la tabla de páginas (suele almacenarse en el PCB del proceso). Un problema adicional viene determinado por el tamaño de la TP -> si un proceso tiene muchas páginas, la TP ocupa mucho espacio.

Independientemente de lo que ocupe un proceso, todos tienen páginas del mismo tamaño, por lo que el S.O. ocuparía mucha MP y podría ejecutar pocas cosas realmente. Esto se soluciona dividiendo la TP en varias páginas y teniendo índices en ellas. Además, según el proceso hacemos que inicialice la TP a las direcciones que va a usar y no a todas -> **Paginación multinivel**.

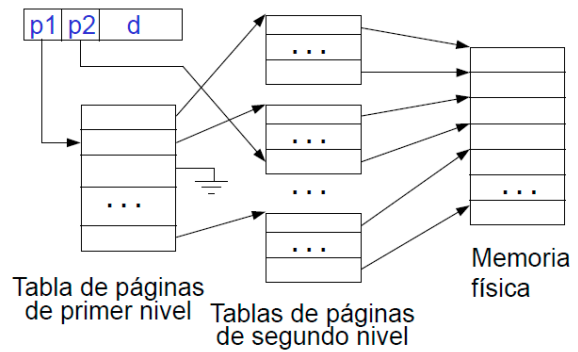
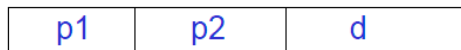
2.2.3 PAGINACIÓN MULTINIVEL

Se basa en paginar las tablas de las páginas. La partición de la tabla permite al SO dejar particiones no usadas sin cargar, hasta que el proceso las necesite. Las porciones que no se usan no necesitan tener tabla de página.

Paginación a 2 niveles:

Dividimos la tabla de páginas en partes del tamaño de una página, por lo que la dirección lógica se divide en:

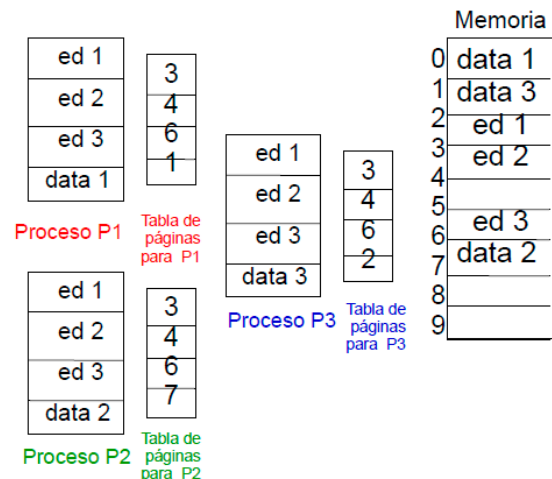
- Número de página (n bits): N° de página p1 (=k)
 - Desplazamiento de página p2(=n-k)
 - Desplazamiento de página d (m bits)
- Una dirección lógica es de la forma:



Páginas compartidas:

Una página en memoria con código puede ser compartida por varios procesos. Las TP de procesos que comparten página tienen la misma dirección base de marco. Las copias se hacen de código que sea de sólo lectura.

Algunos ejemplos son editores, compiladores, o sistemas de ventanas -> se intenta no modificar estas zonas porque habría que informar a todos los procesos = proceso costoso.



2.3 SEGMENTACIÓN

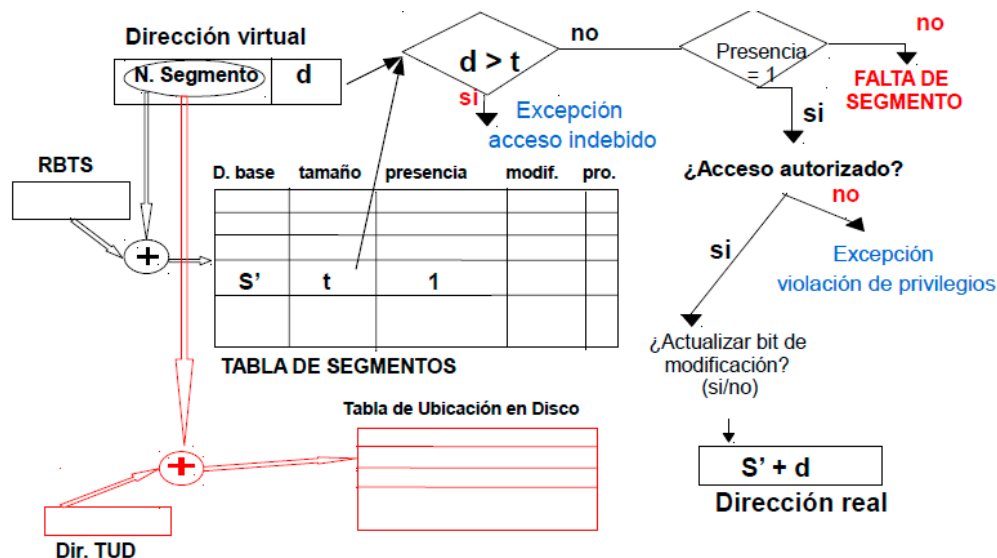
Esquema de organización de memoria. Soporta mejor la visión de memoria del usuario. **Un programa es una colección de unidades lógicas -> segmentos.**

Tabla de segmentos (TS):

Dirección virtual: tupla <n° de segmento, desplazamiento>

La TS aplica direcciones bidimensionales definidas x el usuario en direcciones físicas de una dimensión. Cada entrada contiene: **base** (dir física donde reside el inicio del segmento en memoria) y **tamaño** (longitud del segmento).

Se implementa de esta forma: la TS se mantiene en MP. El RBTS apunta a la TS (suele almacenarse en el PCB del proceso), el STLIR indica el n° de segmentos del proceso (s); este s es legal si $s < STLIR$ (suele almacenarse en el PCB del proceso).

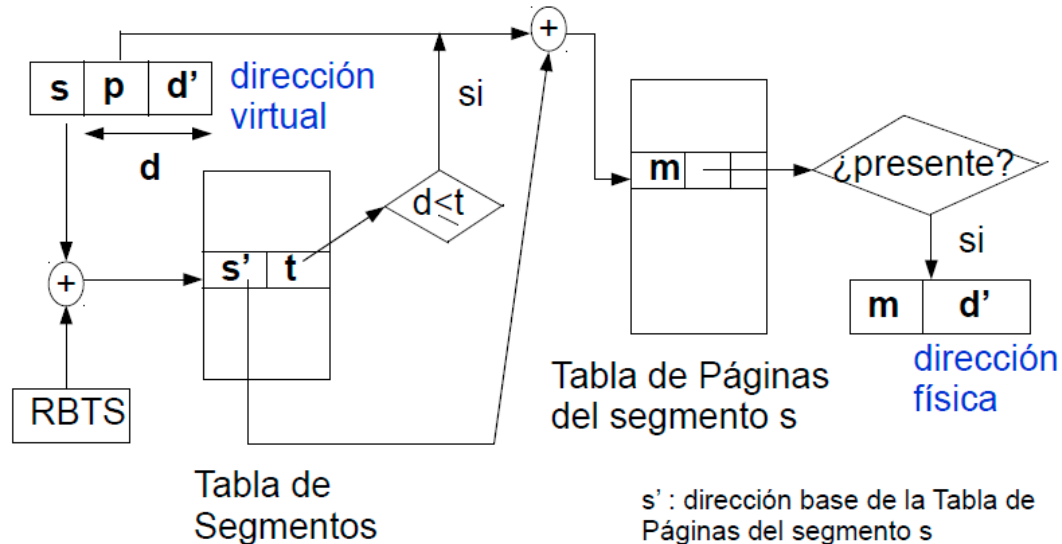


2.3.1 SEGMENTACIÓN PAGINADA

La variabilidad del tamaño de los segmentos y el requisito de memoria contigua en un segmento complica la gestión de MP y MS, la paginación simplifica la gestión, pero complica la compartición y protección.

Se pueden combinar los 2 enfoques logrando la mayoría de las ventajas de la segmentación y eliminando una gestión de memoria compleja.

Esto da lugar a la **Segmentación Paginada**: cada proceso se divide en segmentos de distintos tamaños, pero luego los segmentos están distribuidos en páginas del mismo tamaño.



3. MEMORIA VIRTUAL: GESTIÓN

Tratamos gestión relativa a la paginación.

Criterios de clasificación respecto a:

- **Políticas de asignación**: que marcos de página se asignan a un proceso. Fija o Variable.
- **Políticas de búsqueda** (recuperación): cuando se pone una página en memoria.
 - Paginación por demanda.
 - Paginación anticipada (i= prepaginación)
- **Políticas de sustitución** (reemplazo): cuando es necesario liberar un marco para dejar espacio a nuevo marco.
 - Sustitución local.
 - Sustitución global.

Independientemente de la **política de sustitución** utilizada, existen ciertos criterios que siempre debe cumplirse:

- **Páginas limpias frente a sucias**: se prioriza siempre quitar las limpias para mejorar la eficiencia, ya que al no haber hecho cambios no hace falta salvarla en disco, cosa que sí habría que hacer con páginas modificadas. Se minimiza el coste de transferencia.
- **Páginas compartidas**: se intentan no quitar ya que otro proceso las puede usar más tarde. Se reduce el nº de faltas de páginas.
- **Páginas especiales**: algunos marcos pueden estar bloqueados (no se pueden sustituir)

Influencia del tamaño de página:

- **Más pequeñas**: se desperdicia menos espacio. Crece tamaño de las TP y el nº de transferencias de MP a Disco, se reduce la fragmentación interna. Problema: al tener muchas páginas para un proceso podemos sobrecargar el procesamiento.
- **Más grandes**: grandes cantidades de información sin uso ocupando MP, crece la fragmentación interna (podré ejecutar menos procesos)

Hoy en día se pone de tamaño de página lo que empíricamente se ha visto que funciona, depende del SO.

3.1 ALGORITMOS DE SUSTITUCIÓN

Combinaciones posibles de tipo de asignación en MP y tipo de sustitución:

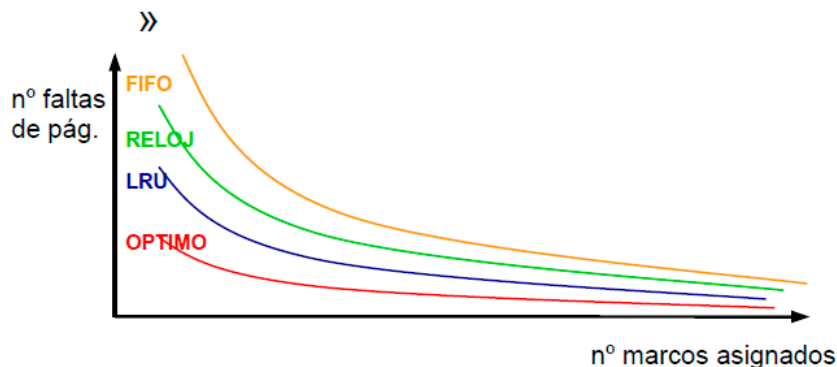
- Asignación fija, sustitución local.
- Asignación variable, sustitución local.
- Asignación variable, sustitución global

Algoritmo:

- Optimo: sustituye página que no se va a referenciar en un futuro o lo haga + tarde.
- FIFO: sustituye página + antigua.
- LRU: sustituye página a la que hace + tiempo se referenció. (Se guarda cada vez que accedamos a una página)
- Algoritmo del Reloj: es la aproximación + eficiente del LRU. Cada página tiene un bit de referencia R (un flag, puesto a 1 por el hardware).
 - Los marcos de página se representan por una lista circular y un puntero a la página visitada hace más tiempo.
 - Selección de una página:
 - 1. Consultar marco actual.
 - 2. ¿Es R=0? :
 - No: R=0; ir al siguiente marco y pasar a paso 1.
 - Sí: seleccionar para sustituir e incrementar pos.

Cuando llegamos al final y no hemos encontrado ninguno, empezamos de nuevo y encontraremos uno sí o sí.

Conclusión: influye más la cantidad de MP disponible que el algoritmo usado. LRU es el que se usa con más frecuencia, y la memoria es el recurso que más limita la velocidad del procesador.



3.2 COMPORTAMIENTO DE LOS PROGRAMAS

Viene definido por la secuencia de referencias a página que realiza el proceso. Es importante para maximizar el rendimiento del sistema de memoria virtual (TLB, alg. Sustitución...)

PROPIEDAD DE LA LOCALIDAD

Existen distintos tipos:

- **Temporal:** posición de memoria referenciada recientemente tiene alta probabilidad de ser referenciada en futuro próximo (ciclos, rutinas)
- **Espacial:** si ha sido referenciada una posición de memoria, las adyacentes tienen alta probabilidad de serlo también (array, secuencias)

CONJUNTO DE TRABAJO

Working Set de un proceso es el conjunto de páginas que son frecuentemente referenciadas en un determinado intervalo de tiempo. Son transitorios, no se puede predecir su tamaño y difieren mucho unos de otros.

$$WS(t,z) = \text{páginas referenciadas en el intervalo de tiempo } t - z \text{ y } t$$

Mantenerlo gasta recursos, pero a la larga nos ahorra tener que cargar/escribir páginas en un futuro. Consiste en mantener siempre en memoria de un proceso aquellas páginas que está utilizando actualmente (marcos referenciados en los últimos x instantes).

El nº de faltas de página no crece mucho si el conjunto de páginas necesarias puede residir en memoria principal, pero si eliminamos de la MP páginas de ese conjunto, la activación de paginación aumenta mucho.

Teoría del Conjunto de Trabajo: un proceso sólo se puede ejecutar si su conjunto de trabajo está en memoria principal y una página no se puede retirar de memoria principal si está dentro del conjunto de trabajo del proceso que está ejecutándose.

3.3 HIPERPAGINACIÓN

Los fallos de página son muy costosos ya que dependemos de la MS (Memoria Secundaria), muy lenta en comparación. Un proceso está trashing si está más en E/S por faltas de página que ejecutándose.

Si un proceso no tiene "suficientes" páginas, la tasa de faltas es alta, lo que implica un bajo uso de la CPU, que el SO incrementa el grado de multiprogramación y que ocurran + faltas de páginas.

Hiperpaginación: el SO está ocupado en resolver faltas de página. Cuando esto sucede la CPU está muy baja y la E/S casi al 99,99%. Si no hay suficiente MP el SO colapsa.

Para evitar que esto suceda podemos suspender algunos procesos (menos procesos -> menos fallos de páginas) y así reducimos la multiprogramación. También podemos evitarlo lanzando menos procesos o aumentando la memoria principal.

Formas de evitar la hiperpaginación:

- Asegurar que cada proceso existente tenga asignado un espacio en relación a su comportamiento -> Algoritmos de asignación variables
- Actuar directamente sobre el grado de multiprogramación -> Algoritmos de regulación de carga.

ALGORITMO BASADO EN MODELO WS

En cada referencia, determina el conjunto de trabajo: páginas referenciadas en el intervalo $(t - x, t]$ y sólo esas páginas son mantenidas en MP

- En la figura se representan las páginas del proceso que están en MP
- Proceso de 5 páginas
- $x = 4$
- En $t=0$ $WS = \{A, D, E\}$, A se referenció en $t=0$, D en $t=-1$ y E en $t=-2$

C, C, D, B, C, E, C, E, A, D									
A	A	A	A	-	-	-	-	A	A
-	-	-	-	B	B	B	B	-	-
-	C	C	C	C	C	C	C	C	C
D	D	D	D	D	D	-	-	-	D
E	E	-	-	-	E	E	E	E	E
* * * * *									

Algoritmo FFP (Frecuencia de Falta de Página):

Para ajustar conjunto de páginas de un proceso, se usan los intervalos de tiempo entre 2 faltas de páginas consecutivas. Si el intervalo de tiempo es grande (mayor que Y) todas las páginas no referenciadas en ese intervalo son retiradas de MP (como si cambiase el WS). En otro caso, la nueva página se incluye en el conjunto de páginas residentes.

t_c = instante de t^o de la actual falta de página
 t_{c-1} = instante de t^o de la anterior falta de página
 Z = conjunto de páginas referenciadas en un intervalo de t^o
 R = conjunto de páginas residentes en MP

$$R(t_c, Y) = \begin{cases} Z(t_{c-1}, t_c) & \text{si } t_c - t_{c-1} > Y \\ R(t_{c-1}, Y) + Z(t_c) & \text{en otro caso} \end{cases}$$

4. GESTIÓN DE MEMORIA EN LINUX

4.1 GESTIÓN DE MEMORIA A BAJO NIVEL

El kernel hace gestión del uso de la memoria física y tanto él como el hardware trabajan con páginas (el tamaño depende de la arquitectura). Las páginas físicas son representadas con el kernel mediante un **struct page**.

Una página puede ser utilizada por:

- La cache de páginas. El campo mapping apunta al objeto representado por `address_space`.
- Datos privados.
- Una proyección de la tabla de páginas de un proceso.
- El espacio de direcciones de un proceso.
- Los datos del kernel alojados dinámicamente.
- El código del kernel.

Zonas de memoria. El tipo `gfp_t` permite especificar el tipo de memoria que se solicita mediante tres categorías de flags:

- Modificadores de acción (`GFP_WAIT`, `GFP_IO`)
- Modificadores de zona. (`GFP_DMA`)
- Tipos (especificación más abstracta).

4.2 CACHÉ DE BLOQUES

La asignación y liberación de estructuras de datos es una de las operaciones más comunes en un kernel de SO. Para agilizar esta solicitud/liberación de memoria. Linux usa el **nivel de bloques** (slab layer).

- El nivel de bloques actúa como un nivel de cache de estructuras genérico.
- Existe una cache para cada tipo de estructura distinta: pej. `task_struct` cache o `inode` cache.
- Cada cache contiene múltiples bloques constituidos por una o más páginas físicas contiguas.
- Cada bloque aloja estructuras del tipo correspondiente a la cache.

Funcionamiento:

Cada bloque puede estar en uno de tres estados: lleno, parcial o vacío.

Cuando el kernel solicita una nueva estructura:

- La solicitud se satisface desde un bloque parcial si existe alguno.
- Si no, se satisface a partir de un bloque vacío.
- Si no existe un bloque vacío para ese tipo de estructura, se crea uno nuevo y la solicitud se satisface en este nuevo bloque.

4.3 ESPACIOS DE DIRECCIONES DE UN PROCESO

Es el espacio de direcciones de los procesos ejecutándose en modo usuario, Linux usa la memoria virtual (VM). A cada proceso se le asigna espacio de memoria plano de 32 o 64 bits único, aun así, se puede compartir el espacio de memoria (usar `CLONE_VM` en hebras).

El proceso sólo puede acceder a determinados intervalos de direcciones de memoria -> áreas de memoria. Las áreas de memoria contienen:

- Un mapa de memoria de la sección de código (**text section**).
- Un mapa de memoria de la sección de variables globales inicializadas (**data section**).
- Un mapa de memoria con una proyección de la página cero para variables globales no inicializadas (**bss section**)
- Un mapa de memoria con una proyección de la página cero para la pila de espacio de usuario

DESCRIPTOR DE MEMORIA

Se puede asignar haciendo una copia del descriptor mediante la orden `fork()`, o compartiendo el descriptor mediante el flag `CLONE_VM` de la llamada `clone()`. Y al **liberarlo** el núcleo decrementa el contador **`mm_users`**, incluido en **`mm_struct`**. Si este llega a 0 se decrementa el contador de uso `mm_count`. Si este contador llega a 0 se libera la `mm_struct` en la caché.

TABLAS DE PÁGINAS

Las direcciones virtuales deben convertirse a direcciones físicas mediante tablas de páginas. En linux tenemos 3 niveles de tablas de páginas.

- La tabla de páginas de más alto nivel es el directorio global de páginas (del inglés page global directory, PGD), que consta de un array de tipo `"pgd_t"`.
- Las entradas del PGD apuntan a entradas de la tabla de páginas de segundo nivel (page middle directory, PMD), que es un array de tipo `"pmd_t"`.
- Las entradas del PMD apuntan a entradas en la PTE. El último nivel es la tabla de páginas y contiene entradas de tabla de páginas del tipo `"pte_t"` que apuntan a páginas físicas: `struct_page`.

4.4 CACHÉ DE PÁGINAS

Está constituida por páginas físicas de RAM cuyos contenidos se corresponden con bloques físicos de disco, su tamaño es dinámico. La caché se realiza sobre el almacén de respaldo, que se encarga de la lectura/escritura de datos de/ a disco.

El proceso por el cual se eliminan datos de la caché junto a la estrategia para decidir qué datos eliminar se denomina **desalojo de la caché de páginas**. Linux selecciona páginas limpias y las reemplaza con otro contenido, si no hay suficientes pues fuerza un proceso para hacer disponibles + páginas limpias, y por último decide que páginas limpias selecciona para eliminar (**Selección de víctima**)

SELECCIÓN DE VÍCTIMA

Algoritmo LRU (Least Recently Used). Selecciona las páginas con el tiempo + antiguo. El problema es el acceso a archivos una sola vez. Para solucionarlo Linux usa 2 listas: active list e inactive list.

- Las páginas de active list no se pueden escoger como víctima, se añaden páginas accedidas sólo si residen en la inactive list.
- Las páginas de la inactive list pueden seleccionarse como víctimas.

OPERACIONES

Una página puede contener varios bloques de disco posiblemente no contiguos (depende del metodo de asignación de bloques a archivos).

- La cache de páginas de Linux usa una estructura para gestionar entradas de la cache y operaciones de E/S de páginas: `address_space`.
- Lectura/escritura de páginas de/en cache.
- Hebras de escritura retardada.

Gestión de Archivos

1. INTERFAZ DE LOS SISTEMAS DE ARCHIVOS

Un **archivo** es una colección de información relacionada y almacenada en un dispositivo de almacenamiento secundario. Poseen espacio de direcciones lógicas contiguas. Su estructura interna (lógica) puede estar distribuida así:

- **Secuencia de bytes:** el tipo de archivo que sea determina su estructura, si es texto -> caracteres, si es código fuente -> secuencia de subrutinas y funciones.
- **Secuencia de registros de longitud fija.**
- **Secuencia de registros de longitud variable.**

Existen varios tipos de archivos: regulares, directorios, de dispositivo...; así como diferentes formas de acceso: secuencial, aleatorio, otros...

Los archivos poseen **atributos** (metadatos) que los caracterizan:

- **Nombre:** única información en formato legible.
- **Tipo:** información necesaria para que sistema soporte diferentes archivos.
- **Tamaño:** tamaño actual del archivo y posiblemente el máximo.
- **Protección:** controla quién puede leer, escribir y ejecutar.
- **Tiempo, fecha e identificación del usuario:** necesario para protección, seguridad y monitorización del uso

Operaciones sobre los archivos: necesarias para definir de forma correcta los archivos

- **Gestión:** crear, borrar, renombrar, copiar, establecer y obtener atributos.
 - **Crear:** crear nuevo archivo y posicionarlo dentro de la estructura de ficheros.
 - **Borrar:** eliminarlo de la estructura y destruirlo.
- **Procesamiento:** abrir y cerrar, leer, escribir (modificar, insertar, borrar, información).
 - **Abrir:** abierto por un proceso, permite realizar funciones sobre él.
 - **Cerrar:** proceso cierra fichero y no puede volver a realizar funciones sobre él.

La mayoría de estas operaciones implican realizar una búsqueda en el directorio para hallar la entrada asociada con el archivo especificado. El SO mantiene la tabla de archivos abiertos que contiene información sobre los archivos abiertos.

1.1 ESTRUCTURA DE DIRECTORIOS

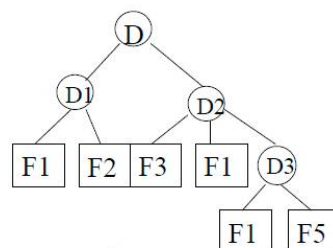
La información sobre los archivos se almacena en una estructura de directorios, la cual reside en almacenamiento secundario. Una entrada de directorio está formada por el nombre del archivo y su identificador unívoco. Es una colección de nodos que contiene información acerca de todos los archivos.

La organización lógica de los directorios debe proporcionar **eficiencia** (rapidez al localizar un archivo), **denominación** adecuada a sus usuarios, ya que el mismo archivo puede tener varios nombres y 2 usuarios pueden tener el mismo nombre para distintos archivos, y **agrupación** (poder agrupar los archivos de forma lógica según sus propiedades

EN ÁRBOL:

Árboles de altura arbitraria, permite a los usuarios crear sus subdirectorios. Cada árbol tiene un directorio raíz y todos los archivos del sistema tienen un nombre de ruta distintivo. Esta estructura permite que los usuarios accedan a los archivos de otros usuarios, además de a los suyos propios -> pero prohíbe la compartición de archivos o directorios. Sus características principales son:

- Necesidad de búsquedas eficientes.
- Posibilidad de agrupación.
- Directorio actual (de trabajo)
- Nombres de camino absolutos y relativos.



Un bit de entrada define si la entrada es un archivo (0) o un subdirectorio (1), cada proceso tiene un directorio actual que debe contener la mayor parte de archivos que interesen al proceso. Los nombres de ruta pueden ser:

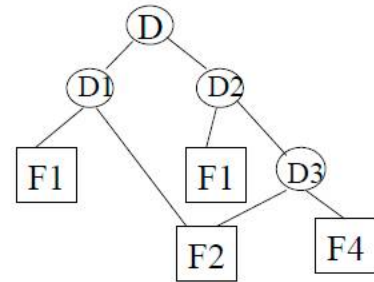
- **Absolutos:** comienzan en raíz y siguen ruta descendente hasta el archivo especificado, indicando los nombres de los directorios de ruta.
- **Relativos:** definen ruta a partir del directorio actual.

EN GRAFO

Sus rasgos principales son la compartición de subdirectorios y archivos, y que son más flexibles y complejos.

El archivo o subdirectorio puede estar en 2 directorios diferentes, lo que implica una compartición (sólo existe un archivo real, x lo que cualquier cambio realizado x un usuario será visible para el otro usuario). La compartición se puede dar de diversas formas, siendo una de ellas los enlaces.

Los enlaces son punteros a archivos o directorios. Si añadimos enlaces a directorio con estructura de árbol, esa estructura se destruye y resulta en una estructura en grafo.



- **Simbólicos:** se crea nueva entrada en el directorio indicando que es de tipo enlace y se almacena el camino de acceso absoluto o relativo del archivo al que se va a enlazar. Se puede usar en entornos distribuidos y posee gran nº de accesos a disco.
- **Absolutos:** se crea nueva entrada en directorio y se copia la dirección de la ED con la información del archivo. Hay problema al borrar los enlaces, la solución es un contador de enlaces.

La principal dificultad de la estructura es evitar que aparezcan ciclos a medida que se añaden nuevos enlaces a la estructura.

1.2 PROTECCIÓN

Proporcionar un acceso controlado a los archivos, establecer lo que puede hacerse y por quién. La **fiabilidad** se proporciona mediante copias duplicadas de los archivos.

Los mecanismos de protección dan acceso controlado, limitan los tipos de acceso a archivos que se pueden realizar: leer, escribir, ejecutar, añadir, borrar, listar.

La principal solución para garantizar la protección es hacer el acceso dependiente del identificador del usuario. Para implementarlo tenemos que asociar con cada archivo y directorio una lista de control de acceso que especifique los nombres de usuarios y los permisos que tienen, pero el problema de estas listas es la longitud.

Podemos solucionarlo clasificando a los usuarios en 3 grupos:

- **Propietario:** usuario que crea el archivo.
- **Grupo:** conjunto de usuarios que comparten un archivo y necesitan acceso al mismo.
- **Público:** resto de usuarios del sistema.

Otra manera es asociar una contraseña con cada archivo y si bien esta solución es efectiva, tiene una serie de desventajas:

- Recordar todas las contraseñas.
- Si solo hay un password -> acceso total o ninguno.

1.3 SEMÁNTICAS DE CONSISTENCIA

Especifican cuándo las modificaciones de datos por un usuario se observan por otros usuarios

1. **Semántica de Unix.** La escritura en un archivo es directamente observable. Existe un modo para que los usuarios compartan el puntero actual de posicionamiento en un archivo
2. **Semánticas de sesión** (Sistema de archivos de Andrew). La escritura en un archivo no es directamente observable. Cuando un archivo se cierra, sus cambios sólo se observan en sesiones posteriores
3. **Archivos inmutables.** Cuando un archivo se declara como compartido, no se puede modificar. Su nombre no puede reutilizarse y su contenido no se puede modificar al ser fijo. Es un mecanismo de sólo lectura.

1.4 FUNCIONES BÁSICAS DEL SA

Entre las funciones básicas que debe tener un SA destaca: tener conocimiento de todos los archivos del sistema, controlar la compartición y forzar la protección de archivos, gestionar el espacio en disco y traducir las direcciones lógicas del archivo en direcciones físicas del disco.

Los discos son el principal tipo de almacenamiento secundario en los SA al ser convenientes para almacenar varios archivos.

2. DISEÑO SOFTWARE DEL SISTEMA DE ARCHIVOS

Un sistema de archivos posee dos **problemas de diseño** diferentes:

1. Definir cómo debe ver el usuario el sistema de archivos
 - definir un archivo y sus atributos
 - definir las operaciones permitidas sobre un archivo
 - definir la estructura de directorios
2. Definir los algoritmos y estructuras de datos que deben crearse para establecer la correspondencia entre el sistema de archivos lógico y los dispositivos físicos donde se almacenan

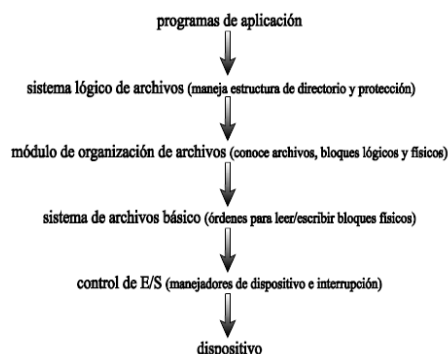
2.1 ESTRUCTURA DEL SA

Organización en niveles (capas): SA está compuesto por muchos niveles diferentes y cada uno de ellos usa las funciones de los niveles inferiores para crear nuevas funciones. El nivel más bajo está compuesto por **controladores de dispositivo** y rutinas de tratamiento de interrupción, encargados de pasar datos entre la MP y el disco.

El **sistema de archivos básico** envía comandos al controlador de dispositivo apropiado, para leer y escribir bloques físicos en el disco. El **módulo de organización de archivos** tiene información sobre los archivos y sus bloques lógicos y físicos. El **sistema lógico de archivos** gestiona información de metadatos, incluyen toda la estructura del sistema menos el contenido de los archivos. Le da al módulo de organización de archivos la información que necesita a partir de bloques de control de archivo.

Bloque de control de archivo: estructura con información de un archivo en uso.

Por eficiencia, el SO mantiene una tabla indexada x descriptor de archivo de archivos abiertos.



2.2 MÉTODOS DE ASIGNACIÓN DE ESPACIO

El principal problema es saber la manera de asignar el espacio a los archivos de forma que el uso del espacio del disco sea eficaz y que se pueda acceder a los archivos rápidamente. Hay 3 métodos para asignar espacio en disco: asignación continua, enlazada e indexada.

- **Contiguo:** cada archivo **ocupa un conjunto de bloques contiguos en disco**. Como **ventajas** presenta que es sencillo, ya que solo requiere el nº de bloque y la longitud, y además son buenos el acceso secuencial como el directo. También tiene **desventajas**: a priori no conocemos el espacio, puede darse un derroche de espacio (problema de la asignación dinámica -> fragmentación externa), y los archivos no pueden crecer si no les realizamos compactación (ineficiente).
- **Enlazado:** Resuelve problemas de la asignación continua, cada archivo es una lista enlazada de bloques de disco, estos bloques pueden estar dispersos en el disco. **Ventajas**: No hay fragmentación externa, no es necesario declarar el tamaño del archivo al momento de crearlo, archivo puede seguir creciendo si hay bloques libres, y basta almacenar el puntero al primer bloque del archivo. **Desventajas**: acceso directo no es efectivo (si el secuencial). Se requiere espacio para los punteros de enlace -> se soluciona con agrupaciones de bloques (clusters). Seguridad por la pérdida de punteros -> se soluciona con lista doblemente enlazada.

Una variante de este mecanismo es el uso de tabla de asignación de archivos (FAT). Una sección del disco al principio de cada volumen se reserva para almacenar la FAT, que tiene una entrada por cada bloque del disco y está indexada según el nº de bloque. Para localizar un bloque solo se necesita leer en la FAT -> se optimiza acceso directo. El problema puede venir con la pérdida de punteros, pero lo solucionamos haciendo una doble copia de la FAT.

Indexado: todos los bloques están juntos en el bloque índice. El directorio tiene su localización y cada archivo tiene asociado su propio bloque índice. Para leer el i-ésimo bloque buscamos el puntero en la i-ésima entrada del bloque índice.

Ventajas: buen acceso directo sin producir fragmentación externa. **Contras:** desperdicio del espacio. Además, está el problema del tamaño del bloque índice, ya que si es demasiado pequeño no puede almacenar suficientes punteros. Soluciones:

- Bloques índices enlazados.
- Bloques índices multinivel:
 - **Problema:** acceso a disco necesario para recuperar la dir del bloque para cada nivel de indexación.
 - **Solución:** mantener algunos bloques índices en memoria principal.
- Esquema combinado.

2.3 GESTIÓN DEL ESPACIO LIBRE

El sistema mantiene una lista de bloques que están libres: **lista de espacio libre**. Puede implementarse:

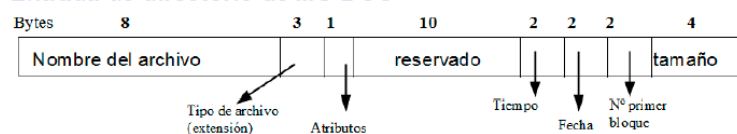
1. **Vector de bits o mapa**. Cada bloque se representa con un bit (0 Libre – 1 ocupado). Su principal ventaja es que es simple y la eficiencia que permite al localizar el primer bloque o n bloques libres. Si no se mantiene en MP es ineficiente.
2. **Lista enlazada**. Enlaza los bloques libres del disco, guarda puntero al primer bloque en un lugar concreto. No derrocha espacio, pero es relativamente ineficiente ya que no es común tener que recorrer la lista de espacio libre.
3. **Lista enlazada con agrupación**. Cada bloque almacena n-1 direcciones de bloques libres. Es rápido obtener muchas direcciones de bloques libres.
4. **Cuenta**. Cada entrada de la lista contiene una dirección de bloque libre y un contador del nº de bloques libres que le sigue

2.4 IMPLEMENTACIÓN DE DIRECTORIOS

Casos del contenido de una entrada de directorio:

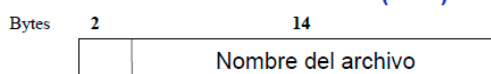
- a) Nombre de Archivo + Atributos + Dirección de los bloques de datos (DOS)

Entrada de directorio de MS-DOS



- b) Nombre de Archivo + Puntero a ES que contenga información relativa al archivo (UNIX).

Entrada de directorio de UNIX (s5fs)



Cuando se abre un archivo, el SO busca en su directorio la entrada correspondiente, extrae sus atributos y la localización de sus bloques de datos y los coloca en una tabla en MP, y cualquier referencia posterior usa la información de dicha tabla.

Implementación de archivos compartidos (o enlace):

1. Enlaces **simbólicos**
 - Se crea una nueva entrada en el directorio, se indica que es de tipo *enlace* y se almacena el camino de acceso absoluto o relativo del archivo al cual se va a enlazar
 - Se puede usar en entornos distribuidos
 - Gran número de accesos a disco
2. Enlaces **absolutos** (o *hard*)
 - Se crea una nueva entrada en el directorio y se copia la dirección de la estructura de datos con la información del archivo
 - Problema al borrar los enlaces: solución -> **Contador de enlaces**

2.5 DISTRIBUCIÓN DEL SA

Los sistemas de archivos se almacenan en discos que pueden dividirse en una o más particiones.

Se puede hacer formateo del disco:

- **Físico**: pone los sectores (cabecera y código de corrección de errores) por pista.
- **Lógico**: escribe la información que el SO necesita para conocer y mantener los contenidos del disco (un directorio inicial vacío, FAT, lista de espacio libre, ...)

Una de las estructuras usadas en disco y en memoria para implementar un SA es el bloque de control de arranque usado para inicializar el sistema localizado por Bootstrap

2.6 RECUPERACIÓN

Como los archivos y directorios se mantienen tanto en MP como en disco, el sistema debe asegurar que un fallo no genere pérdida o inconsistencia de datos, hay varias formas de hacerlo:

1. **Comprobador de consistencia.** Compara datos de la estructura de directorios con los bloques de datos en disco y trata cualquier inconsistencia. Busca y corrige incoherencias.
2. **Copias de seguridad:** usar programas del sistema para hacerlas de los datos de disco a otros dispositivos y de recuperación de los datos perdidos. Podemos solucionar una pérdida de datos restaurando los datos a partir de una copia de seguridad.

3. IMPLEMENTACIÓN DE LA GESTIÓN DE ARCHIVOS EN LINUX

Un **i-nodo** es la representación interna de un archivo. Un archivo tiene asociado un solo i-nodo pero éste puede tener distintos nombres (**enlaces**).

- Si un proceso crea archivo, se le asigna un i-nodo.
- Si un proceso referencia a archivo, se analizan los permisos y se lleva el i-nodo a MP hasta que se cierre.

3.1 SISTEMA DE ARCHIVOS

SO implementa al menos un SA estándar o nativo, en Linux se llaman **ext2**, **ext3** y **ext4**. Hay necesidad de dar soporte a otros SA distintos, para ello el kernel incluye una capa entre los procesos de usuario y la implementación del SA -> **Sistema de Archivos Virtual** (VFS- Virtual File System)

Linux abstrae el acceso a los archivos y a los SA mediante una interfaz virtual que lo hace posible

Flujo de operaciones/datos por las distintas partes del sistema en una llamada al sistema de escritura (write)

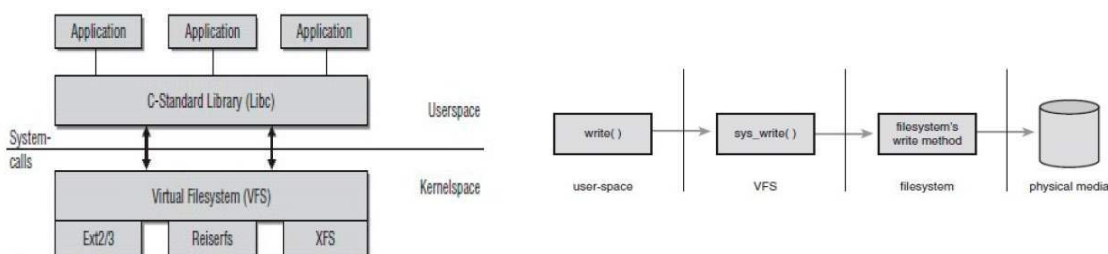


Figure 8-1: VFS layer for filesystem abstraction.

3.2 TIPOS DE SISTEMAS DE ARCHIVOS

Tres clases generales de SA:

1. **SA basados en disco** (Disk-based filesystems): forma clásica de almacenar archivos en medios no volátiles: Ext2/3, Reiserfs, FAT, e iso9660.
2. **SA Virtuales** (Virtual filesystems): generados por el kernel y constituyen una forma simple para permitir la comunicación entre los programas y los usuarios. Ejemplo es el SA proc. No requieren espacio de almacenamiento en ningún dispositivo hardware (la información está en MP).
3. **SA de Red** (Network filesystems): permiten acceder a los datos a través de la red.

3.2.1 SISTEMAS DE ARCHIVOS EXT2

Divide el disco duro en un conjunto de bloques de = tamaño donde se guardan los datos d los archivos y de administración. El elemento central de Ext2 es el "block group". Cada SA consta de un gran nº de grupos de bloques secuenciales.

Boot sector: zona del disco duro cuyo contenido se carga de forma automática por la BIOS y que se ejecuta cuando el sistema arranca.

Cuando se usa un SA, se monta, el superbloque (Sus datos) se almacenan en MP.

3.3 MODELO DE ARCHIVO COMÚN

Para un programa de usuario, un archivo se identifica por un descriptor de archivo (nº entero usado como índice en la tabla de descriptors que identifica el archivo en las operaciones relacionadas con él). El descriptor lo asigna el kernel cuando se abre el archivo y es válido sólo dentro de un proceso. Dos procesos diferentes pueden usar el mismo descriptor pero no apuntan al mismo archivo. Un inodo es la estructura asociada a cada archivo y directorio y contiene sus metadatos.

3.3.1 CONTENIDO DE UN I-NODO

Identificador del propietario del archivo: UID, GID

Tipo de archivo (regular, directorio, dispositivo, cauce, link). Si es 0 -> el i-nodo está libre

Permisos de acceso

Tiempos de acceso: última modificación, último acceso y última vez que se modificó el i-nodo

Contador de enlaces

Tabla de contenidos para las **direcciones de los datos** en disco del archivo

Tamaño

3.4 ESTRUCTURA VFS

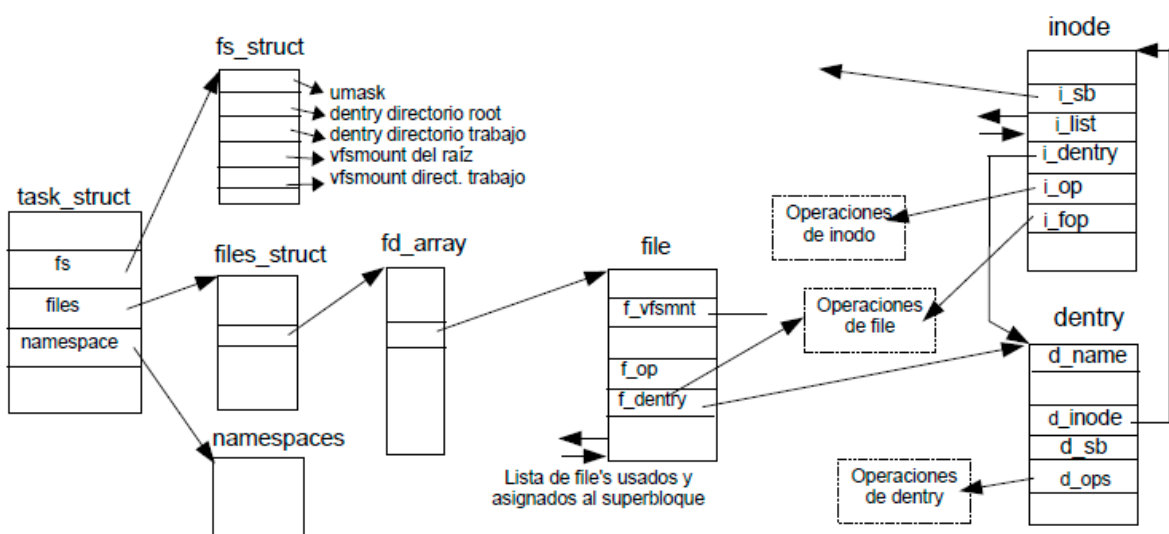
VFS es orientado a objetos, consta de dos componentes, archivos y SA, que necesita gestionar y abstraer. Se representa a un archivo y a un SA con una familia de estructuras de datos hechas en C. Existen 4 tipos de objetos primarios del VFS:

- **objeto superblock:** representa a un SA montado
- **objeto inodo:** representa a un archivo (cualquier tipo)
- **objeto dentry:** representa a una entrada de un directorio
- **objeto file:** representa a un archivo abierto y es una estructura por proceso. Las anteriores son de sistema.

Cada uno de estos objetos primarios tiene un vector de "**operations**". Estas funciones describen los métodos que el kernel invoca sobre los objetos primarios.

- **objeto super_operations:** métodos que el kernel puede invocar sobre un SA concreto, ej: *write_inodo()* y *sync_fs()*.
- **objeto inode_operations:** métodos que el kernel puede invocar sobre un archivo concreto, ej: *create()* y *link()*.
- **objeto dentry_operations:** métodos que el kernel puede invocar sobre una entrada de directorio, ej: *d_compare()* y *d_delete()*.
- **objeto file_operations:** métodos que un proceso puede invocar sobre un archivo abierto como *read()* y *write()*.

Cada SA registrado está representado por una estructura **file_system_type**. Este objeto describe el SA y sus capacidades. Cada punto de montaje está representado por la estructura **vfsmount** que contiene información acerca del punto de montaje, tal como sus localizaciones y flags de montaje. Finalmente, existen dos estructuras por proceso que describen el SA y los archivos asociados con un proceso: **fs_struct** y **file_struct**.



3.5 DESCRIPCIÓN DE UN GRUPO DE BLOQUES

Superbloque: estructura central para guardar meta-información del SA.

- struct ext2_super_block

Descriptores de grupo: contienen información que refleja el estado de los grupos de bloques individuales del SA.

- struct ext2_group_desc

Mapa de bits de bloques de datos y de inodos (Data bitmap, inode bitmap): contienen un bit por bloque de datos y por inodo respectivamente para indicar si están libres o no.

- struct ext2_inode

Tabla de inodos: contiene todos los inodos del grupo de bloques. Cada inodo mantiene los metadatos asociados con un archivo o directorio del SA.

- struct ext2_dir_entry_2

3.6 INFORMACIÓN SOBRE LOS BLOQUES DE UN ARCHIVO

Linux usa un método de asignación de bloques no contiguo y cada bloque de un SA se identifica por un número. En el inodo se almacenan 12 direcciones directas a BD, un primer nivel de indexación, un segundo nivel de indexación y un tercer nivel de indexación (si es necesario)

3.7 MONTAJE Y DESMONTAJE DE UN SA

La llamada al sistema **mount** conecta un sistema de archivos al sistema de archivos existente y la llamada **umount** lo desconecta.

```
mount (<camino_especial>, <camino_directorio>, <opciones>)
```

El núcleo tiene una tabla de montaje con una entrada por cada sistema de archivos montado:

- número de dispositivo que identifica el SA montado
- puntero a un buffer que contiene una copia del superbloque
- puntero al i-nodo raíz del SA montado
- puntero al i-nodo del directorio punto de montaje

Gestión de Entradas/Salidas

1. INTRODUCCIÓN

Las funciones principales de los dispositivos de entrada y salida son **controlar los dispositivos** y **ocultar las características y funcionamientos de éstos al resto del sistema**. Los objetivos generales del software de E/S son:

- Independencia de los dispositivos
- Tratamiento uniforme de archivos y dispositivos
- Tratamiento adecuado de los errores
- Gestionar dispositivos compartibles y dedicados
- Permitir el bloqueo de un proceso mientras se realiza una operación de E/S (por funcionamiento asíncrono dispositivos-CPU)

Los principios del hardware de E/S distinguen dos tipos de dispositivos:

- **Orientados a bloques (Discos, CDROM)**. Almacenan información en bloques de longitud fija. Cada bloque tiene su propia dirección.
- **Orientados a caracteres (Impresoras, ratón)**. Manejan flujos de caracteres sin ninguna estructura. No se pueden realizar búsquedas (**no direccionable**)

El reloj no entra en esta clasificación.

1.1 TÉCNICAS DE E/S

E/S programada: La CPU envía una orden de E/S a un módulo de E/S y supervisa la terminación de la E/S.

E/S guiada por interrupción: La CPU envía una orden de E/S a un módulo de E/S y sigue ejecutando instrucciones. El SO bloquea al proceso que solicita la orden. El módulo E/S interrumpe a la CPU al completar la orden y la RSI() desbloquea al proceso.

Acceso Directo a Memoria (DMA): controla el intercambio de datos entre MP y un módulo de E/S. El procesador envía una orden de transferencia de un bloque de datos al módulo de DMA y resulta interrumpido sólo cuando se haya transferido el bloque.

2. PRINCIPIOS DEL HARDWARE DE E/S

2.1 CONTROLADORES HARDWARE

Objetivo:

- Actuar de intermediario entre el procesador y el dispositivo de E/S.
- El SO se comunica exclusivamente con el controlador, no con el dispositivo que controla.

Estructura de un controlador genérico. Se puede dividir en tres capas funcionales:

- Interfaz con el bus del sistema.
- Controlador (genérico) de dispositivo.
- Interfaz con el dispositivo.

Funciones:

- Control y temporización: controlar la transferencia de datos a CPU.
- Comunicación con el procesador y con los dispositivos.
- Almacenamiento temporal de datos (buffers). La velocidad de transferencia entre los dispositivos y el módulo E/S es mucho menor que entre procesador/memoria, procesador/módulo o memoria/módulo.
- Detección de errores. (bit de paridad, códigos CRC en sectores de disco...)

2.1.1 CONTROLADOR GENÉRICO

Proporciona una abstracción uniforme de los dispositivos (puerto de E/S).

Cualquier operación se realiza leyendo o escribiendo sobre los registros.

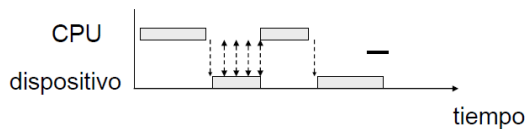
Traduce las órdenes genéricas en controles específicos del dispositivo.

2.2 ESQUEMAS DE COMUNICACIÓN

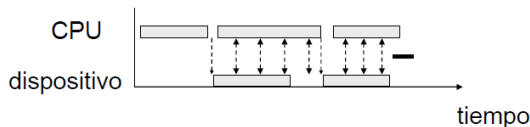
La CPU conoce el final de la operación a través de:

- **E/S programada:**

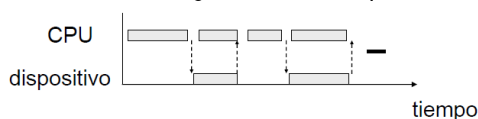
- La instrucción de E/S inicia la operación y espera a que finalice (comprueba el registro de estado -> espera ocupada).



- **Técnica del sondeo (polling):** la instrucción de E/S solo inicia la operación y se chequea periódicamente el estado del controlador.



- E/S controlada por Interrupciones. El controlador genera una interrupción cuando la operación de E/S ha finalizado



¿Quién se encarga de transferir los datos entre memoria principal y el buffer asociado al dispositivo?

La CPU

- En una operación de salida: escribe los datos desde su ubicación en MP al *buffer* del controlador
- En una operación de entrada, operación inversa.

El controlador DMA

- La CPU comunica al controlador la dirección de la fuente/destino de la transferencia y la cantidad y es éste quien se encarga del traspaso.

3. ARQUITECTURA SOFTWARE DEL SISTEMA DE E/S

3.1 PRINCIPIOS DE DISEÑO.

Eficiencia:

- Los dispositivos de E/S son lentos comparados con RAM y CPU. Soluciones:
- Multiprogramación.
- Intercambio a disco (otra E/S).
- Optimizar la E/S de disco.

Generalidad:

- Uniformidad de dispositivos desde el punto de vista de los procesos.
- Uniformidad de gestión de dispositivos y operaciones de E/S por el kernel.

Estrategia de diseño modular y jerárquico para E/S (Arquitectura de capas)

Detalles del dispositivo de E/S en las capas de nivel inferior.

Una capa da servicios a la capa superior ocultando detalles de implementación.

Cambios en una capa no suponen cambios en capas de nivel superior (encapsulación).

Tanto los procesos usuarios como el kernel usan dispositivos en términos de funciones generales: abrir/cerrar, lectura/escritura.

3.2 ARQUITECTURA EN CAPAS.

E/S lógica: gestiona tareas genéricas de E/S para los procesos usuario mediante un id de dispositivo y operaciones de alto nivel: abrir/cerrar, leer/escribir.

E/S de dispositivo: Las operaciones y datos (caracteres en los buffers, registros...) se convierten en las secuencias de instrucciones de E/S para el controlador software. Se pueden utilizar técnicas de uso de buffers para mejorar la utilización.

Planificación y control. Gestión y planificación de la cola de peticiones de E/S y control de las operaciones. En este nivel se manejan las interrupciones y se recoge e informa del estado de la E/S. Aquí se encuentran el controlador software y la RSI.

3.3 FUNCIONAMIENTO DEL SOFTWARE DE E/S.

3.3.1 SOFTWARE EN ESPACIO DE USUARIO

Biblioteca estándar de E/S: Proporciona funciones de E/S y en su implementación se usan wrappers de llamadas al sistema (printf...).

Wrappers de llamadas al sistema para E/S: Proporcionan servicios de E/S mediante la API del SO (write(...)).

Spooling: Un proceso de sistema acepta solicitudes de los procesos usuario y realiza su trabajo.

3.3.2 SOFTWARE INDEPENDIENTE DE DISPOSITIVO

Funciones:

- Suministrar una interfaz uniforme al kernel.
- Asignar nombres a los dispositivos.
- Proteger a los dispositivos de accesos no autorizados.
- Dar un tamaño de bloque independiente (dispositivos de bloques).
- Proporcionar buffering.
- Asignar/liberar dispositivos dedicados.
- Gestión de errores.

Composición:

- Rutinas E/S a nivel kernel (sys_write) accedidas por llamadas al sistema, wrappers o directamente con ensamblador y trampa (syscall).
- Las rutinas de E/S tienen que ser código reentrante.
- Estructuras de datos asociadas a las rutinas de E/S:
- Descriptor de dispositivo: información del dispositivo.
- Bloque de petición E/S (IORB): información de la petición E/S.

Funcionamiento de una rutina de E/S genérica:

- Identificar el dispositivo.
- Comprobar los parámetros de la petición.
- Empaquetar los parámetros en un IORB y enlazarlo en la cola del descriptor de dispositivo identificado.
- Indicar al driver que se ha añadido una petición a su cola.
- Esperar que el controlador software (driver) resuelva la petición.

3.3.3 CONTROLADOR SOFTWARE (DRIVER):

Contiene el código dependiente de dispositivo y conoce la estructura de registros del controlador hardware. Existe un driver por dispositivo (o tipo).

Funcionamiento de un driver genérico:

```
While (true) {  
    • wait(peticion_pendiente);  
    • Sacar IORB de la cola de peticiones;  
    • Iniciar E/S.  
    • Esperar finalización: wait(operacion_completa);  
    • Comprobar errores; Notificar fin E/S al proceso solicitante:  
    • signal(peticion_servida);  
    • EliminarIORB;  
}
```

3.4 BUFFERING

3.4.1 RESTRICCIONES DE LA E/S.

Efectos de la E/S en el SO:

- Al hacer una lectura con DMA:
 - El proceso se mantiene en “Bloqueado” hasta acabar la operación.
 - Los marcos de página que alojan las direcciones virtuales deben estar en RAM: Restringe las decisiones de intercambio.
 - Puede llevar a interbloqueo.
 - Solución:
 - Buffering E/S: hace las entradas antes de las peticiones, y las salidas poco después de la petición.
 - Idea: Cuando un proceso usuario hace E/S, el SO asigna un buffer para la operación en memoria kernel.
 - Las lecturas usan el buffer y al acabar se mueve la información a espacio usuario y se pide otro bloque.
 - Las escrituras usan el buffer y después se hacen sobre disco.

3.4.2 BUFFERING DE E/S.

Variantes del buffering:

- **Doble buffer.** Se asignan dos buffers en memoria kernel a la operación de E/S. Un proceso transfiere datos a/desde un buffer mientras el SO vacía/llena el otro buffer.
- **Buffer circular.** Si un proceso solicita muchas peticiones de E/S el doble buffer puede ser insuficiente → utilizar varios buffers.

3.5 PLANIFICACIÓN DE DISCO

3.5.1 ESTRUCTURA DE UN DISCO MAGNÉTICO.

Un disco es un plato circular construido de metal o plástico que está cubierto con un material magnético.

Los datos se graban en el disco y se recuperan del mismo mediante una bobina conductora llamada cabeza.

La cabeza permanece fija durante una E/S mientras que el plato rota debajo de ella.

La organización de los datos en el plato consiste en un conjunto concéntrico de anillos (pistas) y cada pista tiene la misma anchura que la cabeza.

Los datos se transfieren del disco en unidades de tamaño fijo denominadas sectores.

Tipos:

- **Disco con cabeza fija.** Hay una cabeza de lectura/escritura por pista. Todas las cabezas están montadas en un brazo rígido que se extiende.
- **Disco de cabeza móvil.** Hay sólo una cabeza de lectura/escritura por cada superficie. También en este caso la cabeza está montada en un brazo pero el brazo puede moverse radialmente para situar la cabeza sobre cualquier pista.

Los discos duros incluyen múltiples platos apilados verticalmente con una cabeza móvil por superficie. Todas las cabezas se posicionan conjuntamente sobre pistas que están a la misma distancia del centro del disco (cilindro).

Para el SO:

- El SO ve el disco como secuencia de bloques ($B_0, \dots, B_{n-1}$) de tamaño n (bloques lógicos).
- La información física de disco se referencia por una dirección compuesta: (dispositivo, superficie, pista, sector)
- Hay una correspondencia entre dirección de bloque lógico y dirección física.
- Un bloque lógico puede estar formado por 1 potencia de 2 sectores.

3.5.2 PLANIFICACIÓN DE DISCO.

Una petición tiene tres fases:

- **Tiempo de búsqueda:** poner la cabeza en el cilindro correcto.
- **Latencia rotacional:** Espera hasta que pase el sector referenciado.
- **Transferencia de datos.**

Cuando un proceso pide E/S, su IORB espera en la cola del dispositivo a ser atendido.

El SO puede mejorar el tiempo de servicio de disco planificando la cola de IORBs.

La planificación intenta minimizar la distancia de posicionamiento de la cabeza de E/S.

Algoritmos de planificación:

- **FCFS:** Primero en llegar, primero en ser servido.
- **SSTF:** Primero el que más cerca esté de la cabeza.
- **SCAN:** Va de principio a final y al revés (es decir, de 0-100 y luego 100-0).
- **CSCAN:** Va de principio a final y repite (es decir, 0-100, 0-100...).
- **LOOK:** Como SCAN, pero si hay peticiones en esa dirección.
- **CLOOK:** Como CSCAN, pero si hay peticiones en esa dirección.

Mecanismos de Seguridad

1. INTRODUCCIÓN

Protección: es estrictamente un problema interno

¿Cómo proporcionamos un acceso controlado a los programas y datos almacenados en un sistema?

Seguridad: considera el entorno externo dentro del cual opera el sistema

Distinción entre políticas y mecanismos

- **Políticas (lo que se hará):** qué datos serán protegidos
- **Mecanismos (cómo se hará):** Deben especificar que controles se van a realizar y qué medios van a hacer efectivos dichos controles
- Las políticas varían, los mecanismos deben ser lo bastante generales para abarcar un amplio abanico de políticas

Dos categorías de amenazas:

- **Intrusos (hacker, cracker, etc).** Su objetivo es obtener acceso al sistema o aumentar sus privilegios en el sistema para adquirir información que debería estar protegida.
- **Malware.** Software que explota vulnerabilidades de los sistemas informáticos. Pueden ser programas de aplicación, de sistema o código kernel.
- Ambos pueden explotar vulnerabilidades del software (ej. buffer overflows) que les permiten ejecutar código que abre una puerta trasera (backdoor) en el sistema

2. PROTECCIÓN

Un sistema es una colección de procesos y objetos (hardware o software). Cada objeto tiene:

- Un **nombre** único que lo identifica
- Un conjunto de **operaciones** para acceder a él

Problema de protección: El SO debe asegurar que cada objeto (recurso) es accedido correctamente (sobre el objeto se solicita una operación válida) y, además, que el proceso tiene permiso para realizar la operación válida sobre el objeto.

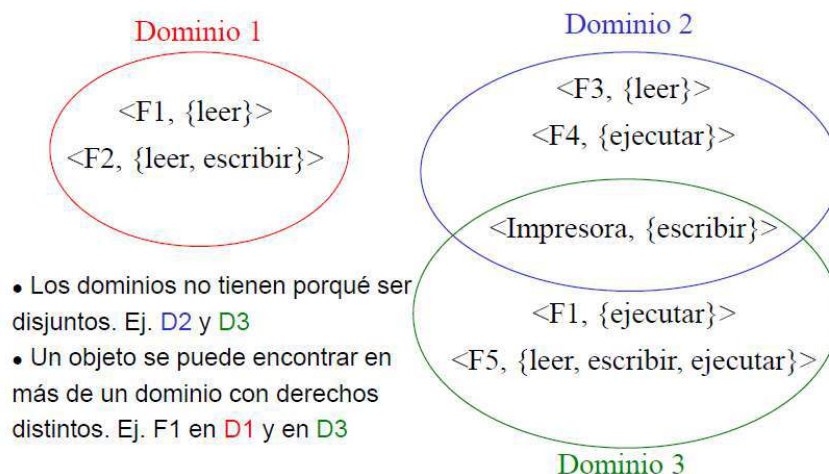
Dominio de protección: define un conjunto de objetos junto con sus derechos de acceso

{ <nombre_objeto>, <cjto_derechos_acceso> }

Derecho de acceso: permiso para realizar una operación sobre un objeto

Un proceso se ejecuta dentro de un dominio. La **asociación proceso-dominio** puede ser:

- **Estática:** el conjunto de objetos disponibles para un proceso es fijo durante la vida del proceso
- **Dinámica:** cambio de dominio durante la ejecución del proceso



2.1 MATRIZ DE ACCESO

Es el modelo que nos proporciona un mecanismo apropiado para especificar distintas políticas, y definir e implementar un control estricto (tanto para asociación estática como dinámica).

Objeto Dominio	F1	F2	F3	F4	F5	Impr.
D1	leer	leer escribir				
D2			leer	ejecutar		escribir
D3	ejecutar				leer escribir ejecutar	

Se pueden incluir los dominios como objetos para indicar el cambio entre dominios (conmutación)

Objeto Dominio	F1	F2	F3	F5	Impres.	D1	D2	D3
D1	leer propiet.	leer escribir					Conmutar	
D2			leer*		escr			
D3	eje.			leer escr. ejec.			Conmutar Control	

Igualmente, cada entrada de la matriz puede incluirse como un objeto más a proteger

2.1.1 IMPLEMENTACIÓN DE LA MATRIZ DE ACCESO

1. Tabla global: conjunto ordenado de triples <dominio, objeto, conjunto_derechos> Problemas:

- Matriz de gran tamaño no cabe en MP
- Difícil agrupar dominios u objetos

2. Listas de acceso para objetos (ACL): almacena la matriz por columnas. Asocia a cada objeto una lista ordenada de pares <dominio, conjunto_derechos>

- Ventaja: Fácil agrupar dominios
- Problema: Saber para un dominio dado, su conjunto de <objeto, conjunto_derechos>

3. Listas de capacidades para dominios: almacena la matriz por filas. Asocia a cada dominio una lista ordenada de pares <objeto, conjunto_derechos>

- Cada elemento se denomina capacidad
- Las listas de capacidades deben protegerse por el SO

4. Cerrojo-Llave:

- Cada objeto tiene una lista de patrones de bits únicos (cerrojos), y cada dominio otra lista (llaves)
- Un proceso en un dominio puede acceder a un objeto si tiene una llave que se corresponde con uno de los cerrojos del objeto

La mayoría de sistemas utilizan una combinación de ACL y listas de capacidades. Ej. S. de archivos de Unix

La ACL se corresponde con necesidades del usuario

La lista de capacidades se utiliza para localizar información del proceso.

• Ejemplos:

ACL (F1) = {<D1,leer>,
<D3,ejecutar>}

Lista de capacidades para D1:

	tipo	derechos	objeto
0	archivo	leer	nºi-nodo F1
1	archivo	leer/escr.	nºi-nodo F2
2	Capac.	conmutar	puntero a lista de capacidades de D2

2.2 REVOCACIÓN DE LOS DERECHOS

Es necesario revocar derechos a objetos compartidos en sistemas con un mecanismo dinámico de protección. Distintos aspectos de la revocación:

- Inmediata o retardada
- Selectiva o general: ¿afecta a un usuario o a todos?
- Parcial o total: ¿se eliminan todos los derechos?
- Temporal o permanente: ¿disponibles de nuevo?

Revocación con ACL: Es fácil e inmediata. Se busca en la lista los derechos a revocar y se eliminan

Revocación con LC: Difícil. Las capacidades están distribuidas por todo el sistema

Distintas implementaciones para la revocación en un sistema con capacidades:

- **Readquisición:** Periódicamente se borran todas las capacidades
- **Punteros:** El objeto tiene una lista de punteros para localizar sus capacidades. Para revocar sólo hay que seguir los punteros y modificar la capacidad. Costoso y general
- **Indirección:** Las capacidades no apuntan directamente a los objetos, sino indirectamente. Imposible revocación selectiva
- **Esquema de llaves:** Cada objeto tiene una llave maestra. Al crear la capacidad, se le asigna la llave. Revocar significaría cambiar la llave maestra

3. SEGURIDAD

Un sistema es seguro si sus recursos son utilizados y accedidos como se espera en todas las circunstancias

Problema: intentar saltarse la protección de recursos.

Requisitos de seguridad: confidencialidad, integridad y disponibilidad de la información

Las violaciones de seguridad pueden ser:

- **Accidentales:** fáciles de tratar
- **Intencionadas:**
 - lectura no autorizada a datos (viola la confidencialidad)
 - modificación no autorizada (viola la integridad)
 - destrucción (viola la disponibilidad)

Se pueden tomar medidas de seguridad a 3 niveles:

- **Físico:** el lugar donde se encuentra el sistema debe estar protegido físicamente
- **Humano:** autorización cuidadosa a los usuarios
- **Sistema Operativo:** identificación

Identificación:

- identifica programas, procesos y usuarios
- ¿cómo? combinación de los siguientes conjuntos de elementos:
 - posesión de un objeto (tarjeta, llave,...)
 - atributo del usuario (huella dactilar, firma,...)
 - conocimiento del usuario (password, username)

Problema de los password: mantenerlos en secreto

Los password pueden ser:

- generados por el sistema: difíciles de recordar
- seleccionados por el usuario: fáciles de adivinar

Formas comunes de conocer un password:

- conocer al usuario o tener información de él
- intentar todas las posibles combinaciones de letras, números y otros caracteres (crypt)

La lista de password de un sistema puede ser:

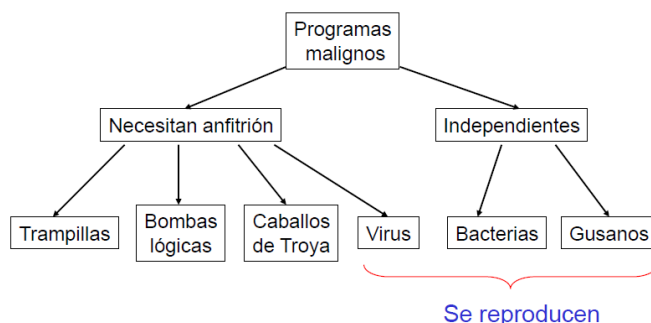
- **privada:** acceso denegado a los usuarios
- **pública:** se tiene acceso, pero su contenido está codificado

3.1 TÉCNICAS DE INTRUSIÓN

Objetivo de los intrusos: obtener el acceso a un sistema o aumentar el conjunto de privilegios

Hacker: talentos informáticos que entran al sistema por el placer de recopilar información que utilizan si la necesitan

Cracker: acceden, y presumen de ello, a los sistemas para los que no tienen autorización. Suelen cometer daños irreversibles o irreparables en el sistema.



Trampilla: punto de entrada secreto e indocumentado dentro de un programa. Utilizado para conceder accesos sin necesidad de identificación (*Juegos de Guerra*)

Bomba lógica: lógica introducida en un programa que, cuando se dan ciertas condiciones en el sistema, ejecuta alguna función no autorizada

Caballo de Troya: rutina secreta e indocumentada que se inserta dentro de un programa útil (ej. editor). La ejecución del programa origina la ejecución de la rutina

Virus: código incrustado en un programa que hace que se inserte una copia de sí mismo en uno o más programas. Además, puede realizar una función no deseada

Bacteria: programa que consume recursos del sistema mediante su reproducción

Gusano: programa que puede reproducirse y enviar copias de sí mismo a través de la red. Además, puede realizar alguna función no deseada

3.2 TÉCNICAS PARA MEJORAR LA SEGURIDAD

1. Monitorización de amenazas: comprobar si se intenta violar la seguridad del sistema buscar patrones de actividad sospechosos

2. Revisión de la información (auditoría): se registra el tiempo, usuario y tipo de todos los accesos a un objeto del sistema y si la operación tuvo o no éxito (archivo *log*). Una vez que la seguridad se ha violado, esta información sirve para:

- determinar cómo y cuándo ocurrió el problema y el daño producido
- recuperarse del ataque
- desarrollar una seguridad mejor

3. Explorar periódicamente distintos aspectos del sistema para evitar los ataques:

- claves cortas o fáciles
- programas no autorizados en directorios del sistema
- procesos no esperados de larga duración
- inapropiada protección de directorios/archivos
- cambios de tamaño en programas del sistema
- programas con acceso para cambiar SUID no autorizados

4. Proteger los datos mediante encriptación (sobre todo de la información transmitida por la red, los passwords, etc.)

3.3 CONTRAMEDIDAS

3.3.1 CONTRAMEDIDAS: IDS

Intrusion detection system (IDS). Servicio que monitoriza y analiza intentos de acceso no autorizados a recursos del sistema. Componentes de un IDS:

- **Sensores.** Recogen los datos proporcionados por otras partes del sistema (ej: archivos de log)
- **Analizadores.** La lógica que determina si ha habido un intento de intrusión. Pueden tomar medidas directamente o informar de la intrusión y las medidas tomadas para evitarla.
- **Interfaz de usuario.** Permite configurar al sistema de detección de intrusiones y comprobar los intentos de intrusión y las medidas que ha tomado para evitarla.

3.3.2 CONTRAMEDIDAS: AUTENTICACIÓN

Authentication. Proceso de verificar una identidad reclamada por o para una entidad del sistema. La autenticación de usuario es la base para el control de acceso al sistema y a sus recursos. Pasos de autenticación:

- **Identificación.** Presentar un identificador al subsistema de seguridad.
- **Verificación.** Presentar o generar información de autenticación que corrobore la vinculación entre la entidad y el identificador.
- **Mecanismos (métodos) de autenticación:** username and password, multi-factor authentication..., (muchos más).

Cuatro categorías para autenticar la identidad:

- Algo que conoce el usuario (password, PIN...)
- Algo que posee el usuario (token). El token puede ser tarjeta electrónica, tarjeta inteligente...
- Algo que el individuo es (biometría estática). Por ejemplo, huella dactilar, retina...
- Algo que hace el individuo (biometría dinámica). Por ejemplo, por patrón de voz.

3.3.3 CONTRAMEDIDAS: CONTROL DE ACCESO

El control de acceso implementa una política de seguridad que especifica:

- quién o qué (por ejemplo, en el caso de un proceso) puede tener acceso a cada recurso específico del sistema
- el tipo de acceso permitido en cada instancia.

Un mecanismo de control de acceso actúa como intermediario entre:

- un usuario (o un proceso que se ejecuta en nombre de un usuario)
- los recursos del sistema, como aplicaciones, sistemas operativos, firewalls, enrutadores, archivos y bases de datos.

Primero, el sistema autentica a un usuario que normalmente quiere tener acceso al sistema en su totalidad. A continuación, el control de acceso determina si el acceso requerido por el usuario se permite o no. El sistema de seguridad mantiene una base de datos de autorizaciones que especifica que tipo de acceso a que recursos está permitido para el usuario

3.3.4 CONTRAMEDIDAS: FIREWALLS

Los cortafuegos (firewalls) protegen un sistema local (o sistema informático de ordenadores interconectados) de amenazas de seguridad basadas en red. El firewall debe permitir el acceso desde el sistema informático local a redes de área amplia e internet. El firewall contiene elementos de seguridad para proteger información sensible dentro del sistema local. Los firewalls pueden ser:

- **Basados en red.** SW appliance, HW appliance o virtual appliance sobre hipervisor.
- **Basados en host.** Demonio o servicio proporcionado por el sistema operativo.

Principios de diseño de firewalls:

- Todo el tráfico de red desde el exterior hacia el sistema local, y viceversa, debe pasar a través del firewall.
- Solo se permitirá el paso del tráfico autorizado, según lo definido por la política de seguridad del sistema local.
- El propio cortafuegos es inmune a la penetración. Esto implica el uso de un sistema reforzado con un sistema operativo seguro